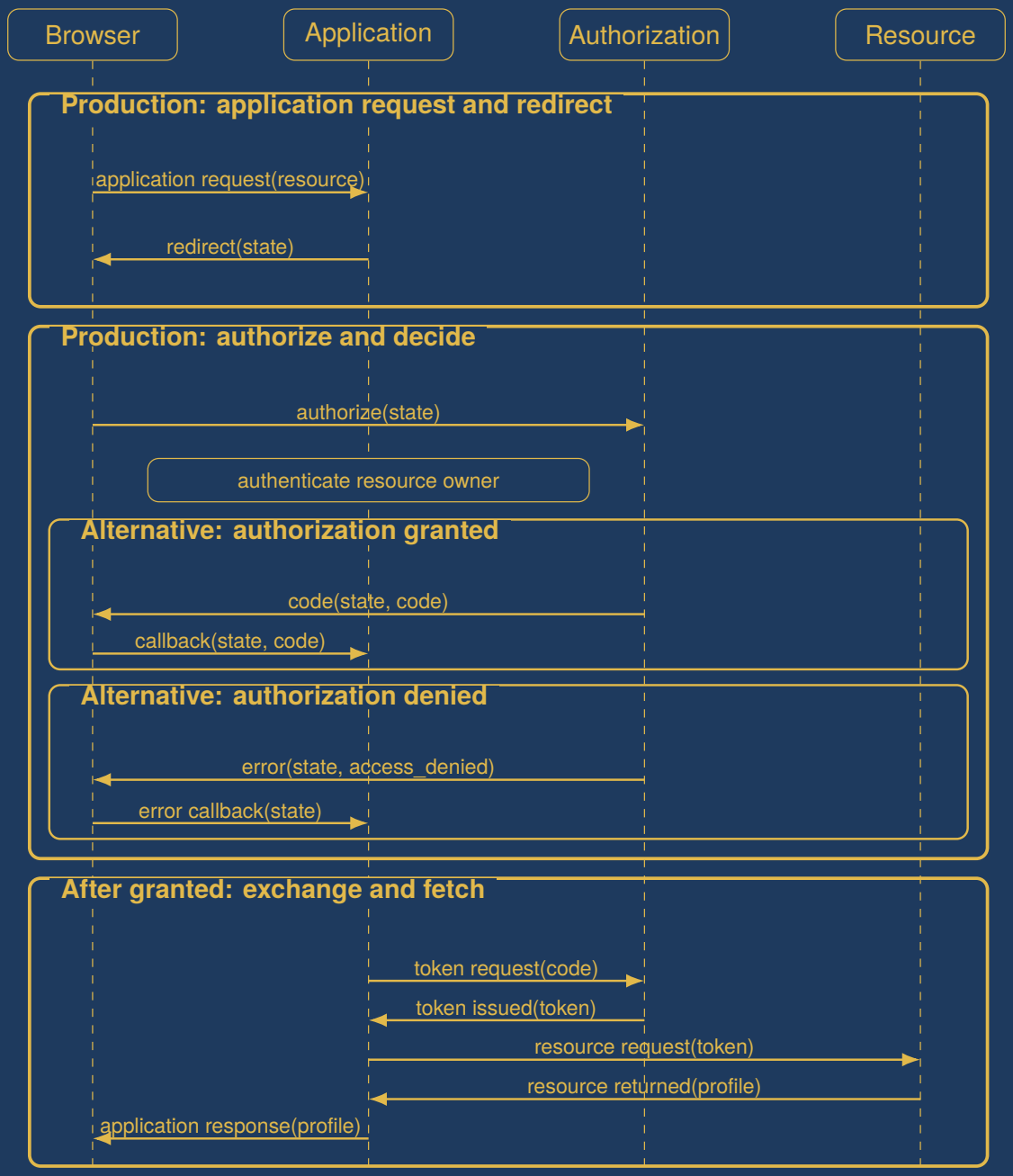
L∃∀NFM Software Engineering

Session types with bounded queues and unreliable messages



Rob Fielding and ChatGPT

Working draft—November 9, 2026

Preface

LeanFM is a way of building software by arguing about it first. You describe what a system should do in ordinary language; an LLM turns that into precise requirements; and LeanFM checks them and shows what they mean, as sequence diagrams, state machines, event traces and counterexamples. You push back, the requirements change, the pictures change with them, and only then is code written.

Its subject is the *behavior* of software at its boundaries: who sends what to whom, in what order, within what time and with what probability, and what must always or never happen. The record it keeps is a *multiparty session type*, one global description of a conversation from which each role's obligations are derived. Session types come from programming-language research, where they guarantee that no party sends a message the other cannot handle or waits for one that will never come. LeanFM uses them for messages between network nodes and calls between APIs alike, and adds what the classic theory leaves out: every queue has a declared size, and every channel declares how it fails. Messages are lost, duplicated, reordered and late; timers fire early; a retried request arrives twice. LeanFM checks the protocol together with those facts.

The precise part is Lean. Every requirement is a Lean definition: the roles, the messages down to their protobuf bytes, the legal orders, and what must always or never happen. Definitions can be checked, so diagrams are generated rather than drawn, and a contradiction (say A must reference B, B needs a C, and policy forbids that C) is found before anyone builds it. Requirements about values, such as an identifier that must refer to a real object, are predicates whose enforcement the implementation mapping describes; requirements over state no observer can see are flagged, so they can be restated over events that can be tested.

Three layers, reviewed in git

Work proceeds in three layers, each checked before the next: the *requirements* (observable behaviour and its generated views), the *implementation mapping* (how each requirement is realized and tested), and the code, written in one pass. Review is what you already do with git: read the diff and commit it. A passing check is evidence for that review, not the review itself.

Nothing lives only in the chat

A conversation with an LLM is temporary. Everything that matters is written into the project's one directory: the book, requirements, protobuf schema, implementation mapping and source. Delete the conversation or start a new session and work resumes from the files. This repository is such a project, and this book is part of its specification, read by the agents that work on it.

How to read this book

Chapter 1 sets out the method. Then come events and scenarios as recorded runs, a worked bakery example, global types as grammars, TFTP (where declaring the physics turns a perfect session type into a famous bug) and the authoring language. Chapter 8 covers the server and the MCP interface through which an agent consults LeanFM's logic. Later chapters apply temporal and modal logic to security and legal examples, and Chapter 19 defines how a new project is set up.

LeanFM is a working draft. The book records what exists alongside what is planned, and the evidence labels of Chapter 1 say what stands behind each claim. The source is public [7].

Contents

Preface i

I Session Types as a Software Engineering Method 1

1 Session Types as a Software Engineering Method 2
1.1 Session types in one page . . . 2
1.2 What the theory assumes, and what LeanFM declares . . . 5
1.3 Every boundary is a channel . . . 5
1.3.1 Masked and surfaced failures . . . 6
1.3.2 A hierarchy of interfaces . . . 6
1.3.3 The deployment is a tree . . . 7
1.3.4 Collapsed interaction diagrams . . . 10
1.3.5 Cut and un-cut: what HTTP allows . . . 11
1.4 Choice is always internal . . . 12
1.5 The requirement is the global type . . . 13
1.6 Comparison with TLA+ . . . 13
1.6.1 An executable queue model . . . 13
1.6.2 The cost of irrelevant state . . . 14
1.6.3 Proposed message-first notation . . . 14
1.6.4 Requirements at the message boundary . . . 15
1.7 Discovery first, consolidation before commitment . . . 16
1.8 Four ingredients . . . 16
1.9 A test for every abstraction . . . 16

2 Session Histories Made of Events 17
2.1 Compute everything from a fork/join event list . . . 18
2.1.1 Nested joins: choose a winner, then cancel the rest . . . 18
2.1.2 Durations from boundary events . . . 19
2.2 Derived state . . . 21
2.3 Time is more than a timestamp . . . 21
2.4 Estimating completion from an event stream . . . 22
2.5 Charts as declarative trace reductions . . . 22
2.6 Round trip through bytes . . . 23

3 Scenarios **24**
3.1 Event envelope and scenario grouping . . . 24
3.2 Derived outputs and provenance . . . 24
3.3 Example: minimal two-player game scenario . . . 24

4 The Bakery: Declared and Measured **26**
4.1 The morning, generated . . . 26
4.2 A sale: what can happen . . . 26
4.3 What we expected, and what the corpus shows . . . 27
4.4 The same questions under both overlays . . . 28
4.5 The deadline lesson: a mean hides the spread . . . 28
4.6 Generated runs . . . 29
4.7 Money and waste . . . 29

5 Global Types as Grammars **31**
5.1 The protocol syntax . . . 31
5.2 A BNF for composable interactions . . . 33
5.3 Parameters carry information . . . 36
5.4 Projection: from the global type to each role . . . 36
5.5 Declared physics . . . 37
5.6 Local types run on actors with bounded queues . . . 38

6 Declared Physics: TFTP and the Sorcerer's Apprentice **40**
6.1 The grammar . . . 40
6.2 The physics . . . 41
6.3 The question . . . 41
6.4 Liveness needs fairness . . . 42
6.5 What the classic session-type reading leaves out . . . 42

7 The Scenario Authoring Language **44**
7.1 Proposed scenario notation . . . 44
7.2 Worked MDP: a stochastic user and deterministic server . . . 45
7.3 Actor specifications and instances . . . 47
7.4 Non-functional requirements: rates and scalability . . . 47
7.4.1 The three USL regimes and capacity . . . 48
7.5 Outages, recovery, and replica populations . . . 51
7.5.1 Redundant, load-balanced runtimes and blast radius . . . 51
7.6 Infrastructure cost and demand-constrained supply . . . 54
7.7 Specified dwell time . . . 55
7.8 Composing requirements means intersecting constraints . . . 56

8 Arguing Requirements through the Lean Web Server **57**
8.1 The argument loop . . . 57
8.2 The server is a staging environment . . . 58
8.2.1 The code that creates the TCP listener . . . 58
8.3 MCP: the agent-facing entrance . . . 59
8.4 Consulting MCP about logic . . . 60

8.4.1 The tools . . . 60
8.4.2 Formulas as data . . . 62
8.4.3 Five verdicts . . . 62
8.4.4 Requirements without observable behavior . . . 62
8.4.5 The design loop . . . 63
8.4.6 Real answers . . . 63
8.5 A tour of the LeanFM chat . . . 66
8.5.1 Progress, cancellation and the Operational trace . . . 66
8.5.2 A private fault barrier and actionable errors . . . 66
8.5.3 Spending brakes . . . 67
8.5.4 Requirements properties and implementation provenance . . . 67
8.5.5 How a chat turn becomes a durable edit . . . 67
8.5.6 Example: ask to see MCP move . . . 68
8.5.7 What should appear in the response . . . 69
8.5.8 Images . . . 69
8.5.9 Screenshot-first documentation . . . 69
8.6 Local credentials and visible billing identity . . . 69
8.7 One project, one directory . . . 70
8.8 What the Lean artifact must settle . . . 71
8.9 Interrogate, characterize, synthesize, replay . . . 71
8.10 The protobuf artifact is part of the requirement . . . 71
8.11 A ladder of checks . . . 72
8.12 How to argue well . . . 72
8.13 Persistent application launch . . . 72

9 Using the LeanFM Chat UI 74
9.1 Open the service and unlock your account . . . 74
9.2 Choose the project before writing the prompt . . . 74
9.3 Send a useful prompt and understand the queue . . . 78
9.4 Read the result before retrying . . . 79
9.5 Keep the book and inspect the generated behavior . . . 79
9.6 Review the specification, then inspect the implementation . . . 81
9.7 Start, open, and stop the app . . . 84
9.8 Reproduce and review the screenshots . . . 86

10 Worked Example: OAuth Authorization Code Flow 87
10.1 Actors, values, and observable events . . . 87
10.2 Production 1: application request and redirect . . . 88
10.3 Production 2: authorize, then grant or deny . . . 89
10.4 Scenario 3: exchange the code and use the token . . . 90
10.5 Reading the scenarios together . . . 90
10.6 The parallel protobuf alphabet . . . 91
10.7 Generate one legal trace and its artifacts . . . 92
10.7.1 Envelope stream and concrete bytes . . . 94
10.7.2 Event graph with stored backpointers . . . 94
10.7.3 Actor projections . . . 95

10.7.4 Event order, queues, and memory . . . 95
10.7.5 Measurements, plots, and verdicts . . . 95
10.8 Interaction diagrams by scenario . . . 95
10.9 Requirements as observable constraints . . . 95
10.9.1 Who knows the password? . . . 97
10.9.2 Byte-level refinement without choosing a transport . . . 97
10.10 Probabilities and charts . . . 98
10.11 Residual state machine . . . 98

11 CTL over Generated Futures 100
11.1 Basic modal logic . . . 100
11.2 LTL: time along one path . . . 100
11.3 CTL: path modality combined with time . . . 101
11.3.1 Least upper bounds and greatest lower bounds . . . 101
11.3.2 Asymptotic cost . . . 102
11.4 Projecting to LTL-style F and G . . . 102
11.5 Persistence and recurrence . . . 103
11.6 Knowledge, obligation, and permission . . . 103
11.7 Possibility means preparedness . . . 104
11.8 Strong implication and the Chewbacca argument . . . 104
11.9 Weak until: AW and EW . . . 105
11.10 Past claims . . . 106
11.11 Fairness and time bounds . . . 106

12 A Legal Example: Lawful, Right, Known, Required 107
12.1 Three meanings of “legal” . . . 107
12.2 The Model Tenancy Deposit Act and a declared code . . . 107
12.3 Knowledge from the causal past . . . 108
12.4 Violation and reparation . . . 109
12.5 Lawful, right, and the four quadrants . . . 110
12.6 What CTL says, and what it does not . . . 110
12.7 Computing what is owed: Catala . . . 111
12.8 Status . . . 112

13 Generated Examples and Probability Overlays 113
13.1 The core says what can happen . . . 113
13.2 The overlay says how likely and how long . . . 113
13.3 Questions and their generated answers . . . 115
13.3.1 The usual run . . . 115
13.3.2 A precedent: the client gives up . . . 115
13.3.3 A counterexample: the late success . . . 116
13.4 The distribution behind the numbers . . . 116
13.5 What comes next . . . 117

14 Feature Completion as Positive and Negative Proof 118
14.1 The logical shape of a feature . . . 118
14.2 Stability review before security review . . . 119
14.3 Security as forbidden observable behavior . . . 119
14.4 The feature-completion ledger . . . 120
14.5 Worked example: the project Source view . . . 120
14.6 Worked example: chat auto-scroll . . . 121
14.7 When the result is not complete . . . 121

15 Lean as the Rendered Language 122
15.1 An unchecked exploratory fixture . . . 122
15.2 From specification to service . . . 126

16 Scope and Open Questions 127
16.1 Proof obligations . . . 127

II Security Protocol Exercises 129

17 Protocol Exercises: Roles, Attackers, and Knowledge 130
17.1 Roles compose by sharing events . . . 130
17.2 A transport-neutral byte algebra . . . 131
17.3 Example 1: an on-call handoff . . . 131
17.3.1 Three roles, written separately . . . 132
17.3.2 What composition catches . . . 132
17.4 Example 2: a sealed-bid auction . . . 133
17.4.1 Secrecy holds, and the auction is still broken . . . 134
17.4.2 The fix: bind the bidder into the commitment . . . 134
17.5 Knowledge and agreement as theorems . . . 134
17.5.1 Calculated knowledge and policy inside CTL . . . 135
17.6 Example 3: pairing a speaker by a code on its screen . . . 136
17.7 Where each modality attaches . . . 137
17.8 Needham–Schroeder, checked . . . 139
17.8.1 Two sessions: reflection . . . 140
17.8.2 A defender on the wire . . . 143
17.8.3 Reassembly . . . 143
17.9 What these exercises demand from the library . . . 144

III Machine Reference 145

18 The LeanFM Web-Server Protocol 146
18.1 One exchange across four representations . . . 146
18.1.1 Conservative default HTTP projection . . . 147
18.2 The conversion, field by field . . . 147
18.3 The running adapter . . . 148
18.4 The accepted HTTP projection . . . 148

18.5 Accepted message endpoints . . . 150
18.5.1 Canonical byte schema . . . 154
18.6 Complete LeanFM task grammar . . . 157
18.7 Authenticate an individual user . . . 157
18.8 Create an account using a user-supplied provider API key . . . 158
18.9 List, create, open, and delete owned projects . . . 158
18.10 Turn a fresh project conversation into checked requirements and implementation . . . 158
18.11 Select user-funded or platform-funded model access . . . 159
18.12 Conduct one durable project-scoped requirements turn . . . 159
18.13 Compact a project conversation into durable specifications before clearing it . . . 160
18.14 Sketch and promote multimodal requirement artifacts . . . 160
18.15 Inspect current, accepted, superseded, and stale artifacts . . . 160
18.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events 161
18.17 Search the public web, follow supplied HTTP URLs, and return clickable citations . . . 161
18.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision . 162
18.19 The grammar as an executable canvas . . . 162
18.20 What is checked, and what remains . . . 162

19 The LeanFM Project Bootstrap Contract 163
19.1 Required project inputs . . . 163
19.2 Conventional generated interface . . . 164
19.3 Required commands and order . . . 164
19.4 Three layers, reviewed in git . . . 164
19.5 Construction is not a runtime dependency . . . 165
19.6 Build, runtime, and publication dependency inventory . . . 165
19.7 Local and server-hosted references . . . 167

Selected Books and Further Reading 168

Bibliography 173

Index 174

Part I

Session Types as a Software Engineering Method

Chapter 1

Session Types as a Software Engineering Method

LeanFM is a session-type method for engineering software. A *multiparty session type* is a global description of a conversation: which roles take part, which messages pass between them, in what order, and where it branches [12]. Each role's *local type* is derived from it by projection, and a role built to its local type cannot send a message nobody expects, or wait forever for one nobody will send. That is the contract a team needs at every boundary where it does not want to read the other side's code.

Session types began as a type discipline for programming languages [10, 11]. LeanFM uses them one level up, as the requirements: the global type is what stakeholders argue about, the local types are what each implementer owes, and recorded production events are checked against both.

LeanFM departs from the textbook theory in one deliberate way: *its channels have physics*. Standard asynchronous session types assume every message arrives exactly once, in order, through a queue that never fills. Real channels are bounded; UDP drops and reorders; a timed-out HTTP call is retried and seen twice; a Kafka consumer replays a batch after a rebalance; a timer cannot tell a slow answer from a lost one. LeanFM makes each channel's capacity and failure behaviour part of the requirement and checks the protocol together with it. Chapter 6 shows the consequence: a grammar that is a perfect session type, and an implementation with a famous bug that appears only once the physics is declared. Messages between network nodes and calls between APIs are handled the same way; they are channels that differ only in their physics.

Evidence labels used throughout: proposed notation (uncompiled); executable code (builds or runs, not proof); finite-model check (exhaustive only within declared bounds); test (named finite cases); generated analysis (named inputs and reducer); Lean proof (kernel-checked under stated assumptions); future interface (planned, unavailable). Labels scope evidence; several may apply to one artifact.

1.1 Session types in one page

A global type is written once, from nobody's point of view. This purchase between three roles is in the Scribble style of Yoshida et al. [25], written in LeanFM's syntax. It is an ordinary Lean module, taken from `SessionExample.lean` and checked every time the file is elaborated (Chapter 5 gives the whole syntax).

```
protocol Purchase (Client, Shop, Bank) {
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q where p.amount = q.price;
       Paid  from Bank to Shop cause p;
       Shipped from Shop to Client;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

Each line is one message: a label, a sender, a receiver, and optionally the earlier message that caused it. A label names a Lean structure, so the payload is its fields, and `where p.amount = q.price` is type-checked Lean. `choice` says exactly one branch happens; the shop decides, because it starts both. Projection keeps the lines a role sends or receives. The shop's:

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

The theory asks two questions of a global type, and so does LeanFM.

One chooser per choice.
Every branch must be started by the same role, which is the one that decides. If Client could start one branch while Shop starts another, both could act at once and leave the protocol. This is the realizability (non-local choice) condition.

Everyone who must know, learns.
A role whose later behaviour differs between branches must be told which branch was taken. In the `soldOut` branch of `Purchase` the bank is never told anything.

Classic projection rejects `Purchase` for that. LeanFM accepts it, for a reason that comes from how servers are built: a role whose part in the session has not started owes nothing to a branch that never contacts it. The bank's part starts when `pay` arrives, as a thread forked for that session; in the `soldOut` branch that thread never exists, so checking it on its own is exact. The bank's projection shows the empty branch:

Client

```
send Order to Shop;
offer {
  receive Quote from Shop;
  send Pay to Bank;
  receive Shipped from Shop;
} or {
  receive SoldOut from Shop;
}
```

Shop

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

Bank

```
offer {
  receive Pay from Client;
  send Paid to Shop;
} or {
}
```

The rule still bites a role that has already taken part. If the client first placed a hold with the bank and the shop then sold out, the bank would be left holding money with nobody to tell it to release:

```
protocol PurchaseWithHold (Client, Shop, Bank) {
  h: Hold from Client to Bank;
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

LeanFM reports:

```
Bank cannot tell which branch Shop chose
```

When the global type passes, roles that follow their local types get *communication safety* (no message arrives that the receiver cannot handle), *session fidelity* (the run follows the global type) and *progress* (no role waits forever). Each holds *under the theory's assumptions about channels*, which are exactly where real systems fail.

1.2 What the theory assumes, and what LeanFM declares

Concern	Standard asynchronous session types	LeanFM
Queue size	unbounded	declared capacity, and what happens when full: block, reject, drop newest, or drop oldest
Delivery	exactly once, in order per pair of roles	declared per channel: ordering, sharding, loss and duplication
Time	absent	timers that may fire early; deadlines on responses
Failure	absent in the core theory; extensions add crash-stop roles [2]	declared: lost messages, restarted roles, expired sessions
Choice	one chooser per branch	the same, and the chooser may act only on its own history
Properties	safety, fidelity, progress by construction	those, plus CTL questions over the product of roles and channels, with fairness stated as a requirement
Evidence	a program that type-checks	also conformance of recorded production event streams

Bounded queues are also what makes exhaustive checking possible. Roles exchanging messages through unbounded FIFO queues are communicating finite-state machines, for which most verification questions are undecidable [4]. Session types avoid this by restricting the protocol's shape [6]; LeanFM instead bounds the queues. Finite roles and finite queues give a finite transition system, so every CTL question is answered exhaustively, with a witness or a counterexample.

1.3 Every boundary is a channel

A channel is any path a message can take between two roles. It declares its physics along six axes (**proposed notation**; the TFTP chapter builds its UDP channels by hand today).

Axis	Values	What it adds to the checked product
capacity	a positive number of messages	a send to a full channel takes the *when full* policy
when full	block, reject, drop newest, drop oldest	block: the sender waits; reject: the sender receives an observable refusal (an HTTP 503 or 429); the drops lose a message silently
ordering	FIFO, per key, none	which queued message may be delivered next
sharding	none, or a key field and a partition count	the channel becomes several queues; ordering holds within one partition only
loss	never, may drop	a transition that removes a queued message
duplication	never, may duplicate	a transition that delivers a message and leaves a copy queued

Most channels are one of a few familiar kinds, named as presets. A declaration starts from one and overrides what differs, as in `udp (capacity := 2)`.

Preset	Typical use	Physics
`reliable`	internal APIs, in-process calls, Go channels	FIFO, never lost, never duplicated, blocks when full
`http`	calls between services	each request and its response may be lost, so the caller has a deadline; a retried request may arrive twice; concurrent requests have no order; a busy server rejects
`udp`	datagrams	no order, may drop, may duplicate, drops the newest message when full
`stream`	TCP, TLS, websockets	FIFO and never lost while open; it may close after any message, and both sides observe the close as the end of the session
`kafka`	event topics	sharded by a key field; FIFO within a partition; never lost within retention; delivered at least once, so may duplicate; the producer blocks when full

Physics never appears inside the global type. The type is written as if every channel were `reliable` (the default for an undeclared channel), with the declarations beside it, one line per channel. A lossy channel costs lines in the type only where the protocol must answer it: a timer and a resend for `udp`, a deadline and a retry for `http`. Declaring a channel `udp` without those lines makes the protocol wrong, and the checker says so. Most internal boundaries are `reliable`; the few that cross a network are where the physics, and the bugs, live.

`tls` is a modifier, not a preset: `tls http` or `tls stream` adds an authenticated peer identity to every message and changes nothing else.

A sharded channel is several queues; a message goes to the partition chosen by hashing its key field, and order holds only between messages with the same key. A session keeps its order across a sharded channel only if all its messages carry the same key, such as the session identifier. LeanFM can check this from the declarations alone: a global type that relies on two messages arriving in order, over a channel sharded by a field that differs between them, is rejected before any state is explored.

1.3.1 Masked and surfaced failures

A lower layer can hide a failure by retrying, and the failure becomes latency. TCP retransmits, so a protocol over a `stream` sees no loss, only delay, until the connection closes. UDP hides nothing, so a protocol over it handles loss, reordering and duplicates itself, as TFTP does. Kafka hides loss inside its retention window and surfaces duplicates instead. Every interface kind says which conditions it *surfaces*, which the protocol must handle, and which it *masks* as latency, which the protocol's deadlines must allow for.

1.3.2 A hierarchy of interfaces

An actor has interfaces, and each interface has a kind. A kind inherits every condition its parent surfaces and adds or removes a few. The table is a **generated analysis** from `TopologyExample.lean`:

kind	surfaces	hidden as latency	notes
`call`	–	–	synchronous; one process
`queue`	–	–	bounded, blocks when full; one machine
`stream`	–	loss and reorder become latency	TCP; a close ends the session
↪ `http`	+timeout, +duplicate	–	request and response; a retry may arrive twice
↪ `api`	+reject	–	a busy server answers 429 or 503
↪ `ingressHttp`	+hostile, +readable	–	from outside; anyone can send or read
↪ `ingressHttps`	−readable	–	TLS: peer identity observable, bytes private
`datagram`	+loss, +reorder, +duplicate	–	UDP; nothing is hidden
`topic`	+duplicate, +per-key order	loss becomes latency within retention	Kafka; at least once, ordered per key

`ingressHttps` is `ingressHttp` with TLS: it removes `readable`, because a hostile network can no longer read the bytes, and keeps `hostile`, because anyone can still send them.

1.3.3 The deployment is a tree

Where the two ends of a channel run adds the conditions of the network between them and decides whether they may talk at all. LeanFM describes a deployment as one tree: islands that may be partitioned from each other, datacenters and machines inside them, groups for replicated and sharded roles, actors, and under each actor its interfaces grouped by *scope*. A node's name is its path (**proposed notation**; `TopologyExample.lean` holds the same tree as Lean values):

```
world {
  outside { browser }
  us {
    datacenter dc1 {
      machine m1 { alice { ingress { https }  internal { api } } }
      replicated x3 by raft { ledger { internal { api } } }
    }
  }
  eu {
    datacenter dc1 {
      machine m1 { bob { ingress { http }  peering { api } } }
      machine m2 { charles { ingress { https }  internal { api } }
                   audit   { local { syslog } } }
    }
  }
  sharded by account across { us, eu } { accounts { peering { topic } } }
}
```

The implicit name of an interface keeps the island, the actor, the scope and the interface, and leaves out the containers between them: `us.alice.ingress.https`, `eu.audit.local.syslog`. Moving `alice` to another machine in `us` does not rename her.

Scopes constrain who may talk. A scope is a promise about callers:

scope	may be called from
`ingress`	anyone, including outside
`peering`	any island, not outside
`internal`	the same island
`local`	the same machine

Collapsing the tree to its top level gives the deployment's permission structure on one page. Each cell lists the scopes in the column's subtree that some actor in the row's subtree may reach, computed from the actors and interfaces actually in the tree:

from \ to	`outside`	`us`	`eu`	`us+eu`
`outside`	–	ingress	ingress	–
`us`	–	ingress, internal	ingress, peering	peering
`eu`	–	ingress	ingress, peering, internal, local	peering
`us+eu`	–	ingress	ingress, peering	–

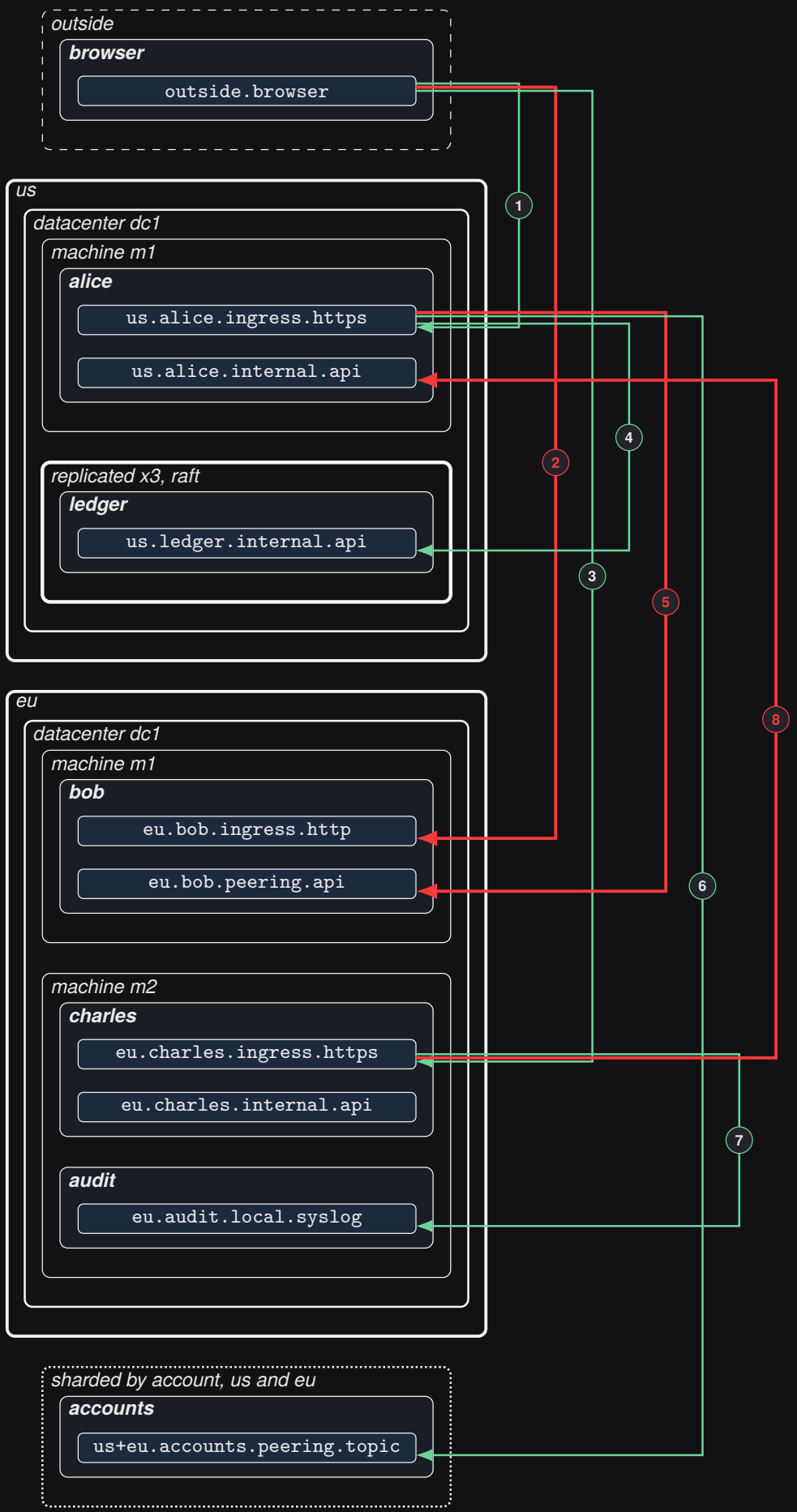
Placement adds the network. A channel between islands may be `partitioned`: for a while, nothing is delivered either way. An end outside every island makes the channel `hostile`. A channel into a replicated group may be rejected (no leader, no quorum), with the consensus inside hidden. A channel into a sharded group keeps order only per key.

The generated graph places every interface in the tree and draws each channel from its sending actor to the receiving interface. A channel is red when its scope forbids it, or when it surfaces a condition its protocol does not handle.

	from	to	conditions surfaced	verdict
1	browser	`us.alice.ingress.https`	timeout, duplicate, hostile	handled
2	browser	`eu.bob.ingress.http`	timeout, duplicate, hostile, readable	unhandled: readable
3	browser	`eu.charles.ingress.https`	timeout, duplicate, hostile	handled
4	alice	`us.ledger.internal.api`	timeout, duplicate, reject	handled
5	alice	`eu.bob.peering.api`	timeout, duplicate, reject, partition	unhandled: partition
6	alice	`us+eu.accounts.peering.topic`	duplicate, per-key order, partition	handled
7	charles	`eu.audit.local.syslog`	loss, reorder, duplicate	handled
8	charles	`us.alice.internal.api`	timeout, duplicate, reject, partition	not reachable: internal to us

Table 1.1: Each channel's reachability and the conditions it surfaces, against what its protocol handles.

Three channels are findings, one of each kind. Channel 8, from `charles` in `eu` to `us.alice.internal.api`, may not exist: `internal` means the same island, so either `alice` exports that API under `peering` or `charles` goes through her ingress. Channel 5, from `alice` to `eu.bob.peering.api`, crosses islands and may be partitioned, which its protocol does not answer. Channel 2 is plain `http` from outside, readable by a hostile network: either the protocol says nothing there matters by handling `readable`, or the interface should be `https`.


outside
browser
outside.browser
us
datacenter dc1
machine m1
alice
us.alice.ingress.https
us.alice.internal.api
replicated x3, raft
ledger
us.ledger.internal.api
eu
datacenter dc1
machine m1
bob
eu.bob.ingress.http
eu.bob.peering.api
machine m2
charles
eu.charles.ingress.https
eu.charles.internal.api
audit
eu.audit.local.syslog
sharded by account, us and eu
accounts
us+eu.accounts.peering.topic
1
2
3
4
5
6
7
8

1.3.4 Collapsed interaction diagrams

The tree also gives every interaction diagram a zoom level. Collapsing a subtree into one lifeline hides every message with both ends inside it, which is how a requirement hides implementation. A global type too large to read whole is read from the top of the tree, opening one node at a time.

The `cause` pointers decide how the remaining arrows are drawn. A message from X to Y is the *return* of the nearest unanswered request from Y to X in its cause chain, however many hidden messages lie between; otherwise it is a request. Requests are solid, returns dashed, and the callee's lifeline carries a bar from each call to its return. One order, at the level of actors:

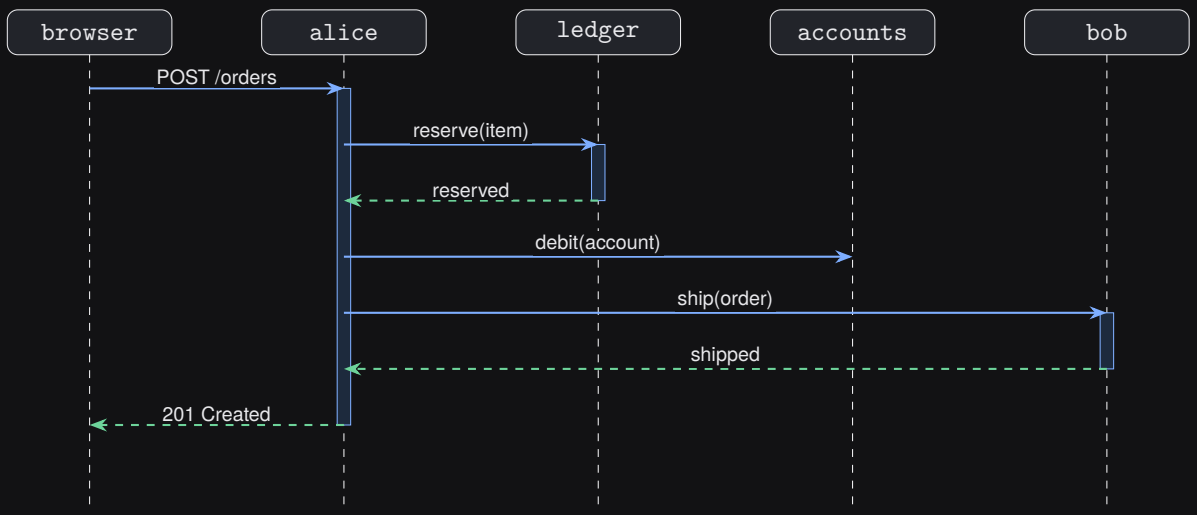


Figure 1.2: One order at the level of actors.

and collapsed to the top level. The reservation vanishes, because both ends are in `us`; `201 Created` is still paired with `POST /orders`, through a cause chain that passes through the hidden reservation and through `eu`. What remains is what one island promises another:

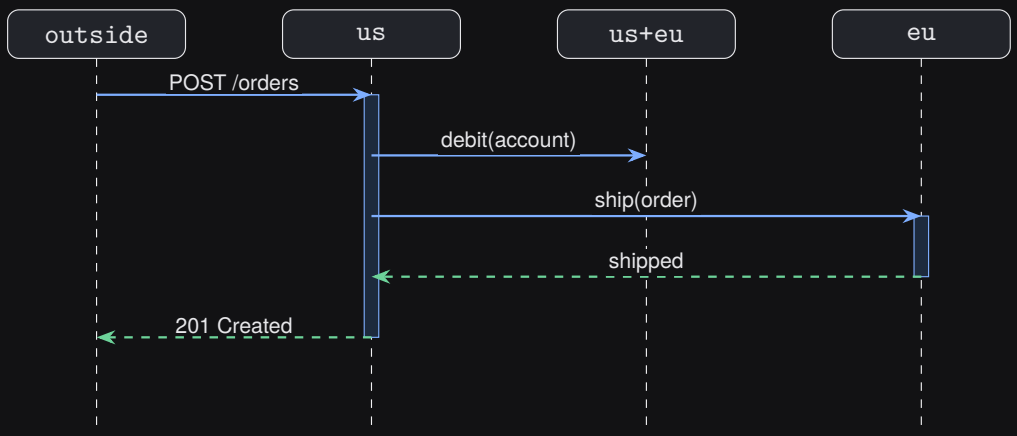


Figure 1.3: The same order with the tree collapsed to islands: five of its seven messages cross a boundary at this level.

A local type is a production. Projecting the global type onto one node of the tree gives that node's local type, an ordinary production of the grammar, cut off where the node's tasks leave its boundary: a call to another node is one terminal, `call eu: ship(order) returns shipped`, and how `eu` produces `shipped` is `eu`'s own production. Folding the crossings above, with the tree cut at its top level:

```
outside ::=
  call us: POST /orders returns 201 Created

us ::=
  on outside: POST /orders {
    send us+eu: debit(account)
    call eu: ship(order) returns shipped
  } return 201 Created

us+eu ::=
  on us: debit(account)

eu ::=
  on us: ship(order) return shipped
```

Each production can be read as a top-level protocol, so requirements stack: `us` is specified against the promises of `eu` and `us+eu`, not their insides. Opening a node replaces its production by its children's, whose crossings must fold back into the parent's. A team can agree on the islands view first and refine each island later; a refinement is acceptable when collapsing it again gives back the agreed view.

Placement is an implementation decision, so the tree's containers belong in `Implementation.lean`; the scopes and what each channel's protocol handles are requirements. Moving `bob` into `us` would remove the partition from channel 5; moving `ledger` out of `us` would make channel 4 unreachable. Either is a checked change, acceptable only if no channel turns red after `lake env lean -run TopologyExample.lean`.

With channel declarations, inter-API design is network-protocol design. A browser call that passes a reverse proxy to a service is a three-role session:

```
q: tls http GET /users/{id}       from Browser to Proxy;
r: http GET /users/{id}           from Proxy to Users   cause q
     where header USER_DN = peer.dn(q);
s: http 200 {user}                from Users to Proxy   cause r;
   http 200 {user}                from Proxy to Browser cause s;
```

The requirement names only what crosses a boundary. Whether `Users` answers from a cache, a database or a replica is left free by not being written down. Negative requirements follow the same rule: "a request without a client certificate never reaches `Users`" is a statement about observable messages on two channels, checked and tested as one.

1.3.5 Cut and un-cut: what HTTP allows

A global type may say that a reply comes from someone other than the role that was asked:

```
fq: X() from A to B;
fr: Y() from C to A   cause fq;
```

Over HTTP this cannot happen: a response travels back on its request's connection. The `http` channel kind therefore adds a rule: every message is either a request to an interface of its receiver, or the response to a pending request its receiver sent to its sender. A protocol that breaks the rule must be *un-cut*, as a proof with a cut is expanded into one without: the missing calls and returns are written out.

```
fq:  X()  from A to B;
fbc: X'() from B to C   cause fq;
fcb: Y'() from C to B   cause fbc;
fr:  Y()  from B to A   cause fcb;
```

Collapse `B` and `C` into one lifeline, as in figure 1.3, and both forms become an ordinary call and return between `A` and `BC`. A cut is a statement made at a coarser zoom level than its names suggest, sound exactly when the roles it jumps between sit inside one node that the caller sees as a single HTTP server. `Purchase` is cut twice: `paid` answers the client's payment to the shop, and `shipped` answers a request the client never made. Un-cut for HTTP, with the client as the only role that calls the shop:

```
o: http POST /orders {item}          from Client to Shop;
choice {
  q: http 200 {quote}                from Shop to Client  cause o;
  p: http POST /payments {quote}     from Client to Bank  cause q;
  r: http 201 {receipt}              from Bank to Client  cause p;
  c: http POST /orders/{id} {receipt} from Client to Shop cause r;
  v: http POST /receipts/verify      from Shop to Bank    cause c;
  w: http 200 {paid}                 from Bank to Shop    cause v;
     http 200 {shipped}              from Shop to Client  cause w;
} or {
     http 409 {soldOut}              from Shop to Client  cause o;
}
```

Every `cause` now names a message its sender received, and every response goes back to the role that asked. The other un-cut is a callback, which needs the shop to expose an interface the bank's island may reach (table 1.1). Choosing is a design decision; LeanFM's job is to reject the cut form on an `http` channel and show both (a **future interface**; the rule above is its definition).

1.4 Choice is always internal

Session-type theory distinguishes *internal* choice (a role selects which label to send) from *external* choice (a role offers to receive any of several). Among actors reading their own queues only the first is a decision: an external choice is the queue delivering what another role decided. So every choice is made by one role, from what it can know: the messages it has sent and received.

Every recorded event carries `priorIds`, the earlier events it follows, and `causeId`, the one event that made it happen (Chapter 2). Realizability is then a rule about pointers: *a role's event may cause-point only to events that role sent or received.* A protocol in which a role reacts to something it never saw cannot be implemented, and is rejected (a **future interface**; the rule defines the validation).

1.5 The requirement is the global type

The requirement is the set of permitted, visible histories: who communicated with whom, which message, what it carried, and which earlier messages it follows. Message bytes and persistent formats become compatibility boundaries once another process, customer or stored data set depends on them. Threads, actors, caches and retry machinery inside a role are realization choices unless they change the observable conversation; modelling them couples requirements to harmless refactoring and multiplies the state space. Channel capacity and failure behaviour, by contrast, *are* observable, because they decide which histories can occur.

1.6 Comparison with TLA+

TLA+ is the specification language most engineers meet first. LeanFM neither extends nor replaces it. A TLA+ specification commonly has variables, an initial-state predicate, a next-state relation, and temporal formulas over the resulting behaviours [15]; a message queue is a sequence-valued variable that actions append to and remove from.

1.6.1 An executable queue model

This module sends an order to Bob’s inbox, then lets Bob consume it and change his control state:

```
---------------------------- MODULE OrderQueue ----------------------------
EXTENDS Integers, Sequences, FiniteSets, TLC
CONSTANTS Clients, Servers, Orders, QueueBound, CacheBits, ClockBound
Actors == Clients \cup Servers
Messages == [kind : {"placeOrder", "placedOrder"},
             order : Orders, replyTo : Clients]
NoMessage == [kind |-> "none", order |-> "none", replyTo |-> "none"]
VARIABLES inbox, pc, current, sent, placed, received, scratch, tick
vars == <<inbox, pc, current, sent, placed, received, scratch, tick>>

Init == /\ inbox = [a \in Actors |-> <<>>]
        /\ pc = [b \in Servers |-> "idle"]
        /\ current = [b \in Servers |-> NoMessage]
        /\ sent = {} /\ placed = {} /\ received = {}
        /\ scratch = [i \in 1..CacheBits |-> FALSE]
        /\ tick = 0

Send(a, b, o) ==
  /\ o \notin sent /\ Len(inbox[b]) < QueueBound
  /\ inbox' = [inbox EXCEPT ![b] = Append(@,
       [kind |-> "placeOrder", order |-> o, replyTo |-> a])]
  /\ sent' = sent \cup {o}
  /\ UNCHANGED <<pc, current, placed, received, scratch, tick>>

Receive(b) ==
  /\ pc[b] = "idle" /\ Len(inbox[b]) > 0
  /\ current' = [current EXCEPT ![b] = Head(inbox[b])]
  /\ inbox' = [inbox EXCEPT ![b] = Tail(@)]
  /\ pc' = [pc EXCEPT ![b] = "busy"]
  /\ UNCHANGED <<sent, placed, received, scratch, tick>>
```

The behaviour is defined later in the same module:

```
Next == (\E a \in Clients, b \in Servers, o \in Orders: Send(a,b,o))
     \/ (\E b \in Servers: Receive(b) \/ Finish(b))
     \/ (\E a \in Clients: ReadReply(a)) \/ CacheStep \/ ClockStep
Spec == Init /\ [][Next]_vars
```

`Next` is a relation between the current variables and their primed next values: one step is a `Send`, `Receive`, `Finish`, `ReadReply`, cache step or clock step, and an action whose guards are false is disabled. `Spec == Init /\ [][Next]_vars` reads: the first state satisfies `Init`, and every later step satisfies `Next` or leaves `vars` unchanged. `inbox'` is the next value of `inbox`; `Append`, `Head` and `Tail` add, select and remove a message. Alice's send updates Bob's inbox, not his control state; Bob's receive does that. The full module commits the order, replies, and checks that no reply precedes placement.

1.6.2 The cost of irrelevant state

The module also has cache-bit toggles and a clock that cannot affect an order. TLC distinguishes their values anyway:

Model	Cache bits	Clock bound	Distinct states
Baseline	0	0	26
Cache4	4	0	416
Cache8	8	0	6,656
Clock7	0	7	208

The counts come from TLC 1.7.4 via `make tla-check`, with provenance in `book/generated/tla-state-space.j`
With S protocol states, b independent cache bits and a counter from zero to k, the model has

$$S\,2^b(k+1)$$

states: each irrelevant bit doubles it, and an unbounded clock (`ClockBound = -1`) makes it infinite, so exhaustive search cannot finish. This is checker state, not application memory. TLA+ supports abstraction, hiding and refinement; the lesson is to omit irrelevant state, not that the language forces it in.

1.6.3 Proposed message-first notation

This is invented TLA+-inspired syntax, neither standard TLA+ nor a compiled LeanFM feature:

```
// Proposed TLA+-inspired notation. Not standard TLA+, not compiled by LeanFM.
message placeOrder(orderId: OrderId, replyTo: ClientId)
message placedOrder(orderId: OrderId)

actor class Client {
  input placedOrder
  output placeOrder
}
actor class Shop {
  input placeOrder
```

```
  output placedOrder
  inbox FIFO(capacity = 2)
  on placeOrder(orderId, replyTo) {
    require commitRecorded(orderId)
    self:replyTo ! placedOrder(orderId)
  }
}

alice = new Client()
bob = new Shop()
alice:bob ! placeOrder(o1, replyTo = alice)
bob ? placeOrder(o1, replyTo = alice)
// Shop handler records the commit and sends placedOrder.
alice ? placedOrder(o1)

web = instances(WebServer, count = 3, route = roundRobin)
documents = shards(DocumentStore, count = 8,
                   key = documentPartition, replicas = 3,
                   writeQuorum = 2)
```

`alice:bob ! placeOrder` reads "Alice sends placeOrder to Bob" and `bob ? placeOrder` "Bob receives placeOrder". Inboxes are first-class actor-owned queues, not arbitrary private variables. Instances of an actor class have their own identities, queues and state; shards partition ownership by key, and replicas with a quorum protect each shard. Such syntax still needs capacity, routing, failure and scheduling rules (Chapter 5); it does not remove their state space.

1.6.4 Requirements at the message boundary

LeanFM starts from the conversation: a global type over decoded production events, whose order is classified as conforming or a counterexample without pretending to observe private variables. A property that depends on hidden state needs an observable event or a justified abstraction; otherwise its status is indeterminate. This is a workflow choice, not a claim that TLA+ cannot express queues: our first TLA+-style attempt had to model internal state before an ordinary observed scenario could be recorded.

Question	TLA+-style state specification	LeanFM session specification
Primary behavior	sequence of states	a global type, and its runs as partial-order graphs of events
Queue representation	usually an explicit variable	a channel with declared capacity and failure behaviour
Conformance input	modeled states or action correspondence	decoded production event stream
Hidden variables	available when declared	omitted unless they affect an observable contract
Generation	behaviors satisfying the state relation	runs generated by the global type and its channel physics
Probabilities	require an additional probabilistic interpretation or tool	declared and measured overlays over the same core
Main risk	state model may drift from instrumentation	event model may omit a hidden cause needed for a property

`Requirements.lean` defines observable behavior; `Implementation.lean` maps it to generation and runtime choices; existing source does not redefine requirements (Chapter 19).

1.7 Discovery first, consolidation before commitment

During discovery the book may hold alternatives that are not accepted. Before a specification is committed, and again before implementation, classify every normative idea as a requirement, an implementation decision, a deferred item, or material to remove. Replace prose that mainly enumerates roles, messages, states, ordering or quantities with a generated view and a short interpretation; each view keeps its prompt, question, event source, reducer, unit and boundary. Only committed, checked artifacts are binding.

1.8 Four ingredients

1. Session types give the backbone: one global description of a conversation, projected to each role's obligations, with well-formedness conditions that rule out unimplementable choices [11, 12, 25].
2. Language theory gives compact generative descriptions of legal histories: a global type is a grammar whose terminals are messages. Dana Richards and Henry Hamburger, the author's college professors, are cited in Richards and Hamburger [21] as where the author learned these methods, not as their originators; for symbolization, validity, proof, quantification and modal reasoning we also draw on Gensler [8].
3. Protocol analysis contributes explicit participants, adversarial choices, trace semantics and tool-supported checking. An attacker is one more role, or a channel an adversary controls. The CSP treatment of Ryan et al. [22], including Steve Schneider, is cited as where the author learned it.
4. Temporal logic turns expectations into queries over the product of roles and channels. CTL is central because it distinguishes claims about some possible future from claims about every one [5].

1.9 A test for every abstraction

> Can a developer state a conversation once, with the physics of its channels, compose it with other conversations, check its consequences, and keep it stable while the implementation of each role changes?

An abstraction that does not improve that workflow does not belong in the core library.

Chapter 2

Session Histories Made of Events

A global type says which conversations are legal. An *event* is one message of one actual session, recorded as it happened; checking an implementation means checking its list of events against the global type and its physics. A small common envelope makes histories comparable across protocols:

```
structure Event (Actor Session Task Kind Payload : Type) where
  timeAt  : ClockTimestamp
  id      : String
  priorIds : List String
  causeId : Option String
  session : Session
  task    : Task
  from    : Actor
  to      : Actor
  kind    : Kind
  payload : Payload

abbrev Trace (Actor Session Task Kind Payload : Type) :=
  List (Event Actor Session Task Kind Payload)
```

`id` identifies a visible fact. `priorIds` lists immediate predecessors, and `causeId` optionally names the one event that directly caused this one (MCP's `check_trace` spells them `prevIds` and `causeIds`, protobuf `prior_ids` and `cause_id`). `from` and `to` are the roles; `session` names the instance of the global type, and `(session, task)` correlates independently progressing conversations; `kind` is a stable discriminator; the payload carries the parameters that flow between productions. `timeAt` comes from a shared monotonic clock whose differences measure elapsed time; its origin does not matter.

Predecessor links form a directed acyclic graph. A fork is several events naming the same predecessor; a join is one event naming every branch it depends on. An end event may also name its start, which makes elapsed time `end.timeAt - start.timeAt` even when attempts with the same `(session, task)` overlap; such an edge may be redundant for ordering but is useful correlation. A predecessor edge means "ordered after", not "caused by".

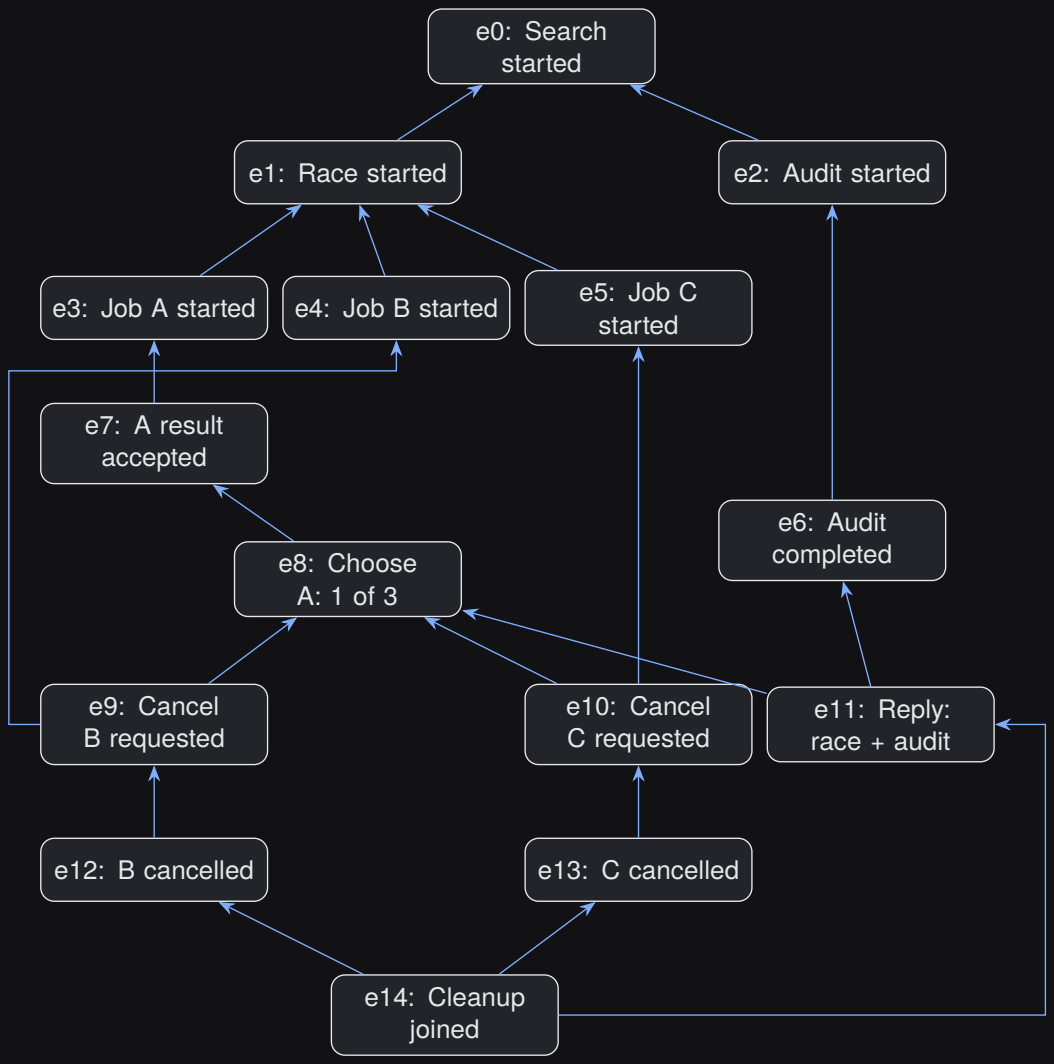
`causeId` is the causal pointer, and it adds or removes no interleavings. A task start may cause messages, which may cause nested task starts, and a terminal points to the event whose work it finishes. The reverse relation gives causal subtrees, which a renderer draws as nested labeled boxes without pretending that concurrent children are ordered. The pointer also makes choices checkable: a role decides only from its own history, so `causeId` must name an event its sender sent or received

(Chapter 1). A response's `causeId` is its request's `id`, which stays unambiguous with many requests in flight because it follows identity, not adjacency.

An m-of-n join also records m, n and the m distinct branch-terminal IDs, members of `priorIds`, that satisfied it; two predecessors alone cannot distinguish two-of-three from an ordinary two-branch join.

2.1 Compute everything from a fork/join event list

2.1.1 Nested joins: choose a winner, then cancel the rest



Arrows show predecessor order with redundant transitive edges removed; full backpointers remain in the event list. The event list and provenance are generated from `NestedForkJoinExample.lean`; this is not a claim about Google's implementation.

The winner event records threshold 1, population 3 and selected terminal `e7`. Its predecessors are `[e1,e7]`, not the results of all three jobs. The outer reply records `[e0,e8,e6]`: task start, chosen

result and completed audit. Reply time is 12 synthetic milliseconds; cleanup ends at 18. These times illustrate ordering, not measured search performance.

Selection is atomic, so racing results cannot give a 1-of-m task two winners. "First" is the coordinator's observed order with a declared tie rule, not worker clocks. A cancellation request is not an acknowledgment, late results never replace the chosen one, and the requirement says whether the reply waits for cleanup (here it does not).

2.1.2 Durations from boundary events

An event is one observation at one instant; a duration comes from two boundary events.

```
structure AnalysisEvent where
  id       : EventId
  priorIds : List EventId
  timeAt   : ClockTimestamp
  bytes    : ByteArray
  reveals  : List ValueRef
```

The first retail purchase in `examples/bakery-events.jsonl` reserves bread baked that morning, tenders payment, and completes in about a second:

Event	Prior backpointers	`timeAt`	Meaning	Bytes	Reveals
e_0	$[]$	28859991	purchase requested	59	product and quantity
e_1	$[e_0]$	28860343	inventory reserved	62	batch allocation
e_2	$[e_1]$	28861004	payment tendered	55	amount
e_3	$[e_1, e_2]$	28861248	sale completed	72	bread and payment

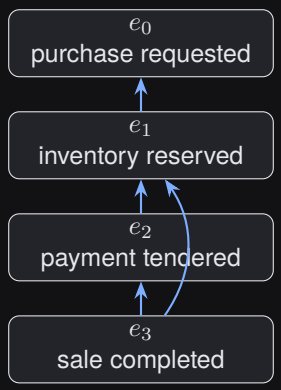


Figure 2.1: The table's immediate predecessor backpointers rendered as an order graph. Each arrow points from an event to an ID in its `priorIds` list: for example, $e_2 \to e_1$. The extra $e_3 \to e_1$ edge preserves direct correlation with the inventory allocation even though e_1 is already an ancestor through e_2.

List order is a storage convenience; event order is the transitive closure of `priorIds`. These quantities are folds over the list:

$$\begin{aligned}\text{reservationTime}(e_0, e_1) &= 28860343 - 28859991 = 352 \text{ ms},\\ \text{paymentTime}(e_1, e_2) &= 28861004 - 28860343 = 661 \text{ ms},\\ \text{purchaseLatency}(e_0, e_3) &= 28861248 - 28859991 = 1257 \text{ ms},\\ \text{wireBytes} &= 59 + 62 + 55 + 72 = 248.\end{aligned}$$

Two events are concurrent when neither is an ancestor of the other:

```
def concurrent (events : List AnalysisEvent) (a b : EventId) : Bool :=
  !reachable events a b && !reachable events b a
```

The trace's maximum concurrency is the partial order's width, its largest antichain; this purchase is a chain, of width one. Incomparable events may appear in either order in a serialization. Runtime concurrency, unlike order width, needs separate start and completion events. In the next trace a request forks into two task attempts whose completions point back to their starts and are joined by the response; the shading is grouping only.

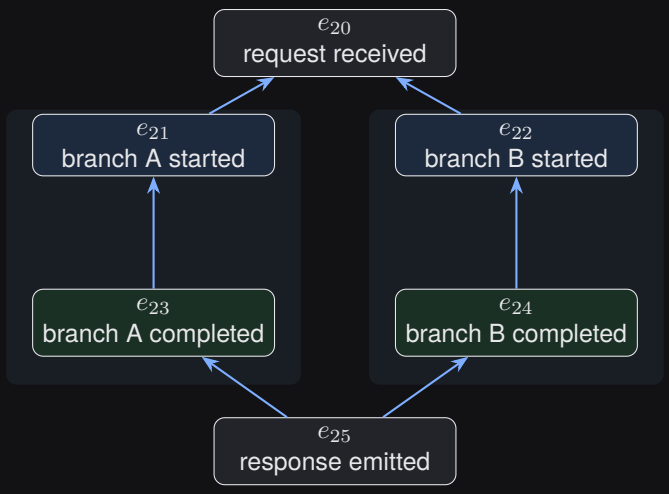


Figure 2.2: A separate synthetic example of task boundaries nested inside a concurrent fork/join. The intervals $e_{21} \ldots e_{23}$ and $e_{22} \ldots e_{24}$ may overlap, while the two branches remain incomparable in predecessor order. Either branch may appear first in a linear serialization without changing the graph.

Elapsed time along a path:

```
edgeTime(prior, event) := event.timeAt - prior.timeAt
elapsed(started, completed) := completed.timeAt - started.timeAt
```

The retail critical path e_0, e_1, e_2, e_3 takes 1257 ms. Queue occupancy, queued bytes, active keys and memory pressure come from adding enqueue and dequeue events and folding in clock order. The whole corpus has 20,000 orders and 81,039 events; streaming it gives these values, generated as macros:

Reduction over `bakery-events.jsonl`	Observed value
sales completed / purchase requests	20,000/20,000
stockouts / purchase requests	0/20,000
morning batches baked	140
FIFO shipments dequeued / enqueued	140/140
maximum simultaneously active purchases	1

The ratios are frequencies for this corpus; the other rows are counts and maxima. Knowledge is another reduction: the actors that can derive a value from their initial knowledge and the bytes they see.

```
def knowers (actors : List Actor) (v : ValueRef)
    (events : List AnalysisEvent) : List Actor :=
  actors.filter fun actor =>
    Derives (knowledgeFrom actor events) v
```

“Only these actors know v” compares that set with an allowed one; secrecy is not a separate primitive (Section 17.5).

2.2 Derived state

A current session, an authenticated principal or an outstanding request is computed by folding the trace, so its evidence stays visible. Private state enters the model only when two executions with the same visible trace may legally diverge, and even then the first remedy is to expose the distinction as an event or parameter.

2.3 Time is more than a timestamp

Timing constraints pair explicit boundaries; changing a timeout changes the requirement, not the order or causal relations. Elapsed time between two events of one `(session, task)`:

```
def elapsed (start finish : Event) : Option Duration :=
  if start.session = finish.session &&
     start.task = finish.task &&
     start.timeAt <= finish.timeAt
  then some (finish.timeAt - start.timeAt)
  else none
```

Latency queries name their boundary kinds and select the start through the terminal’s backpointer, since `(session, task)` alone cannot separate overlapping attempts; reducers never pair by adjacency. Every declared terminal uses the same pairing. A collapsed production keeps its boundary IDs, links, parameters, aggregate deltas and validation status, so opening and closing it changes presentation only.

2.4 Estimating completion from an event stream

`LeanFM.StreamStats` reconstructs attempts from observed terminal-to-start links. For a window with M starts, C matched terminals, S successes and completed latencies $L_1, \ldots, L_C$:

$$\widehat{P}(\text{complete}) = \frac{C}{M}, \qquad \widehat{P}(\text{success} \mid \text{complete}) = \frac{S}{C}, \qquad \widehat{E}[L \mid \text{complete}] = \frac{\sum_i L_i}{C}.$$

Attempts without a terminal are *in flight*: they count as started and load the queues, but are not failures, which would bias both estimates whenever the window closes first. The statistics group by task, actor, time bucket, concurrency or queue occupancy. Specified probabilities and dwell are predictions; these are measurements to compare with them.

2.5 Charts as declarative trace reductions

A proposed chart declaration (illustrative, not implemented) names an event source, a grouping or x expression, and an aggregate or y expression: a reduction, not rendering code.

```
def outcomeShare : ChartSpec :=
  pie "quote outcomes" do
    source events
    where_ (kind == "quote accepted" || kind == "quote declined")
    groupBy kind
    value count

def responseLatency : ChartSpec :=
  line "quote response latency" do
    source (correlate events by "quoteId")
    x clock
    y (elapsed "quote offered" "customer decision")
    series "p50" median

def latencyByConcurrency : ChartSpec :=
  xyplot "latency by concurrent sessions" do
    source completedRuns
    x maxConcurrentSessions
    y (mean (elapsed firstEvent terminalEvent))

def queueByOfferedLoad : ChartSpec :=
  xyplot "queue occupancy by offered load" do
    source completedRuns
    x requestsPerSecond
    y (mean totalQueueOccupancy)

def propagatedGrantProbability : ChartSpec :=
  xyplot "user probability propagated through deterministic servers" do
    source parameterSweep
    x userGrantProbability
    y terminalGrantProbability
```

An XY plot has two numeric axes; a time series's x is the event clock, a categorical chart's are labels. The last example sweeps a parameter to check that deterministic processing transports the user's probability mass. Each generated chart follows the provenance rule of Chapter 3.

2.6 Round trip through bytes

A generated scenario must cross the byte boundary both ways. `ScenarioEvent` in `Requirements.proto` carries identity, the `prior` list, correlation key, actors, `timeAt` and the payload atom. The round-trip check decodes the bytes, compares the value with the generated one, and checks the reconstructed `prior` graph; losing a fork or join fails the build.

Chapter 3

Scenarios

A scenario is LeanFM's named, bounded stream of timestamped events for one run of a session (for example, a quote request, OAuth exchange, or 9×9 game). The global type says which runs are legal; a scenario is one of them, either generated from the type or recorded from production, and it is the concrete thing people point at while arguing about the type.

3.1 Event envelope and scenario grouping

The shared envelope and semantics of `id`, `priorIds`, `causeId`, actor endpoints, correlation and monotonic `timeAt` are defined in Chapter 2. A serialized `scenario` key or aliases such as `src/message` are canonical only if the accepted schema and reducers declare them.

3.2 Derived outputs and provenance

Question-driven outputs include interaction diagrams, residual state machines, latency/throughput charts and outcome distributions. Each `DesiredOutput` must bind its prompt, precise question, event source, reducer, unit and scenario boundary. A listed output is not implemented until its renderer exists; when a generated view carries the explanation, prefer it to repeated prose and add a short interpretation.

3.3 Example: minimal two-player game scenario

Below is an illustrative event fragment for a 9×9 game. Times are milliseconds on the model's monotonic clock:

```
{ "id":"evt-0010", "priorIds":[], "causeId":null,
  "timeAt":30000, "session":"s-g1", "task":"game-g1",
  "from":"alice","to":"server","kind":"MoveRequested",
  "payload":{"move":"E5","color":"B"} }
{ "id":"evt-0011", "priorIds":["evt-0010"], "causeId":"evt-0010",
  "timeAt":30001, "session":"s-g1", "task":"game-g1",
  "from":"server","to":"alice","kind":"MoveAccepted",
  "payload":{"move":"E5"} }
```

The latency pair is `evt-0010` to `evt-0011`: one millisecond.

Prefer runnable scenario traces to prose-only examples, declare their expected derived views and provenance, and keep transport mappings in `Implementation.lean` once the requirements check.

Chapter 4

The Bakery: Declared and Measured

The bakery is the quantitative worked example. Its tasks are written as a *core*, a session between customers, staff and the oven that says only what can happen, plus *overlays*: separate tables of branch probabilities and delays. A *declared* overlay states what we expect; a *measured* overlay is reduced from recorded events. Both answer the same questions about the same unchanged core, and Chapter 13 develops the idea in general. The corpus is the 81,039-event stream `examples/bakery-events.jsonl`, a reproducible synthetic population generated by `scripts/generate_bakery_ev` its numbers describe that corpus, not a real bakery. Every figure and table below is regenerated by `lake env lean -run BakeryFlowExample.lean`.

4.1 The morning, generated

Bread is baked to a schedule, not to order. Each batch is baked, queued on a truck, and stocked at the storefront before opening. The stocking task's core lists only those four messages; the overlay measured from the 140 recorded batches supplies how long each step takes. The figure is the most likely stocking run under that overlay.



Figure 4.1: One batch from oven to shelf, with mean step times measured from the corpus.

4.2 A sale: what can happen

A customer asks for bread. The storefront either reserves stocked inventory or turns the customer away; a reserved order is paid for, and the payment either completes the sale or is declined. The

core states this and nothing more. It already proves that every sale attempt ends and that a customer *can* be turned away.

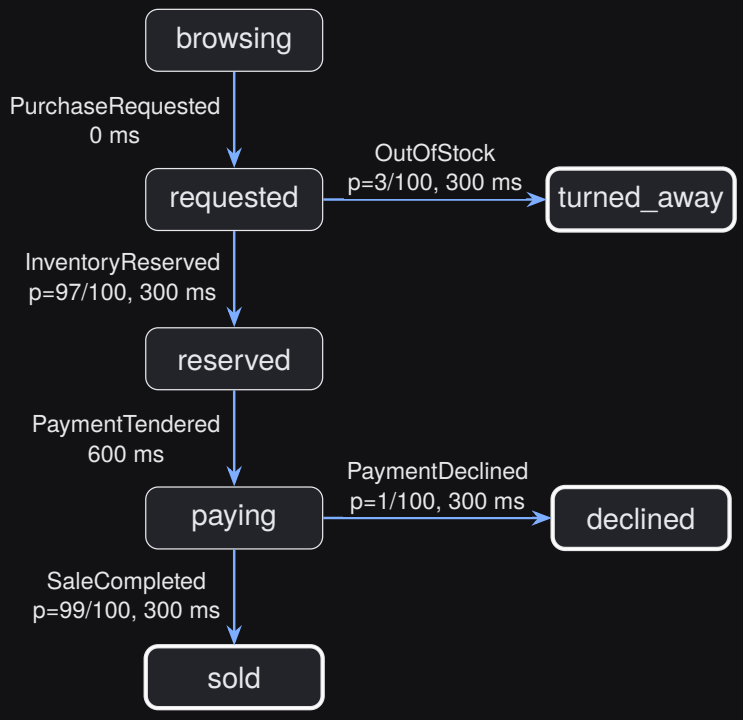


Figure 4.2: The retail-sale core, with the bakery's declared expectations drawn on its edges. Double borders mark terminal states.

4.3 What we expected, and what the corpus shows

The declared overlay was written before looking at any data: a 3% stockout rate, a 1% payment-decline rate, and round-number delays. The measured overlay has exactly the same shape. `LeanFM.Overlay.measure` walks each of the 20,000 recorded sales through the core, counts how often each branch was taken, and averages the time since the previous message. A recorded session that left the grammar would be reported rather than counted.

Transition	Declared p	Declared delay	Measured count	Mean delay
browsing → `PurchaseRequested`	1	0 ms	20000 / 20000	0 ms
requested → `InventoryReserved`	97/100	300 ms	20000 / 20000	289 ms
requested → `OutOfStock`	3/100	300 ms	0 / 20000	—
reserved → `PaymentTendered`	1	600 ms	20000 / 20000	574 ms
paying → `SaleCompleted`	99/100	300 ms	20000 / 20000	300 ms
paying → `PaymentDeclined`	1/100	300 ms	0 / 20000	—

Table 4.1: Declared expectations against branch counts and mean delays reduced from the corpus.

4.4 The same questions under both overlays

Question	Formula	Declared	Measured
Does every sale attempt end?	`AF terminal`	proved	(core, no overlay)
Can a customer be turned away?	`EF turned_away`	proved	(core, no overlay)
How likely is a sale?	`P=? [F sold]`	9603/10000 (0.9603)	1
Sold within 1.2 s?	`P=? [F<=1200ms sold]`	9603/10000 (0.9603)	1; raw data 0.5454
How often out of stock?	branch `OutOfStock`	3/100	0 of 20000; 95% bound $\leq$ 3/20000
Expected sale duration	`E[duration]`	1173.0 ms	1163.0 ms

Table 4.2: One unchanged core, two overlays. The first two rows need no numbers at all.

Two rows deserve a closer reading. The corpus contains no stockout in 20,000 sales, but "never observed" is not "impossible": with zero occurrences in n trials the usual 95% upper bound on the rate is about $3/n$, so the honest measured answer is a bound, not zero. The core, meanwhile, proves that turning a customer away is possible, so the path must still be designed and tested even though the data never exercised it.

4.5 The deadline lesson: a mean hides the spread

The measured overlay records each step's *mean* delay. Its predicted sale takes 1163 ms, inside the 1.2 s deadline, so the model answers $P[F_{\leq 1200\,\mathrm{ms}}\ sold] = 1$. The recorded sales say otherwise: only 0.545 of them finished within 1.2 s. Real sales took between 348 and 1956 ms.

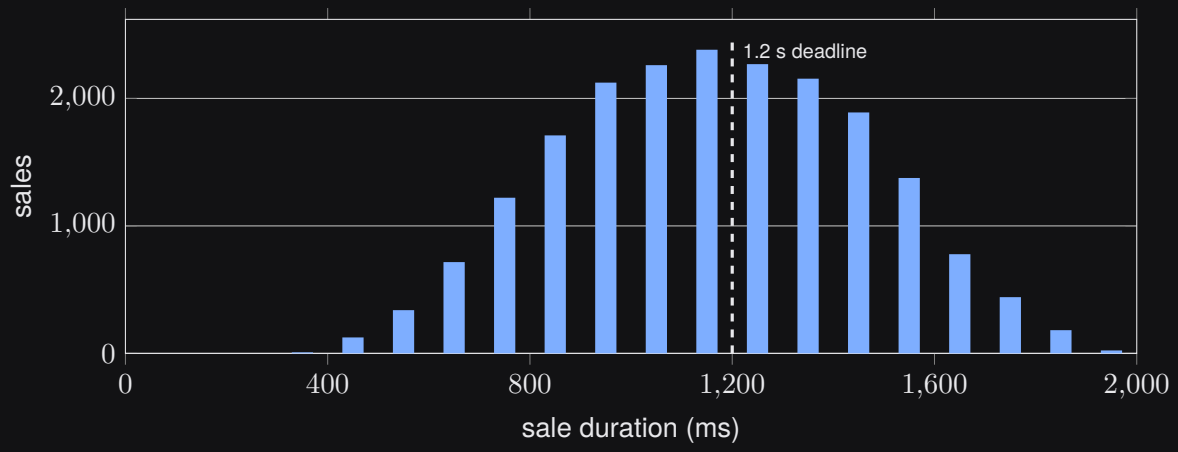


Figure 4.3: Duration of every recorded sale. The mean sits just inside the deadline; almost half of the sales do not.

This is why a measured overlay must eventually carry distributions, not just means, as FizzBee's performance files do: the deadline question is about the spread, and a point estimate answers it with false confidence. Until then, the raw-data column keeps the model honest by comparing its prediction with the recorded fraction.

4.6 Generated runs

The most likely sale under the measured overlay is the one a newcomer should see first. The most likely *failing* sale comes from the declared overlay, because the measured one gives every failure probability zero: it is the stockout, the case the corpus never shows and the core says can happen.

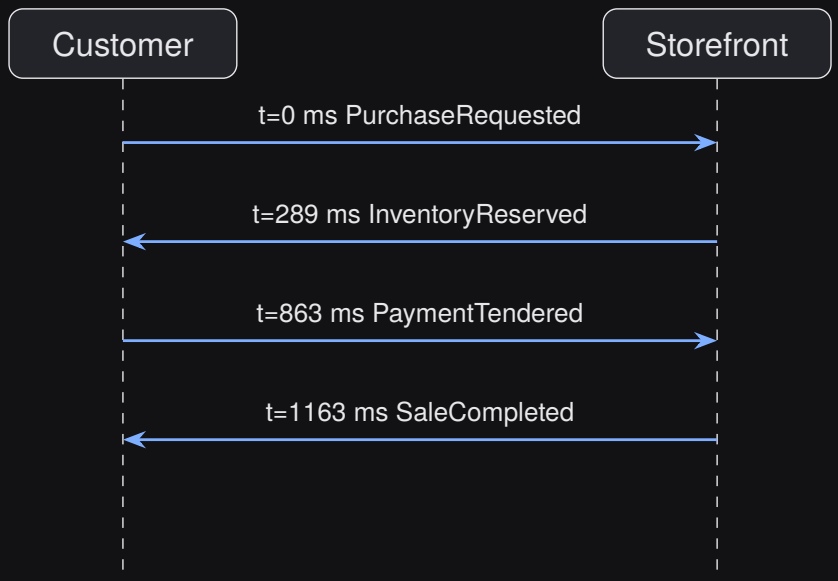


Figure 4.4: The usual sale, with mean times measured from the corpus.

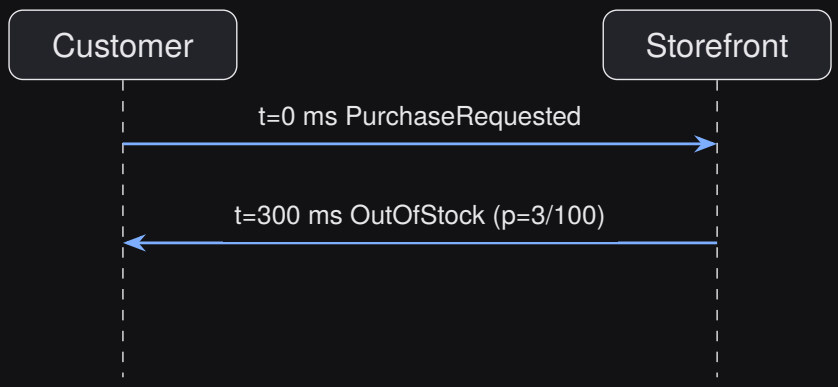


Figure 4.5: The most likely failing sale under the declared overlay: a stockout, probability $3/100$, never seen in the corpus.

4.7 Money and waste

The same corpus answers economic questions through ordinary reducers. For each calendar day, net revenue is the sum of `SaleCompleted` amounts less refunds; ingredient cost comes from `BatchBaked`; employee pay from `ShiftPaid`; and modeled contribution is revenue less ingredients and pay. Donated bread is already in ingredient cost, so its value is reported separately rather than subtracted twice, and it is not a tax saving until tax rules are supplied. Costs that have no events in the corpus are excluded, so this is contribution, not business profit.

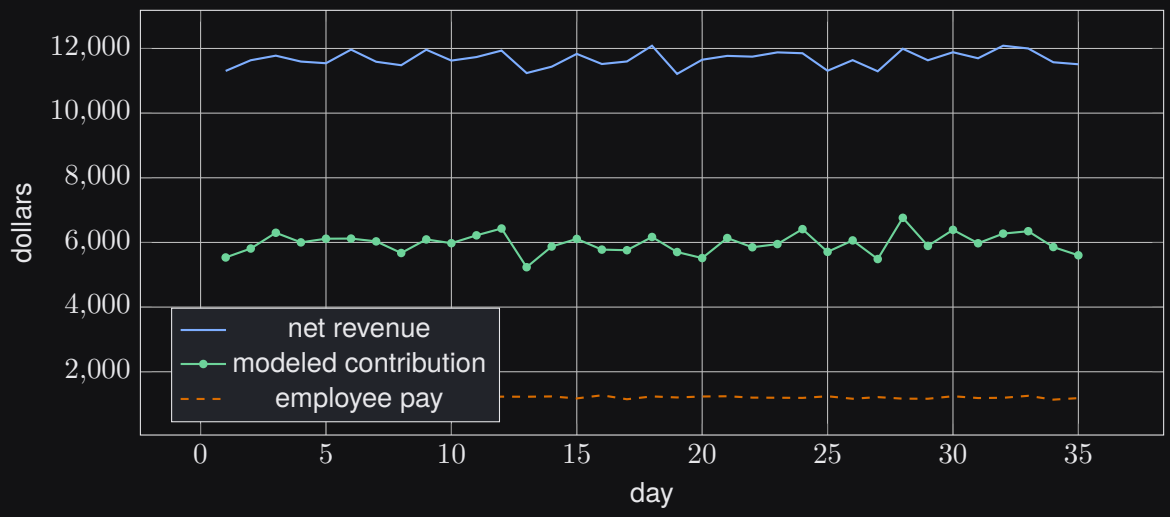


Figure 4.6: Daily economics reduced from sale, batch and payroll events.

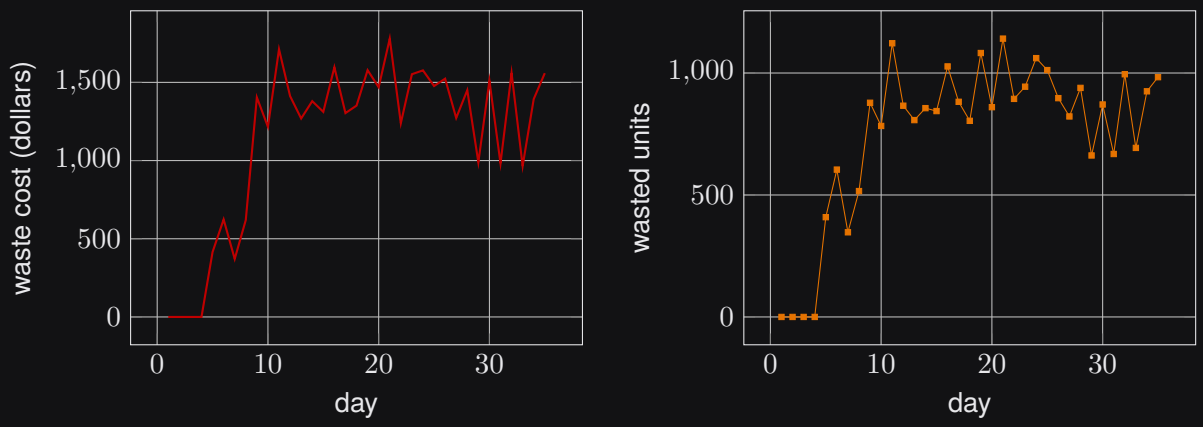


Figure 4.7: Donation-eligible cost and units by day: inventory left when a batch's freshness window closes. Cost and units differ because a donated sourdough loaf and a donated croissant are not worth the same.

Grouping the same events differently—by product, batch age, shift, or order volume—gives further charts without changing the corpus. The axes are always named reductions over events, which is what lets a measured overlay and an economic chart come from one source of truth. How these specifications drive code generation is the subject of Chapter 19.

Chapter 5

Global Types as Grammars

A global session type is a grammar. Its terminal symbols are messages, each with a sender and a receiver; its productions are the type's sequence, choice, parallel composition and recursion; the language it generates is the set of legal conversations. Requirements therefore stack: every protocol reads as if it were the top-level one, and a larger one includes smaller ones.

Global types are written in Lean with the `protocol` syntax of `LeanFM.Session`. A protocol elaborates to a value of type `Global R` for any role type `R`, and the library checks who decides each choice, what each cause may point to, and each role's projection. `SessionExample.lean` holds every protocol in this chapter and runs those checks whenever it is elaborated. For durable artifacts a protocol translates to the older `GrammarExpr` form that the rest of LeanFM's validation consumes. The queue rules at the end are model semantics, not a proof about a Go runtime.

5.1 The protocol syntax

Every message is a Lean structure whose fields are its payload:

```
structure Order    where item : String  deriving TypeName
structure Quote    where price : Nat    deriving TypeName
structure Pay      where amount : Nat   deriving TypeName
structure Paid     where                deriving TypeName
```

A misspelt label is then an unknown identifier, not a new message. A protocol names its roles and lists statements:

```
protocol Purchase (Client, Shop, Bank) {
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q where p.amount = q.price;
       Paid  from Bank to Shop cause p;
       Shipped from Shop to Client;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

`o: Order from Client to Shop;`
: One message. The optional name `o` lets later messages say `cause o` and lets guards read `o.item`.

`cause q`
: This message answers `q`. It becomes the event's cause link, and the checker requires that the sender has sent or received `q`: a role decides only from its own history.

`where p.amount = q.price`
: A Lean `Bool` over the named messages in scope, the guarded one included. It is type-checked when the protocol is written and evaluated over decoded payloads when a trace is checked.

`choice { ...} or { ...}`
: Exactly one branch happens. Who decides is not written: it is the role that starts every branch, here the shop.

`Timeout at Sender;`
: An event at one role only, such as its timer firing; nobody else observes it.

`rec Block { ... continue Block; }`
: A named block that `continue` repeats. `continue` ends its branch, so repetition is always in tail position and the state space stays finite.

`include Replicate Primary A B C;`
: Another protocol, run here with these roles; whatever follows runs where it finishes.

`par { ...} and { ...}`
: Both sides, interleaved, each keeping its own order.

`join 2 of { ...} and { ...} and { ...}`
: Any two of the branches, then the rest of the protocol.

The words are not reserved, so the library takes no identifiers from Lean code. `protocol Purchase (Client, Shop, Bank)` defines `Purchase : R → R → R → Global R`, so it can be instantiated and included with a larger protocol's roles. Each statement elaborates to one constructor:

```
inductive Global (R : Type) where
  | msg    (i : Interaction R) (k : Global R)        -- one message, then k
  | act    (role : R) (label : String) (k : Global R) -- an event at one role
  | choice (branches : List (Global R))              -- one role decides
  | sessions (body : Global R) (k : Global R)         -- any number of copies
  | par    (branches : List (Global R)) (k : Global R)
  | join   (need : Nat) (branches : List (Global R)) (k : Global R)
  | loop   (name : String) (body : Global R)
  | var    (name : String)
  | done
```

A choice does not name its decider. Scribble writes `choice at Shop`, but the name can only be right one way: the decider is the role that starts every branch, by sending its first message or by an event at that role alone. LeanFM reads it off the branches, so there is one way to write a choice, and an empty branch, `or { }`, is that role choosing to do nothing. If Alice's message started one branch

and Bob's the other, both could act at once; the checker reports the race, naming who starts what. This is the non-local-choice (realizability) problem: the choice must be *directed*, internal to one role and external to the others. The second condition, that every role whose behaviour differs between branches is told which was taken, is checked by projection (below). A first attempt at TFTP that puts the sender's timer beside the acknowledgement as a choice has no decider at all:

```
protocol TransferWithTimer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Timeout at Sender;
      continue Block;
    } or {
      Ack from Receiver to Sender cause d;
      continue Block;
    }
  }
}
```

```
no single role decides a choice, so it is a race: Sender starts Timeout; Receiver
    starts Ack
```

The fix, in Chapter 6, moves the timer out of the type and into the channel's physics. Simple finite conversations are regular languages; nested requests need context-free productions; correlation, freshness, uniqueness and authorization need parameter flow or guards.

5.2 A BNF for composable interactions

The syntax, independently of Lean's own grammar:

```
<protocol>  ::= "protocol" <Name> "(" <Role> ("," <Role>)* ")"
                "{" <stmt>* "}"

<stmt>      ::= [<id> ":"] <Msg> "from" <Role> "to" <Role>
                  ["cause" <id>] ["where" <term>] ";"
              | <Event> "at" <Role> ";"
              | "choice" "{" <stmt>* "}"
                  ("or" "{" <stmt>* "}")*
              | "par" "{" <stmt>* "}" ("and" "{" <stmt>* "}")*
              | "join" <natural> "of" "{" <stmt>* "}"
                  ("and" "{" <stmt>* "}")*
              | "rec" <Name> "{" <stmt>* "}"
              | "continue" <Name> ";"
              | "sessions" "{" <stmt>* "}"
              | "include" <term> ";"
```

Sequence is one statement after another. `sessions` runs any number of independent copies of its body beside what follows, as a server runs one handler per request. `<Msg>` and `<Event>` name Lean types; `<term>` is a Lean `Bool` for `where` and a protocol applied to roles for `include`. Guards over named payloads supply the context-sensitive correlation context-free syntax lacks.

A threshold join is declared, not inferred from observed predecessors. A primary writes to three replicas and answers its client after any two have stored the write; the first protocol is included in the second:

```
protocol Replicate (P, A, B, C) {
  join 2 of {
    wa: Write from P to A; Stored from A to P cause wa;
  } and {
    wb: Write from P to B; Stored from B to P cause wb;
  } and {
    wc: Write from P to C; Stored from C to P cause wc;
  }
}

/-- A production that includes another, each written as if it were the whole protocol.
    -/
protocol ReplicatedOrder (Client, Primary, A, B, C) {
  o: Order from Client to Primary;
  include Replicate Primary A B C;
  Done from Primary to Client cause o;
}
```

The continuation's first event records `joinRequired=2`, `joinTotal=3` and the two branch-terminal IDs that satisfied it, which also appear in its `prior` list. Unselected branches are not canceled and may still emit events; cancellation must be an explicit message. The OAuth example's two outcomes, as generated views:

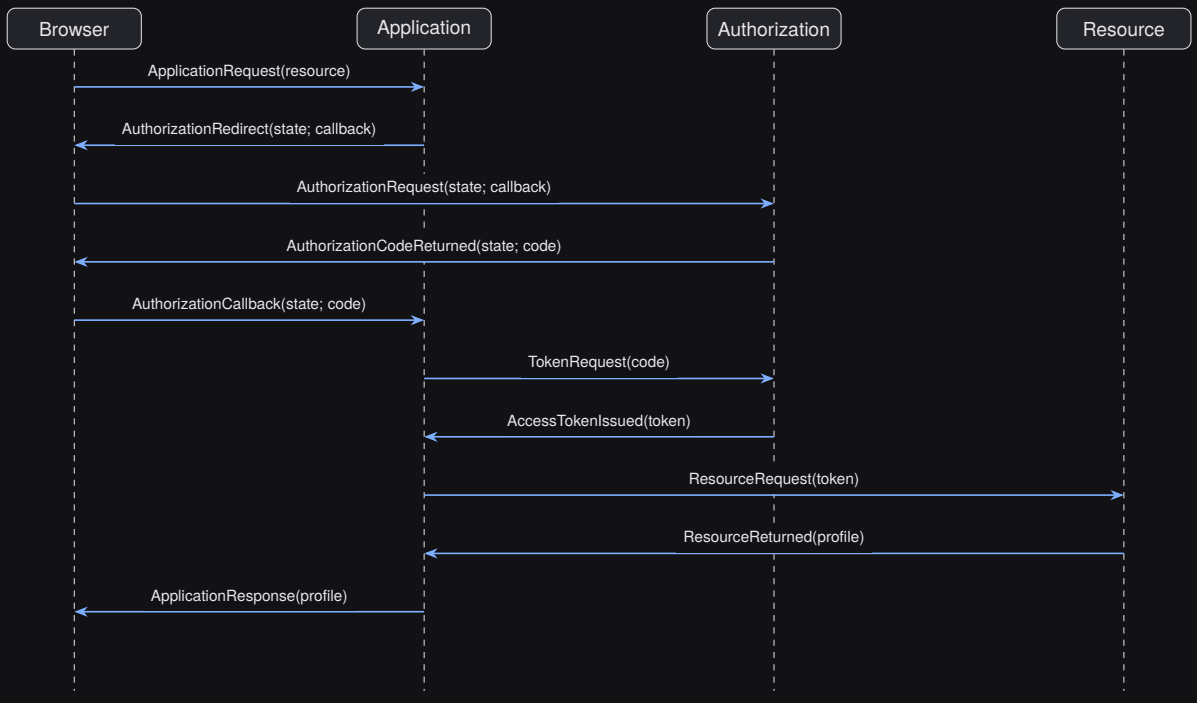


Figure 5.1: Granted OAuth interaction. The resource exchange occurs only after the authorization-code branch succeeds.

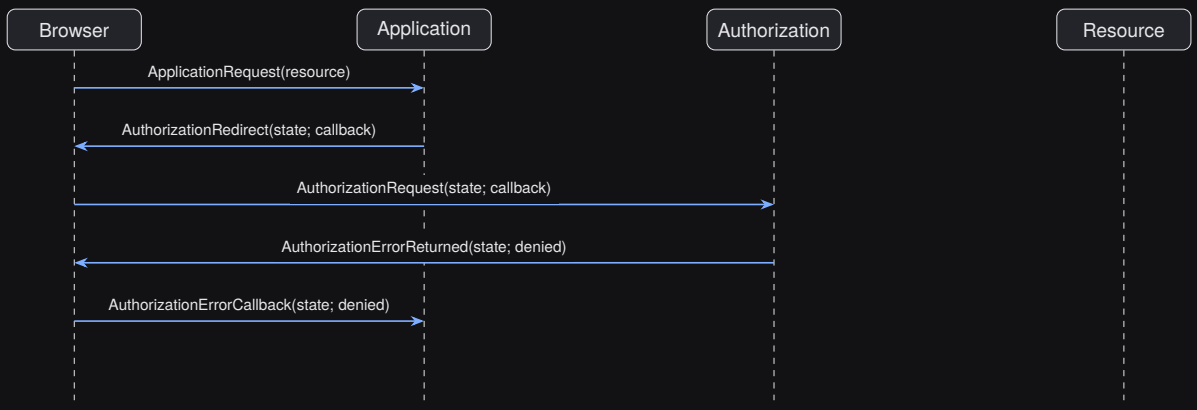


Figure 5.2: Denied OAuth interaction. No token or resource request follows the error.

Chapter 10 gives the full productions and guarded exchange.

5.3 Parameters carry information

A protocol should bind values at one message and reuse them later. A message's name makes its payload available to every later guard:

```
r: Redirect  from Server to Browser;
a: Authorize from Browser to Provider where a.state = r.state;
c: Callback  from Provider to Server cause a where c.state = r.state;
```

The repeated `state` states the information flow that distinguishes a matching callback from an unrelated one.

5.4 Projection: from the global type to each role

Projection derives each role's *local type*: the protocol with every message the role neither sends nor receives erased, and the rest rewritten as sends and receives. A choice the role makes becomes a `select`; another role's becomes an `offer`, resolved by what arrives. `Global.project` for `Purchase`:
Client

```
send Order to Shop;
offer {
  receive Quote from Shop;
  send Pay to Bank;
  receive Shipped from Shop;
} or {
  receive SoldOut from Shop;
}
```

Shop

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

Bank

```
offer {
  receive Pay from Client;
  send Paid to Shop;
} or {
}
```

The bank never learns why it was not paid, and should not depend on it. Where a choice is erased, the branches a role still sees must be the same, or start with different messages it receives,

or the role's part must not have started yet (the bank's empty `or { }`); otherwise projection fails with the reason, as for the hold and the timer above. A local type is the thread an actor runs for one session. LeanFM checks that every projection exists; that projection and composition preserve the global type's properties is not yet proved.

5.5 Declared physics

What happens to a message between send and receive is declared beside the protocol as the *physics* of each channel. In `LeanFM.Session.Physics` a channel's defaults are the reliable channel, so a declaration names only what differs:

```
protocol Transfer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Ack from Receiver to Sender cause d;
      continue Block;
    } or {
      LastAck from Receiver to Sender cause d;
    }
  }
}

def tftpPhysics : Physics Role :=
  { channels := [ (.sender, .receiver, { udp 2 with duplication := false })
                , (.receiver, .sender, { udp 2 with duplication := false }) ]
    timers := [{ role := .sender }] }
```

The checker compares what each channel surfaces with what the protocol handles. TFTP's acknowledgement channel can reorder, and the requirements do not say so:

```
channel Receiver -> Sender surfaces reorder, which its protocol does not handle
```

The six axes of Chapter 1 are the structure's fields, and a preset is a value a declaration updates. Abridged:

```
inductive WhenFull   | block | reject | dropNewest | dropOldest
inductive Sequencing | fifo | any

structure Sharding where
  key        : String
  partitions : Nat := 3

/-- Every field defaults to the reliable channel. -/
structure Channel where
  capacity    : Nat := 16
  whenFull    : WhenFull := .block
  order       : Sequencing := .fifo      -- within one partition
  sharding    : Option Sharding := none
  loss        : Bool := false
  duplication : Bool := false
  masked      : List Condition := []     -- hidden by a lower layer's retries
```

```
  extra       : List Condition := []     -- surfaced by layers a library adds

def udp (capacity : Nat := 2) : Channel :=
  { capacity, whenFull := .dropNewest, order := .any,
    loss := true, duplication := true }
```

Per-key ordering is `fifo` on a sharded channel. The checked system is the product of the local types and their channels, and each field adds or removes transitions:

- `capacity` and `whenFull`: a send to a full queue blocks the sender, delivers a refusal message back to it, or discards a message;
- `order`: under `fifo` only the head of a queue may be delivered; under `any` any queued message may be;
- `sharding`: the channel is `partitions` queues, each with the declared capacity, and a send goes to the queue its key selects; for checking, keys are abstracted to as many distinct values as the protocol's messages can tell apart;
- `loss`: a transition that removes any queued message;
- `duplication`: a transition that delivers a queued message and leaves it queued.

A timer may fire whenever its role is waiting. Each `may` multiplies the state space the checker explores. Chapter 6 builds this product for TFTP: a classically well-formed type whose original implementation duplicates every block once one timer fires early. The rest of this chapter defines the default physics: bounded, reliable, blocking queues.

5.6 Local types run on actors with bounded queues

Each role is played by an actor, which may take part in many sessions at once and has finite inbound and outbound queues. Producing a message may block on a full outbound queue and transport on a full inbound one, so sending, delivery and consumption are distinct. Messages carry a session identifier (the instance of the global type) and a task identifier, whose pair keys actor-local work:

```
structure WorkKey where
  session : SessionId
  task    : TaskId

structure ActorRuntime where
  inboundCapacity  : Nat
  outboundCapacity : Nat
  inbound          : List Envelope
  outbound         : List Envelope
  inFlight         : List (Prod WorkKey GrammarResidual)
  inboundBounded   : inbound.length <= inboundCapacity
  outboundBounded  : outbound.length <= outboundCapacity
```

Consuming an envelope selects or creates the residual for its `(session, task)` pair; there is no actor-wide "current task", and keys interleave while each conversation keeps its order. The queues behave like Go channels, with capacity enforced when an operation is attempted:

```
send(channel, message) =
  if channel.length < channel.capacity
  then sent(channel ++ [message])
  else blocked

receive(channel) =
  if channel = message :: rest
  then received(message, rest)
  else blocked

tryReceive(channel) =
  if channel = message :: rest
  then some(message, rest)
  else none
```

A blocked operation leaves the channel unchanged until space or data appears. `tryReceive` returning `none` means empty now, lets the actor continue, and is not an optional payload. The transitions:

- an actor step appends a produced envelope to its outbound queue;
- a transport step moves the outbound head to the destination's inbound queue when both operations are within capacity;
- a receive step removes an inbound envelope and advances the matching `(session, task)` residual;
- sending to a full outbound queue blocks the sender without enqueueing;
- transport to a full inbound queue blocks without removing the outbound head;
- blocking receive on an empty inbound queue blocks, while nonblocking `tryReceive` returns `none` and permits later work.

Queue order is not predecessor order: FIFO decides which envelope is available next, while `priorIds` records the event graph, forks and joins included. Incomparable branches may commute, and a predecessor pointer does not claim causation.

Chapter 6

Declared Physics: TFTP and the Sorcerer's Apprentice

A grammar does not say what happens to a message after it is sent: whether it queues, how large the queue is, whether the network loses it, duplicates it or delays it. Those facts are part of what a protocol means. The example is the Trivial File Transfer Protocol (TFTP) [23, 24] and its best-known defect, the *Sorcerer's Apprentice* bug, named and fixed in RFC 1123 [3].

This is the book's central claim in its smallest form. As a multiparty session type [12], the original and the fixed sender are the same well-formed protocol. With the physics declared, LeanFM finds a walk in which one early timer makes every later block travel twice for the original, and proves that none exists for the fix.

Status: the verdicts are real `check_ctl` results, and the protocol and its physics are checked Lean in `SessionExample.lean`. The product of roles and queues is still built by a small script, `video/tftp_model.mjs` in the companion video repository; deriving it from the declarations is future work.

6.1 The grammar

TFTP moves a file in numbered blocks. The sender sends `DATA` n; the receiver answers `ACK` n; the sender then sends `DATA` $n{+}1$. A block shorter than the full size ends the transfer.

In LeanFM's session-type syntax, with the physics beside it:

```
protocol Transfer (Sender, Receiver) {
  rec Block {
    d: Data from Sender to Receiver;
    choice {
      Ack from Receiver to Sender cause d;
      continue Block;
    } or {
      LastAck from Receiver to Sender cause d;
    }
  }
}

def tftpPhysics : Physics Role :=
```

```
{ channels := [ (.sender, .receiver, { udp 2 with duplication := false })
              , (.receiver, .sender, { udp 2 with duplication := false }) ]
  timers := [{ role := .sender }] }
```

Without a timer this is a textbook global type: the receiver chooses, answering `Ack` for a full block and `LastAck` for a short one, and the projected roles never deadlock or receive an unexpected message. But a real network loses packets, so the sender needs a timer that resends a block when no `ACK` arrives in time. That retransmission is the whole engineering content of TFTP, and the classic reading has nowhere to put it, because it assumes every message arrives exactly once, in order.

6.2 The physics

`tftpPhysics` above declares each channel as `udp` with one difference written out. Each direction holds at most two packets, delivered in either order or dropped, and the sender's timer may fire any time a block is unacknowledged, since a slow answer looks like a lost one. The network is declared never to duplicate: every duplicate below is made by the sender. The model has three blocks. The two senders differ in one line, the reaction to a repeated `ACK`:

Original (RFC 783).
Every `ACK` n, even a repeated one, makes the sender send `DATA` $n{+}1$.

Fixed (RFC 1123).
A repeated `ACK` is ignored. A block is sent again only when its own timer fires.

The checked machine is the product of sender, receiver and both queues: 201 reachable states and 891 transitions for each sender. Its events are `timeout`, `drop`, `data` (a new block delivered), `dup`k (block k delivered again, and acknowledged again), `ack` (the first `ACK` for the current block), `dupack` (an `ACK` for an earlier block) and `done`.

6.3 The question

Suppose the timer fires early once, at the start, so `DATA` 1 is in flight twice, and afterwards nothing is lost and the timer never fires. Can block 3 still arrive twice? From the state after that `timeout`:

$$E\,[\,\neg(\mathit{timeout} \vee \mathit{drop})\;U\;\mathit{dup}_3\,]$$

"Some path has no `timeout` and no `drop` until block 3 is delivered a second time." For the original sender LeanFM proves it, with the witness in table 6.1; the queue columns are read from the states on the path.

The pattern repeats every four steps: a doubled block brings a doubled `ACK`, whose first copy sends the next block and whose late second copy sends it again. Nothing fails and every packet arrives, yet the duplication sustains itself, like the brooms that keep multiplying, doubling the traffic of real transfers. For the fixed sender the formula is disproved over all 509 reachable nodes: the repeated `ACK` 1 is ignored and the duplication dies out.

step	event	meaning	DATA queue	ACK queue
1	`timeout`	timer fires early; DATA 1 sent again	1, 1	–
2	`data`	DATA 1 delivered	1	1
3	`dup1`	DATA 1 delivered again; ACK 1 sent again	–	1, 1
4	`ack`	ACK 1 received; DATA 2 sent	2	1
5	`data`	DATA 2 delivered	–	1, 2
6	`dupack`	ACK 1 received again; DATA 2 sent again	2	2
7	`dup2`	DATA 2 delivered again; ACK 2 sent again	–	2, 2
8	`ack`	ACK 2 received; DATA 3 sent	3	2
9	`data`	DATA 3 delivered	–	2, 3
10	`dupack`	ACK 2 received again; DATA 3 sent again	3	3
11	`dup3`	DATA 3 delivered again; ACK 3 sent again	–	3, 3

Table 6.1: LeanFM's witness for the original sender: one early timer, then no loss and no further timeouts, and every later block is sent and delivered twice.

6.4 Liveness needs fairness

Two more questions are asked of both senders.

$$AG\,EF\ terminal \text{ holds}, \qquad AF\ terminal \text{ fails}.$$

From every state some continuation finishes, but not every path does: LeanFM returns a lasso in which the timer fires forever, each copy dropped by a full queue, and nothing is delivered. No protocol can force a network to deliver, so a promise that the transfer *will* finish needs a stated fairness assumption on the channel, such as "a packet sent infinitely often is delivered infinitely often".

6.5 What the classic session-type reading leaves out

sender	as a session type	with the physics declared
original (RFC 783)	well-formed lock step	one early timer doubles every later block
fixed (RFC 1123)	the same well-formed lock step	no cascade; checked over all 509 nodes

Table 6.2: The same grammar, two meanings.

The grammar is the same in both rows of table 6.2; only its meaning changed. With the theory's unbounded, reliable, ordered queues [12] a retransmission timer is unnecessary, and the Sorcerer's Apprentice cannot even be expressed. LeanFM's position follows from the actor view:

- Every choice is an internal choice of one actor, made only from that actor's own history: what it sent, and what arrived in its queue (`prev` and `cause`).
- Every queue has a declared capacity, and a declared policy for when it is full.
- Every channel declares what the network may do: drop, reorder, duplicate, delay.

- Liveness claims carry their fairness assumptions as named requirements.

UDP protocols make the physics impossible to ignore, and TFTP is the smallest with a famous bug that stays invisible until the queues and the early timer are modeled.

Chapter 7

The Scenario Authoring Language

Proposed notation: this chapter develops a surface language from ordinary Lean `def`, parameters, `let`, typed values and `do` notation, with library operations for scenarios, endpoints, choices, probability and rendering; names may change. The worked MDP is a **finite-model check**, and its figures are **generated analyses**.

7.1 Proposed scenario notation

A scenario is a named fragment of a global type: a grammar whose terminals are observable messages, each with a sender and a receiver. `from_` and `to_` (avoiding Lean's `from`) set the endpoints copied into each event, and an `emit` without both is invalid. `let x := e` evaluates a Lean expression; `let x <- declare T "x"` introduces an immutable typed symbolic value bound by generation or matching. Reusing a binding expresses identity, and receiving an event binds its fields for later messages. `emit` and `receive` are the same kind of terminal, differing only in projection.

```
def requestQuote (customer shop : Actor) : Scenario :=
  scenario "request a quote" do
    let requestId <- declare RequestId "requestId"
    let quoteId <- declare QuoteId "quoteId"
    let amount <- declare Money "amount"
    from_ customer; to_ shop
    emit "quote requested" [field "requestId" requestId]
    from_ shop; to_ customer
    emit "quote offered"
      [field "requestId" requestId, field "quoteId" quoteId,
       field "amount" amount]
```

Early requirements can use semantic message names and typed fields without a protobuf schema; that suffices for ordering, correlation and temporal properties. A later refinement maps the fields to a concrete alphabet, with the obligation of language inclusion once representation details are erased: serialization, retries and internal state may be chosen there, but no visible behavior the abstract grammar forbids.

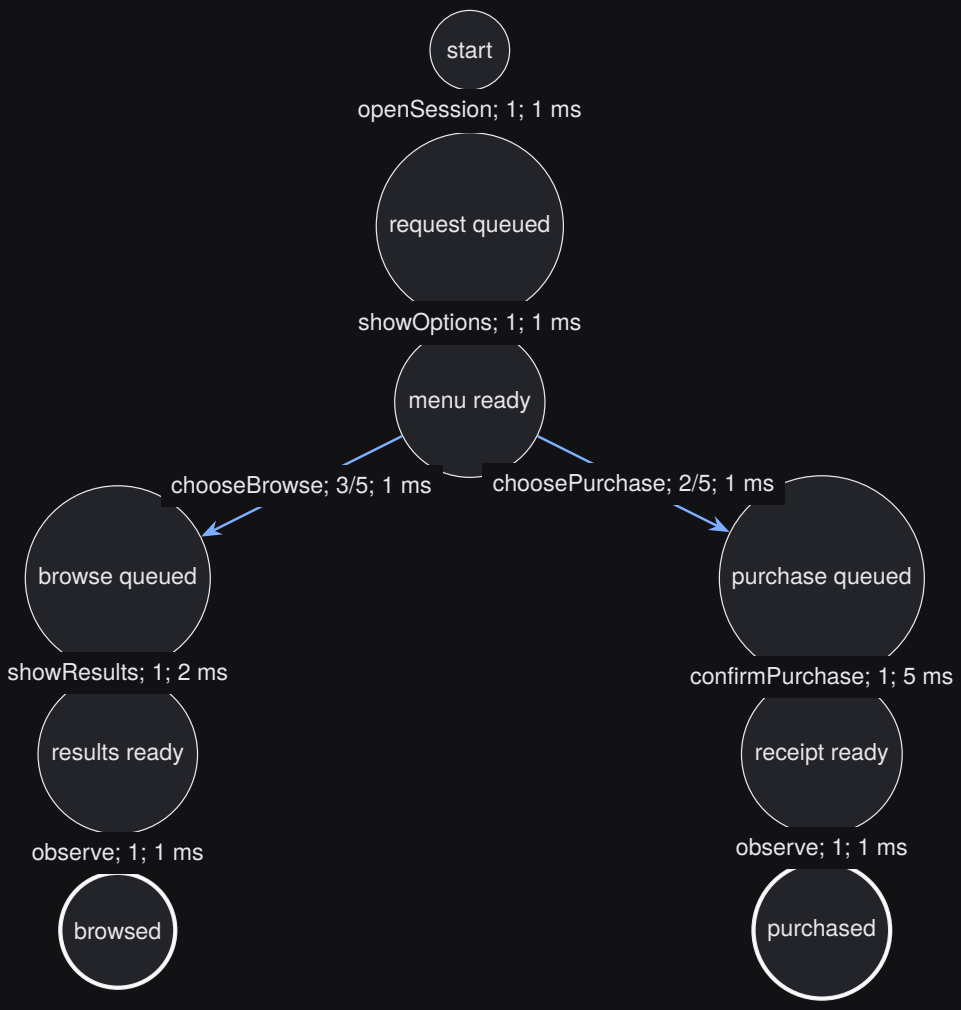
Named inclusions are reusable productions, not textual copies; they can export values for later events, and a compiler resolves references, checks exports and rejects cycles. A quote outcome, for example, chooses between named accepted and declined branches, each renderable on its own (diagram contract: Chapter 3; generated examples: Chapter 5).

`oneOf` states nondeterminism without frequencies; `weightedOneOf` supplies a distribution when one is supported. The first distinguishing terminal decides who chooses (Chapter 5), and deterministic service transitions carry probability mass forward. Requirements may start with unresolved choices or symbolic weights; refinement resolves them and keeps every abstractly legal branch, even one the implementation gives probability zero.

7.2 Worked MDP: a stochastic user and deterministic server



Weights and probability-one server edges come from the checked `Choice` values.



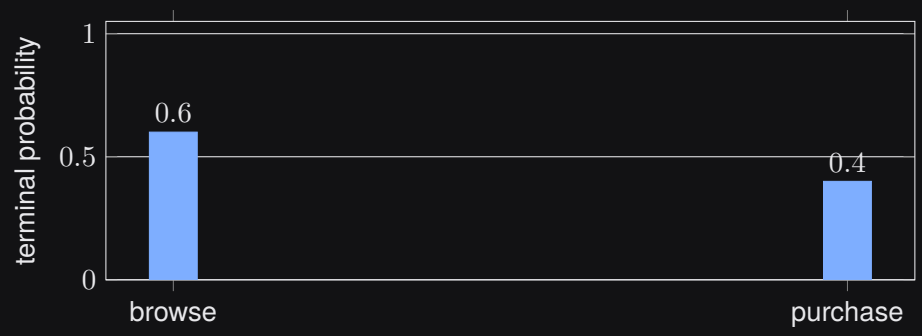
The branch weights are relative integers normalized by their sum:

$$\frac{3}{5} + \frac{2}{5} = 1.$$

Here the weights live inside the model, and the MDP checker rejects an empty chance set and zero weights. The probability overlays of Chapter 13 are a separate mechanism, preferred for new work: the task carries no numbers, the overlay's exact fractions must sum to one, and a branch may have probability zero while staying legal.

Assign reward 1 to browsing and reward 8 to purchasing. The terminal paths and their durations give

$$\mathbb{E}[R] = \frac{3}{5}(1) + \frac{2}{5}(8) = \frac{19}{5}, \qquad \mathbb{E}[T] = \frac{3}{5}(6) + \frac{2}{5}(9) = \frac{36}{5}\ \text{ms} = 7.2\ \text{ms}.$$



`StochasticUserMDPExample.lean` checks the distribution, deterministic server states, terminal paths, mass conservation and both expectations before generating these views. The evaluator expands a bounded number of steps under one fixed policy, so the example is a finite Markov chain. It does not compute an optimal policy, solve an unbounded fixed point, or range over schedulers.

7.3 Actor specifications and instances

An actor name in a grammar denotes a reusable specification, not necessarily one process. A finite population gives that specification concrete identities:

```
actor Client
  |>.instances "client-" 20 (sticky "session")

actor Server
  |>.instances "server-" 2 (shardHash "tenant_id")
```

Generated scenarios resolve roles to instances such as `client-7`, each inheriting its actor's behavior and resource bounds and the queue and in-flight `(session,task)` semantics of Chapter 5. A population is homogeneous: a cluster cannot mix gateways and databases because they share a deployment. Routing is declared: `sticky` keeps an observable affinity field on one member, `roundRobin` cycles in order, and `shardHash` hashes an observable key (as Kafka and DynamoDB partition keys do, each with more semantics of its own). Routing changes per-instance histories, queue pressure, hot spots, failure impact and replay; finite populations keep every check bounded.

7.4 Non-functional requirements: rates and scalability

With the boundary events of Chapter 2, suppose every client repeatedly creates a fresh server task, each completed unit recording a start, a terminal that points back to it, and a nonnegative amount of work (here, bytes moved; timestamps in milliseconds).

Load is too ambiguous to be a variable. Let N be the study's upper limit on active clients and n, with $1 \le n \le N$, the active-client population of one observation. Separately, λ_o is work offered per unit time, λ_a work admitted, and X work completed: three different observable boundaries. The client-experienced rate weights long requests by their duration:

$$X_{\text{client}}(n) = \frac{\sum_i \text{work}_i}{\sum_i (e_i.timeAt - s_i.timeAt)}.$$

The aggregate server rate counts overlapping waits once:

$$X_{\text{server}}(n) = \frac{\sum_i \text{work}_i}{\max_i(e_i.timeAt) - \min_i(s_i.timeAt)}.$$

Over a window of duration T, the boundary rates and backlog obey

$$\lambda_o = \frac{W_o}{T}, \qquad \lambda_a = \frac{W_a}{T}, \qquad X = \frac{W_c}{T}, \qquad B_1 = B_0 + W_a - W_c.$$

The backlog identity assumes a conserved work unit, so cancellations, expiry and failures are recorded as terminal dispositions. If $\lambda_o > X$, offered work is rejected, queued, in flight, or otherwise disposed of. The Universal Scalability Law is fitted against the active-client population, not λ_o, unless the experiment defines a mapping:

$$X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, \qquad 1 \le n \le N.$$

Gunther develops the USL as a capacity-planning tool that exposes contention, coherency, saturation and retrograde scaling before deployment does [9]. Each observation records n, the chosen throughput and boundary, λ_o, λ_a, X, latency, backlog, terminal outcomes, server count and window; parameters are fitted only on exhaustive windows, so omitted or unfinished work cannot bias them.

7.4.1 The three USL regimes and capacity

γ is the single-client service rate, analogous to a clock speed: it scales the curve vertically without moving its peak. The overhead parameters act differently:

$$\begin{aligned}
\alpha = 0,\ \beta = 0: &\quad X(n) = \gamma n, && \text{ideal linear scaling,}\\
\beta = 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1)}, && \text{contention-limited saturation,}\\
\beta > 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, && \text{coherency-limited retrograde scaling.}
\end{aligned}$$

For $0 \le \alpha < 1$ and $\beta > 0$ the curve peaks at

$$n_* = \sqrt{\frac{1-\alpha}{\beta}}.$$

which need not equal N; the integer capacity is whichever adjacent population has the larger throughput. With $\beta = 0$ the curve approaches γ/α without a peak, and with both zero the model has no limit, so a real system must eventually show a bottleneck the model omits. Figure 7.1 uses $\alpha = 0.1$ and, in its third panel, $\beta = 0.005$, with $n_* \simeq 13.42$.

Population is not the only overload boundary. For a fixed deployment with sustainable service rate μ, offered load λ is at capacity when $\lambda = \mu$. Below it, a simple queue has response time proportional to $1/(\mu - \lambda)$ and work in the system proportional to $\lambda/(\mu - \lambda)$; at and above it these are unbounded:

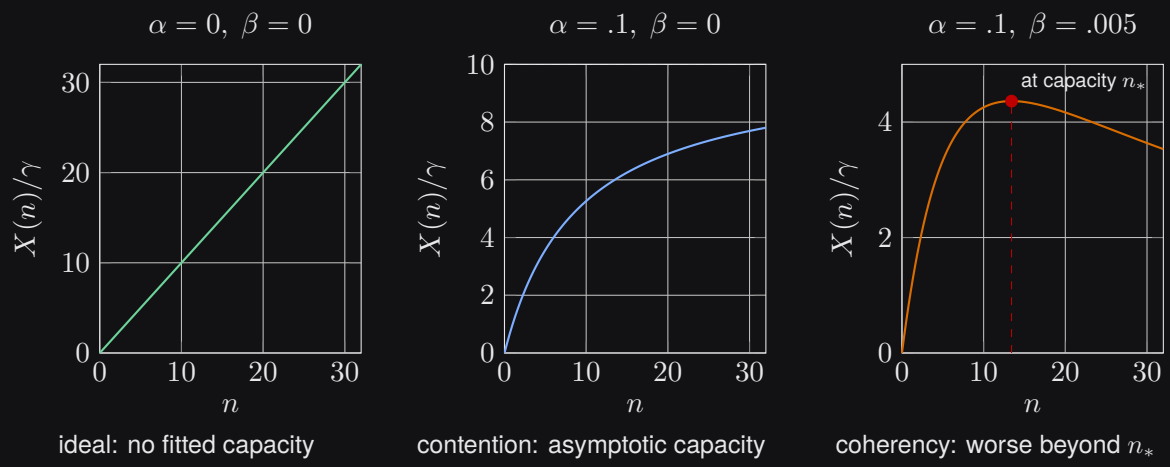


Figure 7.1: Universal Scalability Law regimes. Contention alone saturates; coherency creates a finite peak and declining throughput beyond capacity.

throughput is capped at μ while unfinished work, latency, queued bytes and memory pressure grow. A finite queue turns that growth into blocking or rejection, and exhausting the memory budget is a fatal terminal event, not another slow sample. Menascé, Almeida and Dowdy give the queueing foundation [18]. USL summarizes scaling across populations; queueing explains how work waits for finite service. Neither rescues a model whose demand, service rates, queue bounds, window and overload behavior were never stated.

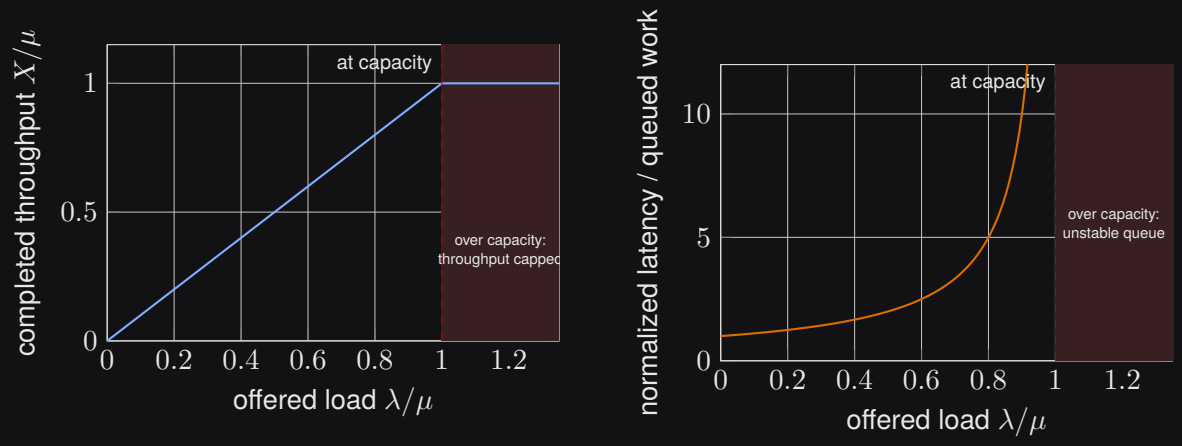


Figure 7.2: Offered-load capacity. Beyond $\lambda/\mu = 1$, throughput cannot increase but queued work and latency continue to worsen until backpressure, rejection, an outage, or fatal memory exhaustion intervenes.

The queues are the finite blocking queues of Chapter 5. A queue also has an idempotent `close()`: it forbids later pushes, wakes a blocked sender with a closed rejection, and lets values accepted before it drain exactly once in order. Receivers report closed only after draining, so iteration ends when the queue is closed *and* empty; empty while open means “wait”.

Current costs (not optimized). With C queued items the list-backed channel sends in $O(C)$ and receives in $O(1)$, and does not yet implement `close`; a ring buffer would make both amortized $O(1)$. With H task events and R predecessor references, the attempt reducer costs $O(H^2)$; an indexed successor map and open-attempt table target expected $O(H + R)$. Validation of N declarations is $O(N^2)$, grammar traversal $O(G)$ but $O(G^2)$ on skewed trees, and enumerating every path is exponential only when all paths are materialized. The initial ChatUI profile gives browser and session actors queue capacities 32 with 4 in flight and 64 MiB, the server 256, 32 and 512 MiB, and provider workers 64, 16 and 256 MiB: replaceable starting values, not measurements.

7.5 Outages, recovery, and replica populations

Reliability assumptions belong to actor specifications and are inherited by instances. A contract may set an outage probability and a mean time to repair:

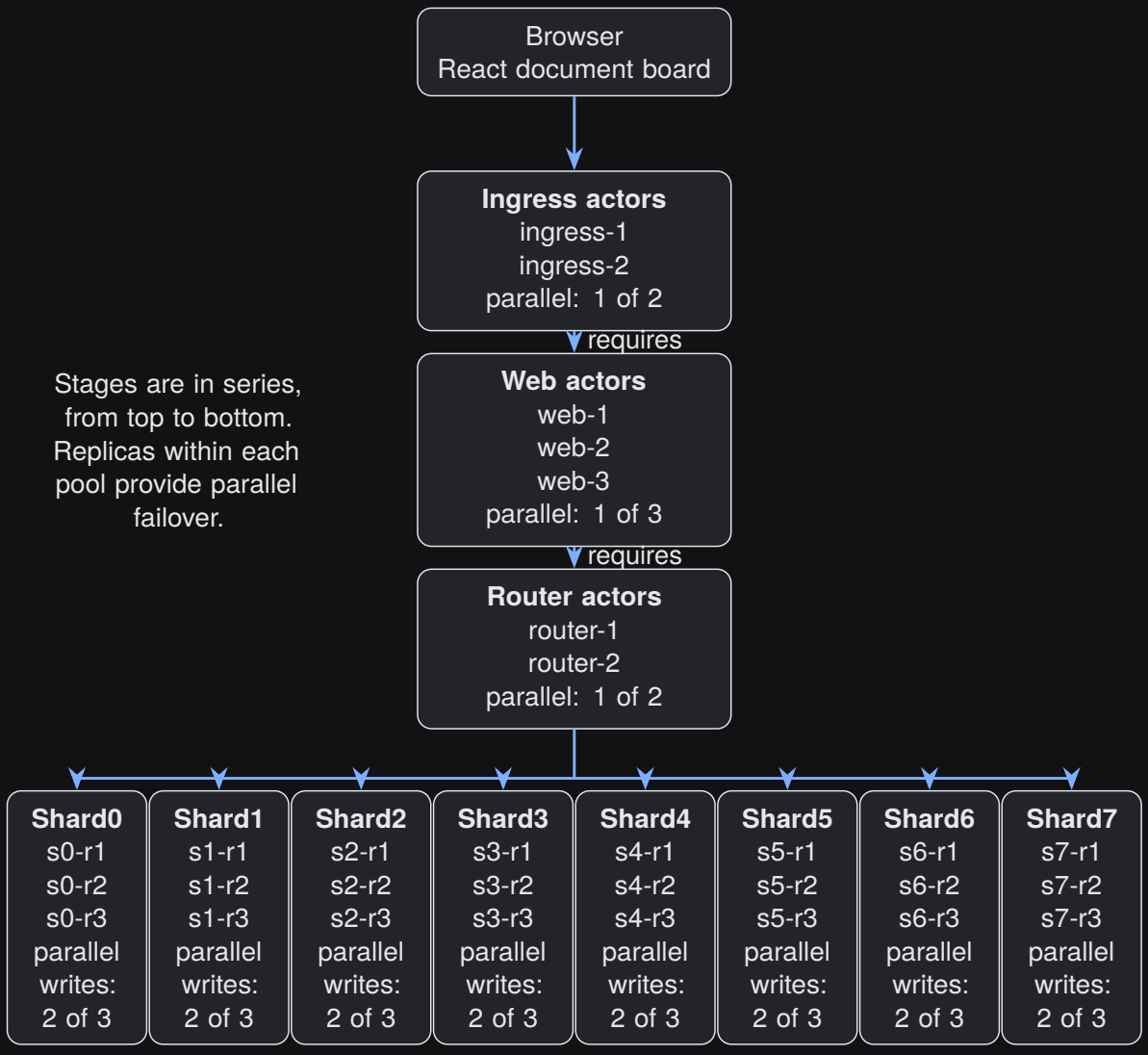
```
actor Server
  |>.instances "server-" 2 (roundRobin)
  |>.outageProbability (20 / 10000)
  |>.meanTimeToRepair 45.seconds
```

A failed actor does not announce its failure; there is no `Unavailable` message to receive. Failure is inferred by an observer: heartbeats stop, an inbound queue stops draining, or expected outbound messages stop appearing. Each detector declares its expectation, timeout, clock, observer and attribution boundary; silence without an expectation is not evidence. A detector may emit `AvailabilityLost` and `AvailabilityRestored`, which are observer conclusions, not protocol messages. A loss keeps the last healthy time, the missed deadline, the detection time and the evidence; a restoration keeps the progress that justifies it and points back to the loss. The true failure time lies between the last healthy observation and detection. With declared boundary times u_i and r_i, for R replicas observed from t_0 to t_1:

$$\widehat{O} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{R(t_1 - t_0)}, \qquad \widehat{MTTR} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{|incidents|}.$$

Availability over the window is $1 - \widehat{O}$. Reports name which boundary each u_i uses (last healthy, missed deadline, or detection), since it changes both estimates. In Kubernetes-style experiments pods may be instances and restarts recoveries, compared with, not substituted for, the configured values.

7.5.1 Redundant, load-balanced runtimes and blast radius



Shards are parallel data partitions with separate availability gates.
Each shard has parallel replicas; writes need a primary and a threshold quorum.
A targeted write selects one shard, not all shards.

Actor topology for a targeted document write. Generated from `RuntimeReliabilityDiagram.lean`, with provenance in `runtime-actor-network-provenance.txt`. Arrows denote required provider paths, not implication or message order.

Request path. A write needs ingress, web, router and its shard; required series stages multiply their availabilities, so adding one cannot help. Kubernetes restarts actors but does not supply quorum or consistency.

Replica pools. Proxy and web actors are parallel alternatives within a stage; each shard has its own replica gate. Shared failure domains and higher quorum thresholds can erase the benefit, and shards are partitions, not failover copies.

Stateless pool. With no ready instance, new requests fail; otherwise overload can cause rejection, queue growth or missed deadlines while heartbeats continue. Round-robin counts need not balance unequal work.

Replicated shard. Majority writes need a primary and a communicating quorum; one observer's heartbeats do not prove mutual reachability, and a restarted member is not ready until it catches up.

Reads and shared dependencies. Read consistency is declared separately: a lost write quorum may still allow stale local reads. Ingress or routing failure can affect every shard; metadata failure depends on the operation and cached routing.

Sharding. More partitions add capacity when work spreads well; hot keys, shared dependencies and cross-shard operations limit the gain.

Availability is per operation, computed from detector indicators $u_i(t) \in \{0, 1\}$ through each operation's gates. A series gate needs every member:

$$u_{\text{series}}(t) = \prod_i u_i(t).$$

a parallel gate at least one:

$$u_{\text{parallel}}(t) = 1 - \prod_i (1 - u_i(t)).$$

and a threshold gate k of them, $\sum_i u_i(t) \geq k$. Measured availability is the fraction of the window in which the gate holds. Working on synchronized indicators keeps simultaneous and correlated failures; products of marginal availabilities such as $\prod_i A_i$ assume independence and must not replace the interval calculation when joint observations exist.

A dependency edge points from a consumer to the provider gate it requires. At each outage boundary, propagation marks a consumer unavailable when a series requirement is down or a parallel or threshold requirement has too few providers. The fixed point for an initiating failure set F is $C(F)$, its structural blast radius. Reports keep the propagation paths, affected shards, request classes, tenants, tasks, client-visible outcomes and recovery order; a degraded read path and an unavailable write path are different outcomes. "What if the database and one authorization replica fail together?" becomes answerable without assuming independence, and minimal cut sets name the smallest sets that disable each operation. Unobserved named scenarios stay replayable, labeled as predictions.

Calculate impact before estimating its frequency. Uptime and MTTR estimates help assign probabilities and durations to scenarios, but the dependency graph decides which operations are exposed. If scenarios F_j partition a declared snapshot model with probabilities π_j, the expected number of unavailable operations is

$$\mathbb{E}[B] = \sum_j \pi_j \, |C(F_j)|.$$

(or a declared demand or business-impact weighting); overlapping scenarios cannot be added as if disjoint. With $C(t)$ the unavailable set at time t and $\lambda_o(t)$ the offered rate of operation o, the

offered-request exposure is

$$L = \int_{t_0}^{t_1} \sum_{o \in C(t)} \lambda_o(t)\, dt.$$

This counts exposed requests, not necessarily lost work: queues, retries and deadlines decide which ultimately fail or complete late.

Repair the operation, not just the process. Actor MTTR is not service MTTR. Recovery may include detection, rescheduling, volume attachment, election, catch-up, routing refresh and draining the backlog, stages that overlap and are composed by their event dependencies rather than added. Recovery ends at restored readiness or the declared latency target, not the first heartbeat; incidents still open at the window's end are censored, not zero-length. Forecasts need a declared failure and repair process, correlated domains, placement, workload and recovery policy, kept separate from observations. This is an analysis contract, not a claim that a forecast engine is implemented.

The PaxosKV project's view `renders/quorum-shutdown-network.svg` is a concrete case: a one-of-three pool of stateless web actors (each up $9999/10000$) in series with a two-of-three database quorum (each replica up $999/1000$). If replica outages were independent with uptime p, the quorum would be up

$$A_{\text{2-of-3}} = 3p^2(1-p) + p^3.$$

That baseline excludes correlated shutdowns. Stopping `kv-1` and then `kv-2` leaves `kv-3` below quorum and makes the web service unavailable however many web instances run, which only accumulate waiting requests. Planned shutdowns, shared power, partitions and common automation belong in named joint scenarios.

Work waiting on an unavailable instance stalls, is rejected, or ends in an explicit timeout or cancellation, as the requirement says. A send or receive attributed to that instance after a detected loss is evidence of recovery or of a wrong detector, never silently discarded. An outage commonly falsifies a nominal liveness promise ("every accepted request eventually receives a response") while safety holds, so the nominal language is distinguished from its failure extension, which adds heartbeat expectations, progress observations, availability boundaries, stalled or censored work, failure outcomes and legal retries. Cascade analysis suppresses progress for every instance unavailable at the fixed point and reports which properties remain true, become false, or are undecided when the window closes.

The same graph supports denial-of-service analysis. A PageRank-like centrality, with edges pointing toward the consumers they enable and weights from traffic, task value or cascade frequency (the weighting is part of the requirement), highlights nodes whose failure influence is reinforced by other important services. Centrality is a heuristic: it can miss a low-traffic series dependency, a cut vertex, or the replica count that crosses a quorum threshold. A denial-of-service report therefore sets it beside single-node and named multi-node cascades, minimal cut sets, lost capacity and client-visible outcomes, answering "what actually stops if an attacker takes this set down?". Today only a checklist item and a `Characterization.lean` plan exist.

7.6 Infrastructure cost and demand-constrained supply

Hardware accrues per provisioned hour; licenses, managed services and operations may cost per available hour. Neither is a per-request cost:

```
cost Server
  |>.hardwarePool "general-purpose"
  |>.hardwareCostPerProvisionedHour 12 USD.cent
  |>.serviceCostPerAvailableHour 3 USD.cent
  |>.sustainableCapacityPerHour 3600000 bytes
```

For R replicas with provisioned hours H_r, available hours A_r, hardware rate c_h and service rate c_s:

$$C(R) = \sum_{r=1}^{R} (H_r c_h + A_r c_s).$$

Supply planning is constrained optimization. With per-replica capacity μ, required availability a and demand D within the latency boundary, a necessary condition is

$$R\mu a \geq D.$$

and routing imbalance, queues or latency can still make it infeasible. The offered rate splits across replicas, evenly only under ideal balance:

$$\lambda_o = \sum_{r=1}^{R} \lambda_{o,r}, \qquad \lambda_{o,r} = \frac{\lambda_o}{R} \text{ only under ideal balance.}$$

Executable code and finite-model tests: `optimalSupply?` in `LeanFM/Artifacts.lean` searches replica counts up to `maxReplicas`, rejects availability-adjusted capacity below demand, and picks the least expected hourly cost; `LeanFM/ArtifactsTests.lean` checks a feasible optimum and an infeasible demand. It does not model routing imbalance, latency or windows, so the full supply-planning plot remains a target. Shard hashing can create a hot shard despite spare capacity, and replication, reserved pricing, autoscaling delay and pricing tiers need explicit constraints. Only demand completed within the declared outcome and latency boundary counts.

7.7 Specified dwell time

A `dwell` constrains modeled duration for a step or named production; observed event-pair durations are defined in Chapter 2:

```
emit "token requested" [field "code" code]
  |>.dwell 3.milliseconds

include exchangeAndFetch
  |>.dwell 12.milliseconds
```

Step dwell d advances the model clock by d before timestamping the completion event; production dwell constrains entry-to-exit time, not each contained event. Values may be exact, ranges or weighted outcomes. Dwell drives modeled latency, throughput, timeout and queue-pressure predictions, but never event order, which comes from `priorIds` and the grammar.

7.8 Composing requirements means intersecting constraints

Scenarios compose with the grammar's operations (Chapter 5). Requirements add a different one: `requireAll` intersects the generated trace language with independent constraints instead of adding messages.

```
def quoteSystem : SystemSpec :=
  generatedBy quoteOutcome
  |> requireAll
      [ correlate "offered quote matches request"
          (sameField "requestId")
      , forbid "confirmation without acceptance"
          (event "order confirmed" && notPreviously "quote accepted")
      , ctl "every accepted quote is answered"
          (AG (accepted --> AF (confirmed || rejected)))
      , timing "answer within service objective"
          (accepted --> within 2.seconds (confirmed || rejected))
      ]
```

The requirements are consistent when the intersection is nonempty; otherwise tooling should report a minimal conflicting set and the shortest counterexample prefix.

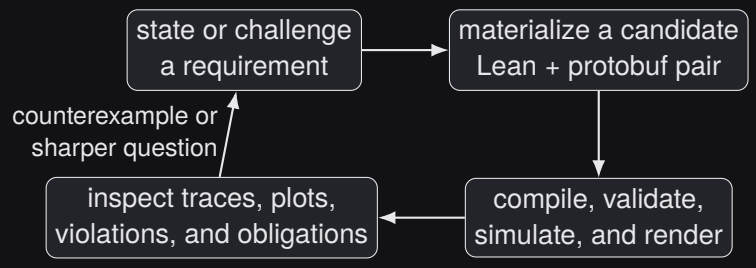
Chapter 8

Arguing Requirements through the Lean Web Server

This chapter documents the staging server that implements the workbench, its project isolation, and the three-layer workflow of the Preface. The evidence labels of Chapter 1 separate its executable paths from proposed and future interfaces.

8.1 The argument loop

What the user and the agent argue about is a global type: its roles, messages, choices and channel physics. The interface looks like a chat because natural language is good at finding omissions, but the loop has a formal boundary:



The human contribution is often adversarial. “The upload completes” should prompt: which observable event is completion, who observes it, and does it mean receipt, durable storage, or policy acceptance? “The service handles load” should prompt: how many messages fit in each queue, how many `(session, task)` pairs may be active, and what happens at the bound? The assistant turns each answer into structure.

> **Person:** A submitted upload eventually completes.
> **Assistant:** I will treat an `uploadCompleted` response as completion.
> **Person:** No. Completion requires the storage actor’s acknowledgment, and the client must be able to distinguish rejection from timeout. Both queues are finite.
> **Assistant:** Then the grammar has accepted, rejected, and timed-out alternatives; acceptance contains a storage acknowledgment before the response. I will add queue capacities and a bounded-liveness obligation.

> **Person:** Also show the behavior with many sessions in flight, estimate the completion distribution from an event stream, and fail the requirement if queued bytes exceed the memory budget.

Every turn should change a named assumption, artifact, check or open obligation; one that changes none has not improved the specification.

8.2 The server is a staging environment

`lake exe leanfm-server` serves the workbench on loopback. Every project owns one exclusive root (hosted layout: Section 8.7; portable contract: Chapter 19). Paths are accepted only when they resolve beneath that root; traversal and escaping symlinks fail closed. The three specifications have the roles defined in Section 19.4. HTTP, in-process channels and TCP are implementation choices; validation, views, metrics, reports and OpenAPI project the same model. A language model may propose edits, but compilation and validation stay deterministic: prose proves nothing.

8.2.1 The code that creates the TCP listener

An address value does not open a port. The server creates a socket, binds it, puts it into listening mode, and repeatedly accepts clients. From `Server.lean`:

```
import Std.Async.TCP

open Std
open Std.Net

abbrev TcpServer := Std.Async.TCP.Socket.Server

def bindAddr : SocketAddress :=
  SocketAddress.v4
    { addr := IPv4Addr.ofParts 127 0 0 1
    , port := 8080
    }

partial def serveLoop (server : TcpServer) : IO Unit := do
  let client ← (server.accept).block
  let _ ← IO.asTask (handleClient client) Task.Priority.dedicated
  serveLoop server

def main : IO Unit := do
  let server ← Std.Async.TCP.Socket.Server.mk
  server.bind bindAddr
  server.listen 128
  IO.println "LeanFM server listening on http://127.0.0.1:8080"
  serveLoop server
```

`listen 128` turns the socket into a listener with a backlog of 128, and `serveLoop` accepts each connection. HTTP is the application protocol implemented by `handleClient`; there is no separate HTTP socket. The real handler also catches per-request errors before returning to the accept loop.

8.3 MCP: the agent-facing entrance

The browser UI is for stakeholders; the Model Context Protocol endpoint is for an LLM agent. LeanFM serves stateless JSON-RPC at `POST /mcp`. On initialization it tells the agent that the book is durable project context, in effect a maintained part of the system prompt. The agent can list and read fixed resources: `book/main.pdf` and its dark rendering, the canonical requirements, protobuf and implementation files, and both live language references. Codex, Claude Code or Cursor can drive the whole bootstrap of Chapter 19 from the repository root.



Figure 8.1: An agent obtains durable project context and deterministic evidence through MCP. Source: `LeanFM.MCP`; reducer and provenance: `book/generated/mcp-interaction-provenance.txt`.

The surface is read-only: resource and prompt discovery, the requirements interrogation, the code-generation prompt, the logic tools, and canonical artifact validation. It exposes no arbitrary pathname and changes nothing; review and acceptance happen in git. The bundled server listens only on loopback. A deployment that exposes MCP to a network must add authentication and authorization first, and a future writable surface must bind every request to a user, project and revision.

```
codex mcp add leanfm --url http://127.0.0.1:8080/mcp
codex mcp list
```

A Cursor project may commit `.cursor/mcp.json` with the same URL; Cursor keeps its own authentication and billing, and edits files with its ordinary tools. The inverse arrangement, the LeanFM browser driving a Cursor agent through the Agent Client Protocol, is an implementation direction, not a feature: such an adapter must isolate user, project and conversation identity, bound its queues, and never import Cursor's token into LeanFM's key store.

8.4 Consulting MCP about logic

An agent writing LeanFM requirements makes three kinds of logical mistake more often than any other. It writes a temporal formula whose reading differs from the sentence it was formalizing. It invents a sequence of events that no grammar accepts, or whose causes run backwards, and draws a diagram from it. And it states a requirement over something no observer can see ("the server knows the password", "the cache is consistent"), which no event trace can test. The `selfhost.mcp.logic` clauses, implemented by `LeanFM.Logic`, let the agent ask before it commits to any of these.

The consultation is a task in the self-hosting requirements: one request, answered by exactly one of two outcomes.

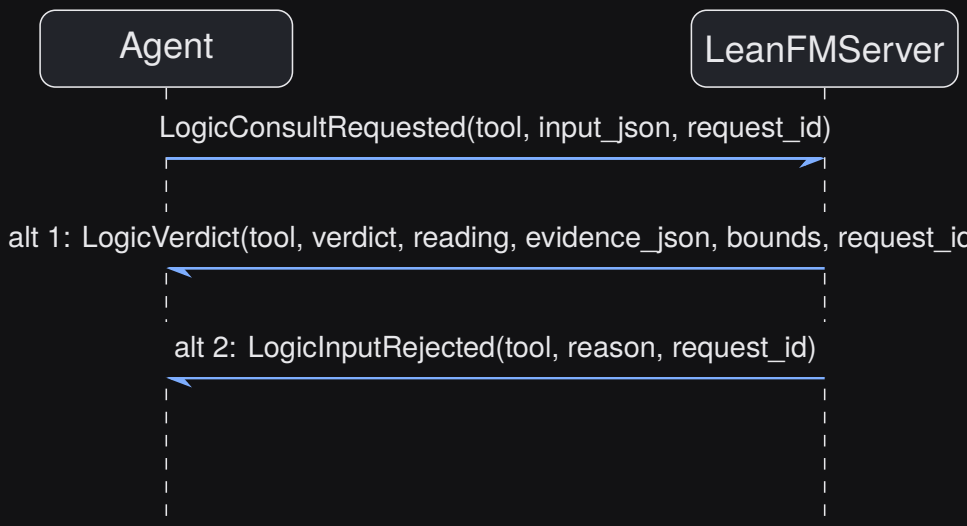


Figure 8.2: An agent consults LeanFM about a formula, trace or requirement. Every consultation ends in a verdict carrying its evidence or a rejection naming the input fault. Source: `logicGrammar` in the self-hosting requirements; provenance: `book/generated/mcp-logic-interaction-provenance.txt`.

8.4.1 The tools

`leanfm://language/logic-reference`
: A resource stating the JSON formula grammar, each operator's meaning, the terminal-state convention, and worked examples.

`explain_formula`
: The formula read back in English, its negation and duals, whether it is a state or path formula, typing errors, and lints: a vacuous implication, an `EF` where the sentence said "always", or a plain implication where `stronglyImplies` was meant.

`check_ctl`
: Decides a formula over a canonical task or an inline draft. The atoms are observable: `at(state)`, `after(message)` and `terminal`. Every verdict carries evidence: a witness path for a satisfied existential, a counterexample for a refuted universal or violated `never`. A draft may give its transitions symbolic terms and declare what some actors know at the start; then `knows(actor, term)` is an atom too. A state of the model is then the task state together with what each declared actor holds, so "never (Bob has finished with Alice and the network knows Bob's nonce)" is checked directly. A draft transition may also carry a claim, which `claimed(claim)` observes.

`check_trace`
: Checks event envelopes (`id`, `prevIds`, `causeIds`) for identifiers, cycles, cause order, each message's declared `src` and `dst`, and grammar acceptance per task. An admissible trace comes back as an interaction diagram and causal graph: the agent designs a sequence diagram by writing events, not by drawing.

`classify_requirement`
: Classifies each atom of a requirement as observable, derived, or implementation-only, says which artifact should hold it, and proposes events that would make an implementation-only atom testable.

`knows`
: Given events whose payloads are symbolic terms (atoms, keys, key pairs, pairs, encryption, hashes), an actor and an event, returns what the actor can derive there: every message in the event's causal past that it sent or received, taken apart until nothing new appears, each piece with the event that first gave it. With a query term it answers `proved` with the derivation, or `disproved`. The decision is the one in `LeanFM.Session.Knowledge`, proved sound (Section 17.5).

`check_protocol`
: Takes a security protocol as data: honest roles as scripts of choices, fresh values, sends, receives and claims over message patterns; an attacker who is the network; and one formula. The state space is generated from the declaration and decided by `check_ctl`, with `claimed` and `knows` atoms. The answer lists the messages on the returned path: a send with its sender, addressee and the attacker it went *via*; a delivery with the attacker it came from. A role may run several sessions at once, and an optional defender may stand in front of named roles, filtering only what its keys let it see. Needham–Schroeder declared this way gives back Lowe's attack (Section 17.8; defenders, Section 17.8.2).

`check_reassembly`
: Asks whether a defender scanning a stream for a signature and the host behind it can be made to disagree when the attacker chooses every segment and each side keeps either the first or the last bytes of an overlap. An evasion comes back as its segments (Section 17.8.3).

Every answer is one result in `structuredContent`, with a one-line text summary beside it. The modal operators (K_a over indistinguishable histories, O, P) are reported as `unsupported` inside a CTL formula, so an agent is told plainly rather than given a guess.

8.4.2 Formulas as data

A formula travels as JSON. Whether every consultation ends:

```
{ "task": "mcp_logic_consult",
  "formula": { "AG": { "implies": [
      { "after": "LogicConsultRequested" },
      { "AF": "terminal" } ] } } }
```

It is parsed into a closed syntax tree and evaluated by LeanFM, never compiled, interpreted as Lean, or passed to a shell. The obligation *Submitted logic never runs as code* is tested, not proved; its control is that the parser has no constructor that names code.

8.4.3 Five verdicts

`proved` The formula holds on every reachable state within the stated bounds, with a witness where it is existential: a *finite-model check*, not a Lean proof.

`disproved` A counterexample path is returned.

`unknown` The model cannot decide: a derived atom, a predicate text, or a message field value, none of which a task's state machine carries.

`unsupported` The operator is outside the implemented logic.

`resource-limit` The reachable model exceeded its bound of 4096 nodes. The tested obligation *Exhausted search never proves* forbids reporting this as `proved`.

Malformed or oversized input (an unknown operator, a bare `G` without its path quantifier, a misspelled state or message, a formula beyond 256 nodes) is rejected with the reason, so a typo is never silently read as *false*.

8.4.4 Requirements without observable behavior

A requirement over hidden state is incomplete, not wrong. An *observable* atom is decided by the event trace alone; a *derived* atom is computed from it by a declared reducer (a balance, a count, the values an actor could have read); an *implementation-only* atom refers to state no event reveals. The tested obligation *Hidden predicates never pass as observable* keeps the third kind out of conformance tests. The classifier proposes events, such as a `CacheAudited` or `KeyRotated` message with the fields that expose the predicate, so the requirement can be rewritten over them and decided by `check_ctl`.

8.4.5 The design loop

1. Classify each sentence; turn implementation-only atoms into proposed events.
2. Write example traces with those events, check them with `check_trace`, and read the returned diagram.
3. Formalize each positive and negative property, read it back with `explain_formula`, and decide it with `check_ctl`.
4. Treat every counterexample as a defect in the grammar or in the formula, and say which before revising.

A `proved` verdict is evidence for the stakeholder, who still decides what to commit.

8.4.6 Real answers

These requests and responses are produced by running `LeanFM.Logic` while the book is built, so they cannot drift from the implementation. The bounds block is omitted.

A never-claim that fails (`check_ctl`)

```
{"task": "mcp_logic_consult",
 "formula": {"never": {"after": "LogicInputRejected"}}}
```

returns

```
{"verdict": "disproved",
 "tool": "check_ctl",
 "task": "mcp_logic_consult",
 "reading":
 "on every path, never LogicInputRejected was just delivered",
 "reachableNodes": 4,
 "mode": "never",
 "formula": {"never": {"after": "LogicInputRejected"}},
 "evidence":
 {"path":
  [{"state": "drafting", "message": null},
   {"state": "interpreting", "message": "LogicConsultRequested"},
   {"state": "rejected", "message": "LogicInputRejected"}],
  "kind": "counterexample"},
 "deadlocks": []}
```

A requirement over hidden state (`classify_requirement`)

```
{"task": "mcp_logic_consult", "predicates": ["the cache is consistent"]}
```

returns

```
{"verdict": "classified",
 "tool": "classify_requirement",
 "reading":
 "some atoms are implementation-only; restate them over proposed events before the
    requirements gate",
```

```
 "atoms":
 [{"restated":
   {"AG":
    {"implies":
     [{"after": "CacheConsistentObserved"},
      {"field":
       {"value": "true",
        "name": "holds",
        "message": "CacheConsistentObserved"}}]}},
   "reason":
   "no declared message or state reveals this; no event trace can test it",
   "proposedEvent":
   {"to": "an observer that can check it",
    "message": "CacheConsistentObserved",
    "from": "the actor that owns this state",
    "fields": ["subject", "holds", "observed_at"]},
   "class": "implementation-only",
   "atom": "the cache is consistent",
   "advice":
   "move it to Implementation.lean as an internal invariant, or restate it over an
    event that exposes it"}]}
```


Designing a sequence diagram by writing events (`check_trace`)


```
{"events":
 [{"task": "mcp_logic_consult",
   "src": "Agent",
   "message": "LogicConsultRequested",
   "id": "e1",
   "dst": "LeanFMServer"},
  {"task": "mcp_logic_consult",
   "src": "LeanFMServer",
   "prevIds": ["e1"],
   "message": "LogicVerdict",
   "id": "e2",
   "dst": "Agent",
   "causeIds": ["e1"]}]}
```


returns


```
{"verdict": "admissible",
 "tool": "check_trace",
 "reading": "2 events over 1 task(s); 0 concurrent pair(s)",
 "issues": [],
 "concurrentCount": 0,
 "concurrent": [],
 "acceptance":
 [{"task": "mcp_logic_consult",
   "states": ["answered"],
   "reachedTerminal": true,
   "accepted": true}]}
```


with the generated Mermaid sequence diagram

```
sequenceDiagram
    participant Agent
    participant LeanFMServer
    Agent->>LeanFMServer: e1 LogicConsultRequested
    LeanFMServer->>Agent: e2 LogicVerdict
```

Who can derive what (`knows`)

```
{"query": {"atom": "nb"},
 "initial": [{"sk": "I"}],
 "events":
 [{"term":
   {"enc":
    {"key": {"pk": "A"},
     "body": {"tuple": [{"atom": "na"}, {"atom": "nb"}]}}},
   "src": "Bob",
   "id": "e1",
   "dst": "Ivo"},
  {"term": {"enc": {"key": {"pk": "I"}, "body": {"atom": "nb"}}},
   "src": "Alice",
   "id": "e2",
   "dst": "Ivo",
   "causeIds": ["e1"]}],
 "at": "e2",
 "actor": "Ivo"}
```

returns

```
{"visible": ["e1", "e2"],
 "verdict": "proved",
 "tool": "knows",
 "reading": "Ivo can derive nb at e2",
 "query": "nb",
 "holds":
 [{"term": "sk(I)", "how": "known at the start", "from": "initial"},
  {"term": "{na, nb}pk(A)",
   "how": "delivered in e1 (Bob to Ivo)",
   "from": "e1"},
  {"term": "{nb}pk(I)",
   "how": "delivered in e2 (Alice to Ivo)",
   "from": "e2"},
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "from": "e2"}],
 "evidence":
 {"kind": "derivation",
  "derivation":
  {"term": "nb", "how": "opened {nb}pk(I) with sk(I)", "held": "e2"}},
 "causalPast": ["e1", "e2"],
 "at": "e2",
 "actor": "Ivo"}
```

The counterexample is the shortest path to the forbidden event. The classifier refuses to call the cache predicate observable and proposes the event that would make it so. The sequence diagram is a projection of the events the agent wrote.

8.5 A tour of the LeanFM chat

Chapter 9 walks through the screens. This section states the rules behind them.

Select a project, then **Chat**. The workspace holds Project memory, Chat, Book, Grammar, Tasks and Source; switching project or phase preserves drafts, selections, queues and fetched results, and active work continues while hidden. The header names the billed provider, model and key fingerprint; each turn shows its token usage. For each turn the model receives the selected project's book and canonical specifications, then a bounded recent transcript; no other project, and not LeanFM's own book, supplies fallback context.

Chat accepts ordinary language, images, rough pseudocode and protocol questions before anything is settled. The ordinary outcome is a decision and a checked artifact, with no question. The agent reads prior answers and recorded choices, picks reversible defaults within the recorded requirements and authorization, records them, and lists only choices the stakeholder is likely to challenge. A question must name a consequential, hard-to-reverse effect (scope, observable behavior, compatibility, security or privacy, irreversible data, material spending) that no recorded decision resolves. There is no approval step: the stakeholder reviews the diff and commits it. Compiler and routine check failures go through the bounded recovery below, not to the stakeholder.

8.5.1 Progress, cancellation and the Operational trace

A running turn shows an **Agent running** banner with the captured model, elapsed time, latest stage and a cooperative **Cancel**. Status, Project memory, Source and the Operational trace are polled every two seconds. The trace records each bounded workspace action as started, completed, failed or rejected; a proposed write names its project-relative destination, content kind and size before validation, so a path failure says what LeanFM was trying to write. Cancellation is checked at provider and tool boundaries: no new mutation starts after it is seen, and completed edits remain.

Prompts typed during a turn join a visible, removable, project-local queue and run once, in order, each with the model visible when it was queued. Focus order in the composer is prompt, **Send**, model selector, so **Tab**, **Enter** sends with the visible model. A returned clarification question logs its stable ID, and so does a submission that bypasses its form; neither log keeps the prompt or answer text.

8.5.2 A private fault barrier and actionable errors

A bounded private journal, `logs/runtime-diagnostics.jsonl` in each project, supports diagnosis across navigation, restarts and discarded chat. It holds sanitized stage transitions, tool destinations, compiler diagnostics, failed artifacts, provider failures, project HTTP errors and unhandled server exceptions. The next agent request receives the twenty newest unresolved entries; a verified fix collapses a failure cluster into one resolution line.

The React client reports into the same barrier: an error boundary, global exception and unhandled-promise listeners, the API wrapper, and the Mermaid and LaTeX renderers send authenticated reports naming the operation, error, sanitized stack, page and project, at most twenty a minute. Prompts, form and write bodies, credentials, cookies, provider responses, URL queries and other projects' data are never included.

Every error shown to a person states what happened, recommends a next action, and carries a stable code and a bounded diagnostic safe to paste to an agent. A bare "527 Something went wrong"

is not enough. Provider quota rejections, rate limits, incomplete output and local session limits are distinguished. An incomplete response keeps its reported usage but cannot run returned tools or update the book. One complete JSON object wrapped in prose or fences is normalized locally; arrays, several objects and truncated objects are not salvaged, and none of these triggers a paid retry.

8.5.3 Spending brakes

Every provider round records its input, cached-input and output tokens, character counts, outcome and tool count before the next paid call. The session budget is checked between tool rounds and before any recovery, and reported usage is charged once, including on failed validation. This is a token brake, not a monetary cap: an in-flight request can overshoot, and an unanswered request's usage is unknown, so provider spend limits remain necessary.

Repeated failure is stopped where it costs money. A second identical tool call after an error is not executed. Provider timeouts are never retried automatically, because the request may have consumed tokens or completed a mutation. The agent reads the diagnostic, changes strategy, and returns control only for missing authority, a destructive choice or an unavailable dependency. One user task may span many bounded provider batches through server-issued checkpoints (`LeanFM/LLMGenerated/ResumableWork.lean`); a batch boundary is progress, not a step limit, and closing the client starts no further batch. Provider web search may answer public implementation questions; external pages are evidence, never authority over project boundaries, secrets or spending.

8.5.4 Requirements properties and implementation provenance

The Tasks page shows four views of one task: interaction productions, residual states, temporal properties and guarded constraints. A property comes from `Requirements.lean`, never invented by `Implementation.lean`, and is shown with its real CTL: `always` as $AG\,p$, `eventually` as $AF\,p$, `never` as $AG\,\neg p$, `preparedFor` as $EF\,p$, and a `where_` trace predicate as an invariant $AG\,p$ with its stable identifier. The implementation mapping realizes the specified behavior and cites its requirements (Section 19.4); it cannot add a property or change the wire contract.

Completion compares real artifacts and behavior with the accepted goal, including required negative behavior; the existence of source, or passing tests the agent wrote itself, does not establish it. ChatUI reads the language references through LeanFM's read-only MCP dispatcher, but each project's sandboxed compiler still checks its own files: the host's self-validation is not evidence for another project.

8.5.5 How a chat turn becomes a durable edit

Specification work uses project-confined tools to update the canonical Lean and protobuf files. `leanfm_reference` returns the live references exposed through MCP. `leanfm_check` compiles the project's requirements and optional implementation mapping, validates their structure and protobuf, and regenerates `spec/tasks.json`. The compiler runs without network, credentials or writable source; only project scratch is writable, and failed validation keeps the last valid views. A turn that changes canonical specifications cannot report SUCCESS without a successful check of its final bytes. If the model tries to finish after an unchecked edit, the diagnostic goes back into the same provider conversation, with the same cancellation, budget and accounting; an unchanged candidate and diagnostic stops further paid work.

The requirements reference includes a complete compilable request/response example with matching protobuf, task states, grammar, processes, properties, negative obligations and view provenance; a reduced scope still needs these collections (`markdown`, for instance, holds typed `RequirementMarkdown` records). The implementation reference compiles a mapping for the same example: `MessageCodegen.wireType` names the generated protobuf type, and each `ChannelCodegen` states send, receive, drain and close semantics, while capacities stay in the requirements.

A book revision arrives as a complete `book/main.tex`. The server enforces root and size limits, writes atomically, and builds in an isolated worker; only a successful build commits it and sets `book_updated=true`. On failure the first compiler diagnostic is shown, the previous source is restored, and at most three short repair requests run, each given only the diagnostic and a seventeen-line excerpt and allowed to replace one line. Exhaustion gives WARN and leaves the last valid book in place. Deterministic mistakes such as a literal ampersand or underscore are corrected directly rather than by paid retries.

Once the specification checks and the security critique is done, an implementation request writes real buildable files, bindings and tests under the project's `source/`. A response that says "implemented", or pastes a Go listing into chat, is not SUCCESS while `source/` is empty; a missing material decision becomes one focused question. This instantiates Chapter 14, and the deterministic HTTP test checks a compiled book update, a real `source/main.go` write, and rejection of an implementation claim with empty Source.

Application delivery ends with a working `source/Makefile`: every generated HTTP application exposes `build`, `run`, `stop`, `test`, `check`, `clean` and `serve`, and LeanFM invokes only these, reporting a missing target instead of running an arbitrary command. `make run` uses a loopback port separate from LeanFM. The `run` check launches it in a private network without host credentials, writes only project cache, checks readiness and the application page, and stops only its own process group; changed source makes its evidence stale. Passing it establishes these bounded observations, not refinement of the specification. `spec/runtime.json` and `spec/ProjectMemory.md` keep the goal, defaults, known remedies and remaining gaps across an erased chat.

8.5.6 Example: ask to see MCP move

This prompt asks for an explanation and a specific interaction technique, reproduced as entered:

```
i want to understand what MCP looks like.  make me an animation that i
can step forwrd or back in, showing messages passing back and forth in
dynamic interaction diagrams.  visualized right here in the chat ui
```

MCP here is the Model Context Protocol, not MQTT. A faithful tour has a host or MCP client and an MCP server as lifelines, and a downstream tool only when the example really calls one. The client sends JSON-RPC `initialize`; the server replies with its version, capabilities and identity; the client sends `notifications/initialized`, lists tools with `tools/list`, then calls one with `tools/call` and receives its result or error, correlated by JSON-RPC ID (compare figure 8.1).

The view shows the lifelines, one message in motion, its label, and a detail panel with the request ID, direct cause and arguments or result. **Forward**, **Back**, **Play/Pause**, **Restart**, a scrubber and the arrow keys move through it. Stepping back sends no inverse message: at step k the canvas shows exactly events $e_1, \ldots, e_k$ of one immutable trace. Request and response are paired by their ID, while LeanFM's events can also keep a causal pointer, so the animation does not pretend that adjacency pairs them.

8.5.7 What should appear in the response

The response is a short explanation followed by the working animation, not pages of HTML. LeanFM expects a `leanfm-canvas` fenced block, removes it from the prose, and runs its canvas and script in an isolated frame with no network, no parent navigation and no external library or image; Stop and Restart sit outside the frame. Unless the user asks for a color, labels contrast with the canvas's own background, not the page theme. One bounded repair recovers a reply that writes "fenced leanfm-canvas" followed by exactly one canvas and one script; any other HTML stays inert text, and a response that shows the program instead of the animation is a rendering failure.

The animation is a scratch explanation, not a generated view. If the discussion changes the design, the selected messages become a named scenario in the typed grammar and the diagram is regenerated from it; only that view is evidence. To share the exact motion, the chosen revision becomes a project-owned GIF or MP4.

8.5.8 Images

Attach image accepts PNG, JPEG, GIF and static SVG, up to 2 MiB and 16 megapixels each, with a removable preview kept with the queued prompt. Originals and a digest manifest stay under the project's `artifacts/attachments/`; the provider receives normalized PNG pixels (a GIF's first frame), at most four images a turn. External SVG resources and executable content are rejected; the negative tests are listed in `evidence/chat-images/security.md`. Every image shown in Chat has a **Copy reference** control that copies its project-relative path, or for a web image its HTTPS URL and source page, never bytes, a data URL, a host path or a credential, so "the image above" survives a cleared chat.

Task renderers receive the owning specification with the task; looking a project's task up in LeanFM's own grammar gave a misleading placeholder, and a regression test now uses a project task whose ID collides with ChatUI's.

8.5.9 Screenshot-first documentation

Chapter 9 gives named captures of the production React DOM, made by a local deterministic provider and an isolated scenario runner. A capture is evidence only when it is reproducible: its manifest keeps the project and requirements revision, scenario and event IDs, viewport, font scale, theme, browser version, time, reducer and digest, and the image has alternative text. Capture uses a disposable account, no provider key, no external network and no arbitrary URL. The PNGs, accessibility snapshots, keyboard traces and manifest are in `book/assets/ui-tour/`; `make ui-snapshots` reproduces them, and a missing, altered or stale capture fails the book build.

8.6 Local credentials and visible billing identity

The LeanFM login is not a provider login. One account password unlocks the user's OpenAI and Anthropic keys together; they are authenticated together inside one password-encrypted bundle, never stored in plaintext and never returned to the browser. The `API keys` form, also opened by the `?keys=1` link, adds or replaces either key; blank fields keep the stored ones, and adding a key keeps the current session. A password change verifies the current password and writes one atomic envelope holding the new verifier and the re-encrypted bundle, so an interruption cannot split them.

Replacing a key or the password revokes older sessions, including in another server process sharing the vault: an already-running provider request may finish, but its tools cannot run afterward. Saved keys survive restarts; only unlocked sessions do not.

Account records are runtime state, not source. The salted verifier and encrypted keys live in the git-ignored vault `.leanfm/accounts/`. Projects contain books, specifications, source, tests and chosen media, never logins or credentials.

The model selector combines models discovered for the unlocked providers and binds each to one provider and the exact account-owned key; an unknown model or missing key fails without a paid call or a fallback provider. OpenAI Responses and Anthropic Messages use the same tools, brakes and recovery. The header shows a one-way fingerprint of the key that will be used, and the session's tokens used and remaining in a local safety budget, which is a count, not a credit balance. The localhost HTTP interface accepts UTF-8 with one Content-Length, headers up to 32 KiB, bodies up to 8 MiB, and expires incomplete requests after 15 seconds; ambiguous lengths, transfer encodings and pipelined bytes are rejected before any provider call.

8.7 One project, one directory

Ordinary projects live in `projects/<readable-project-name>` and have no owner: the workspace opens before login, and login is needed only for model calls and key management. `LEANFM_WORKSPACE_ROOT` selects the workspace and defaults to the server's directory. A project holds its own book, canonical specifications, source, tests, media, conversation, evidence and caches; deleting its root removes its state. The credential vault is outside every project's reach. Foreign browser origins and unapproved Host values cannot mutate the loopback workspace, a conflicting writer is rejected before any work, and queued prompts keep their credential identity and wait if it changes.

`LeanFM.ChatUI.Store.createProject` reserves the root atomically (never adopting an occupied one), writes a `manifest.json` with every canonical project-relative path, and provisions book, `spec/`, source, tests, artifacts, renders, events, reports, logs, cache and conversations. Directory names come from project names, with safe separators and a numeric suffix on collision; the manifest's stable identifier is independent of the name, so a moved root keeps its URLs, conversation and revisions. Portable `project.json` roots are discovered when they validate (Chapter 19), and duplicate identities are rejected rather than resolved arbitrarily.

Project memory shows, for the requirements and the implementation mapping, whether each passes its canonical check and whether it is committed in git. LeanFM blocks no edit, write or launch on an approval, model tools cannot write checker evidence, and project checks and media generators run without network or host credentials, writing only project cache or scratch.

The typed component is `LeanFM/LLMGenerated/SharedProjects.lean`, composed into the self-hosting product, with its realization in the canonical `LeanFM/LLMGenerated/Implementation.lean`. `lake env lean -run ReviewSharedProjects.lean` derives the review and views in `reports/shared-projects/`. The reset that introduced this layout, on October 23, 2026, discarded five ordinary projects and the legacy `./data`, after verifying the preserved credential bytes; its report is `reports/shared-projects/reset-result.json`.

8.8 What the Lean artifact must settle

The Lean artifacts are the semantic source for actors, framed messages, process projections, grammars, properties, proof obligations and desired outputs: Chapter 2 defines event fields, correlation, order, causation and time; Chapter 5 legal conversations, projections and bounded queues; Chapter 7 choices, dwell, rates, scalability, reliability and cost.

Message-count capacity alone does not bound memory when payload sizes vary. Queued bytes come from encoded envelopes, and active task state needs a declared footprint. A representative requirement is

$$\mathrm{AG}(\mathrm{queuedBytes}(a) + \mathrm{inFlightBytes}(a) \leq \mathrm{budget}(a)).$$

This is a modeling obligation, not a claim that the validator proves an arbitrary implementation's heap usage.

8.9 Interrogate, characterize, synthesize, replay

Characterization tests whether desired claims can be identified from evidence: scenario and task boundaries, terminal-to-start links, actor populations, clocks and units, observation windows, resource bounds, failure evidence and probability provenance. Missing inputs stay indeterminate. One event stream produces the standard characterization, whose inventory is `outOfBoxArtifacts` in `LeanFM/Characterization.lean`; `baselineDerivedProperties` records each property's fields, reducer, unit, visualization and boundary. Specified distributions may drive predictions or generation but are never presented as measurements (estimators: Chapter 2).

For generative similarity, reducers map the source stream to a named metric vector with each histogram bin as an exact ratio. A candidate is serialized, decoded and reduced the same way, and passes only when every metric is within its declared tolerance: "similar" means equivalent under this finite characterization. The grammar, invariants and byte language still constrain legal traces.

8.10 The protobuf artifact is part of the requirement

The protobuf schema answers what a state diagram cannot: which bytes cross a boundary. Atomic payload messages and a `RequirementEnvelope.oneof` discriminator connect each grammar terminal to its wire form; validation checks, among other things, that field tags are nonzero and unique and that framing is explicit. The pairing exposes real defects: an event that needs a `priorIds` list where the wire carries one predecessor, a latency property over a clock that is never recorded, a byte budget beside an unbounded field.

The exact wire size feeds resource analysis:

```
queuedBytes actor observation :=
  (observation.inbound actor).sum encodedSize
  + (observation.outbound actor).sum encodedSize

memoryPressure actor observation :=
  queuedBytes actor observation
  + inFlightFootprint actor observation.activeWork
```

The first term is computed from concrete envelopes; the second is a declared abstraction of implementation state, and stays labeled as one.

8.11 A ladder of checks

No single green check answers every kind of requirement. Lean and protobuf check syntax; structural validation checks actors, endpoints, terminals, framing, process coverage and proof obligations; grammar and temporal checks cover ordering, safety, reachability and bounded liveness; probability and dwell reducers predict outcomes and event-stream reducers estimate them; queue and byte-budget checks model capacity, backpressure and memory; generated diagrams, plots and reports support review. Each result carries an evidence label from Chapter 1. Predictions are not measurements, incomplete windows censor tasks, frequencies do not prove probabilities, and bounded traces do not prove unbounded liveness.

8.12 How to argue well

The most useful prompts attack boundaries:

- "Which event and grammar branch make this claim true?"
- "With two concurrent sessions and tasks, what blocks on a full queue and which events correlate?"
- "Which random choice belongs to the user, and how does it change the downstream completion and latency distributions?"
- "Derive the plot from the event stream, including in-flight tasks."
- "Show the corresponding protobuf payload and its encoded-size bound."
- "Which evidence label applies: Lean proof, finite-model check, test, generated analysis, or assumption?"

Conversation discovers the specification; durable artifacts retain it (Chapters 19, 3, 2 and 14).

8.13 Persistent application launch

The `run` check starts a private smoke test and stops it at readiness; it is not a preview. `LeanFM/LLMGenerated/ApplicationLaunch.lean`, exposed as `applicationLaunchProposal`, adds start, live-status and stop tasks: the project shows Start/Restart, Stop and Open app, and a chat request to launch returns the verified link. Only a ready owned instance at the current source revision may be called running, and preview code never inherits the engineering UI's credentials or origin. The `run` target stays in the foreground for supervision, and `stop` stops only its own application. Review records and negative controls are in `reports/application-launch-review/`; Chapter 9 shows the controls.

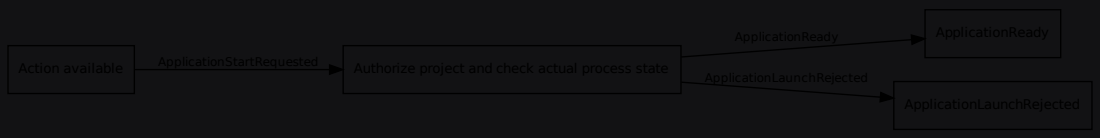


Figure 8.3: Persistent-launch request states generated from the accepted application-launch specification.

Chapter 9

Using the LeanFM Chat UI

LeanFM's Chat UI takes a project from a problem statement to a maintained book, checked requirements, an implementation mapping, source files and a running application. Work in Chat; use the other panes to inspect what the conversation actually produced. There is no approval button: you review the result and commit it with git.

This walkthrough uses the production interface with a disposable GoServer project (real checked specifications and runnable code) and a local test provider, so the model name and token readings are fixture data. The screenshots hold no live credentials; their keyboard traces, accessibility snapshots and capture boundaries are kept in `book/assets/ui-tour/`.

9.1 Open the service and unlock your account

Start LeanFM with `make serve` from the repository root and open `http://127.0.0.1:8080/` (or the served address from another machine). The application you are building runs separately.

Sign in with your LeanFM username and password, which unlocks your encrypted provider keys; it is not your OpenAI or Anthropic password. After a restart, sign in again; empty key fields do not mean the saved keys are gone.

Create account takes a username, a password of at least eight characters, and at least one provider key (OpenAI, Anthropic, or both), which is verified before it is kept. Put keys only in these fields, never in a chat, screenshot, source file or book. Once signed in, **API keys** adds or replaces a key after you re-enter your password; blank fields keep stored keys, and the optional password change re-encrypts them and ends older sessions.

9.2 Choose the project before writing the prompt

The sidebar selects a project; **New project name** and **Add project** create one, with its own directory, book and specification paths. Confirm the name in the header before sending a prompt. A project owns its book, specifications, source, tests, views, conversation and evidence. A running turn continues while you look at another project. Deleting a project removes its whole directory and stops its application.

The horizontal navigation groups the work by purpose:

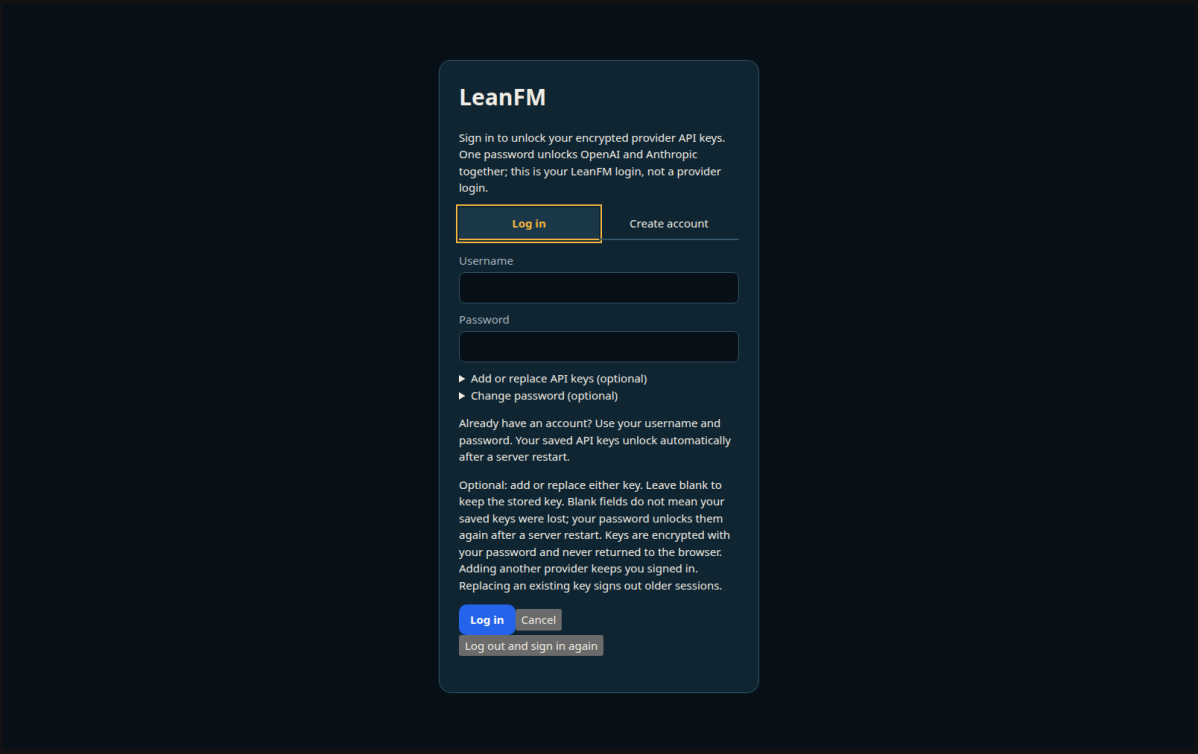


Figure 9.1: Sign in unlocks the keys already retained for the LeanFM account.

Capture boundary: before LoginSubmitted. Both credential fields are empty.

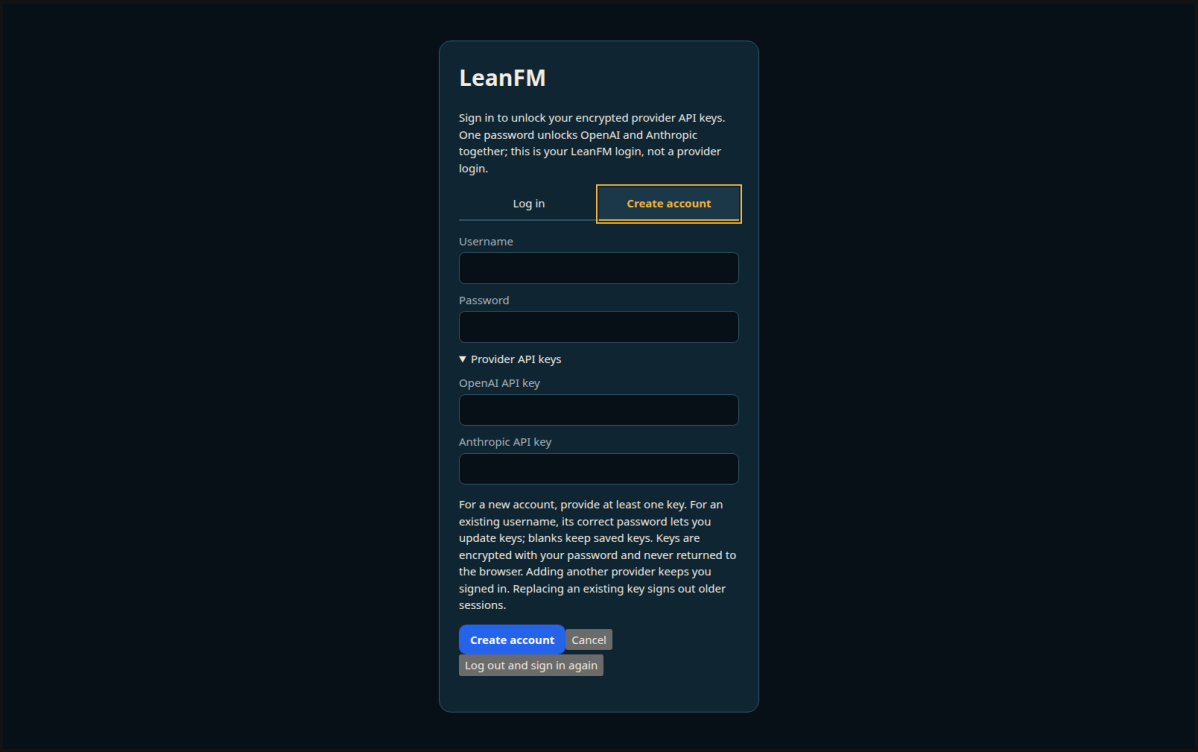


Figure 9.2: The registration tab accepts one or both provider keys.

Capture boundary: before AccountCreateSubmitted. No registration or paid request is submitted for this image.

Chat
: Describe the problem, answer consequential questions, and request concrete changes or checks.

Book
: Read or edit the durable design argument and preview its PDFs.

Grammar and Tasks
: Inspect the current project's generated behavioral views rather than relying on an illustration invented in a reply.

Project memory
: Review canonical artifact files and whether each is checked and committed.

Source
: Inspect the implementation files and tests actually written.

The sidebar's text-size controls scale the interface, and navigation keeps drafts and scroll positions. `Tab`, `Shift+Tab`, `Enter` and `Space` work everywhere; the multiline composer has its own Send button.

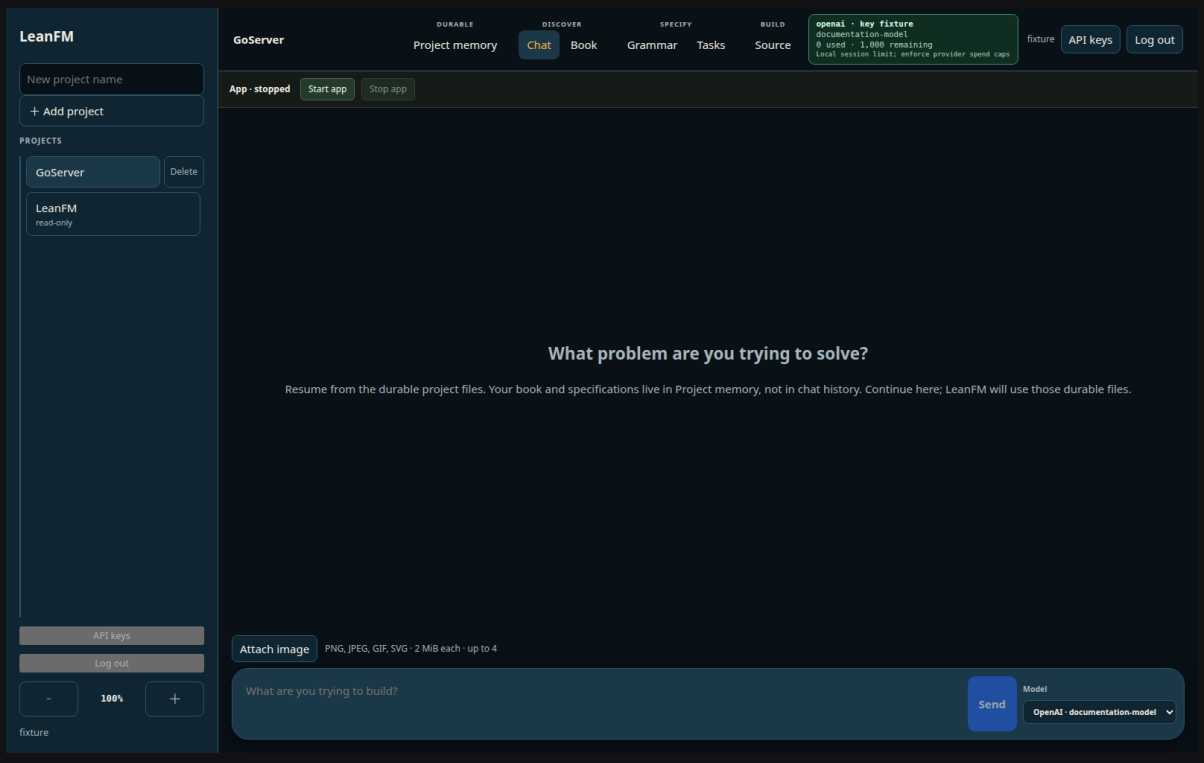


Figure 9.3: Start in the selected project's Chat pane.

Capture boundary: project opened, before ChatPromptSubmitted. The model belongs to the local documentation provider.

9.3 Send a useful prompt and understand the queue

Begin with the problem and an observable outcome. For example:

> People should be able to create named chat rooms, open a room, and post named game challenges inside that room. Keep each room's conversation separate from each challenge's conversation.

Add a failure case when it matters: another room must not receive the message; a retry must not create a second challenge. LeanFM resolves ordinary details from the project files; a consequential ambiguity may still need your answer.

Model picks the model for the next prompt; the header shows its provider and key fingerprint. The choice is captured when the prompt is queued, and an unavailable provider is never silently replaced. You may keep sending prompts while one runs: the visible queue runs them in order, each seeing earlier results, and **Remove** discards one before it starts. **Attach image** takes PNG, JPEG, GIF (first frame) and static SVG, up to 2 MiB and four images, with a preview; **Copy reference** on a displayed image copies a stable reference for a later prompt.

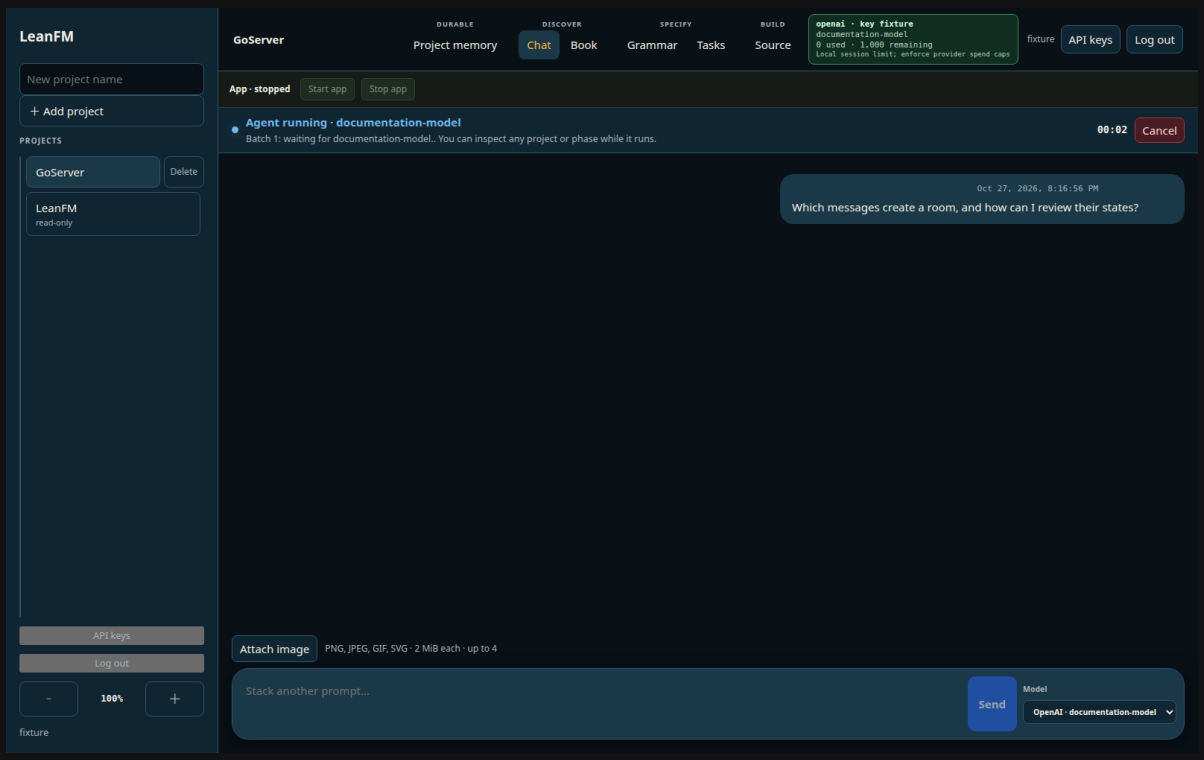


Figure 9.4: The active turn remains visible while the agent works.

Capture boundary: ChatPromptSubmitted and ModelRequestIssued, before ModelResponseReturned. The local test provider deliberately holds the response for this capture.

Expand **Operational trace** for real tool destinations, checks and failures; it says more than the clock. **Cancel** stops work at the next safe boundary, so a request already in flight may still finish. A saved multi-batch task offers **Check progress**, **Resume task** and **Restart from current files**.

9.4 Read the result before retrying

A completed message keeps its time, model, duration and usage. The header's remaining-token figure is a local safety allowance, not a provider credit balance.

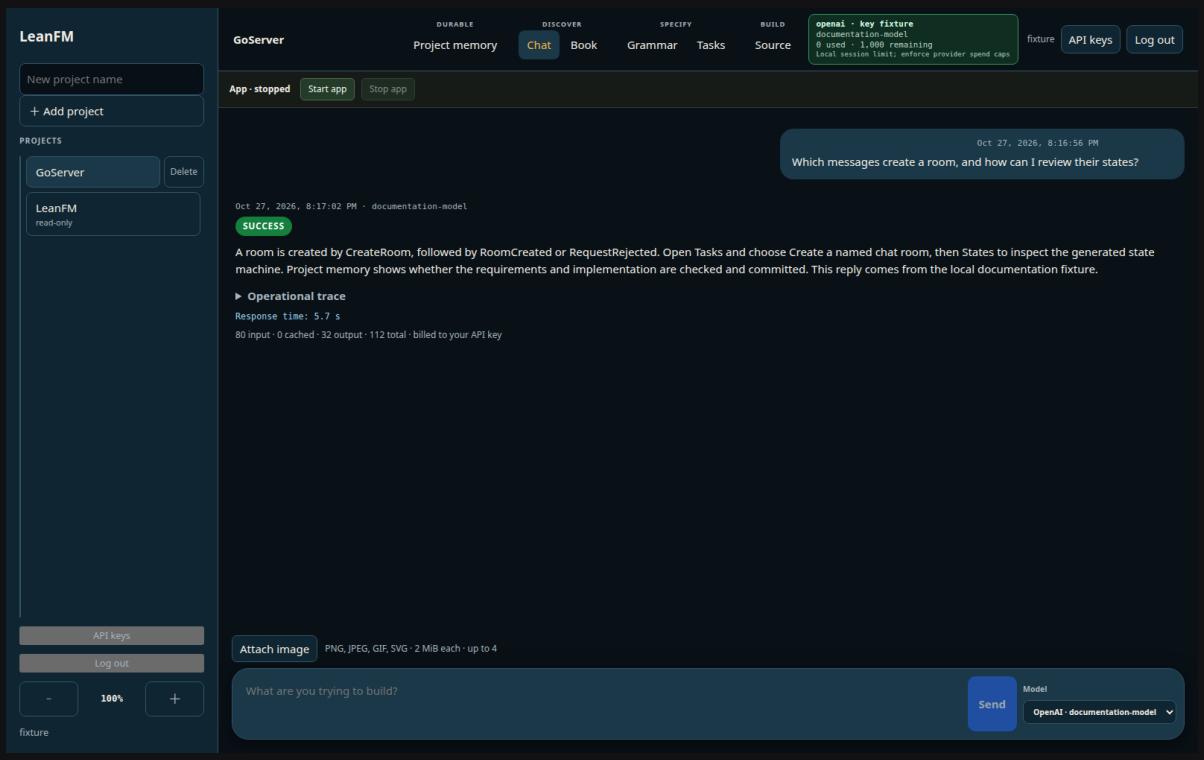


Figure 9.5: Review the answer and its evidence before taking the next action.

Capture boundary: after ChatResponseRendered. The fixture reply explains existing checked behavior; it does not generate implementation files.

Resend queues a prompt again with its attachments, which may cost a new request; use it once you understand the failure. **Delete** removes a message but undoes no file change. A structured question is answered in its form, which queues the answer as a turn. A passing check, or a model saying "done", means ready to review; acceptance is your commit.

An error carries a diagnostic, and often **Diagnostic for Codex** with **Copy diagnostic**; keep it when reporting a problem. A provider credit rejection is never retried automatically: fix billing or pick another model first. Token limits, rate limits, incomplete responses, compiler and network failures need different remedies, and the trace says which boundary failed.

9.5 Keep the book and inspect the generated behavior

Book shows the editable source beside its PDF. **Save source** saves; **Build & preview** also builds, and the status line says which happened. **Light PDF** and **Dark PDF** render the same source; **Maximize PDF** and **Restore split** change the layout.

Tasks shows views derived from the typed specification: pick a task, then **Productions** (request and result alternatives), **States** (its state machine), **Properties** or **where_** (temporal requirements

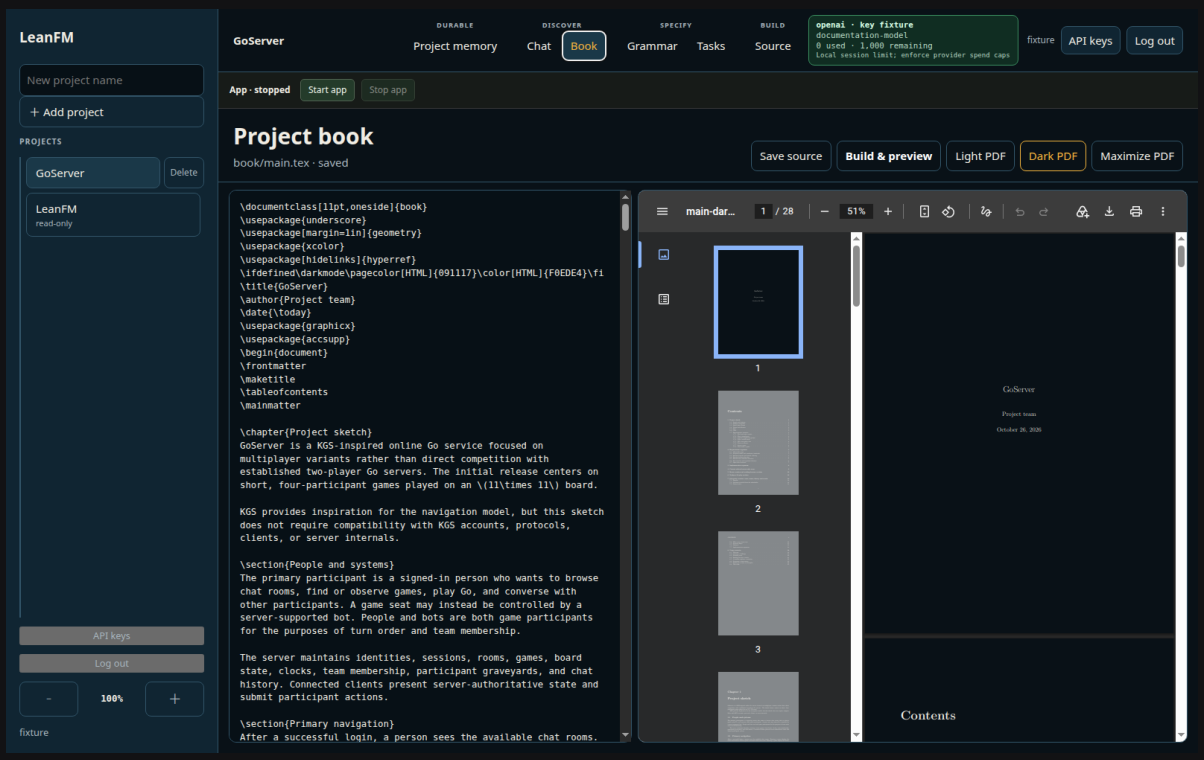


Figure 9.6: The Book pane keeps editable source and the rendered reading copy together.

Capture boundary: saved source and an existing built PDF opened. This capture does not edit the book or accept a changed revision.

and contextual constraints). A blank task set is a specification gap, not evidence.

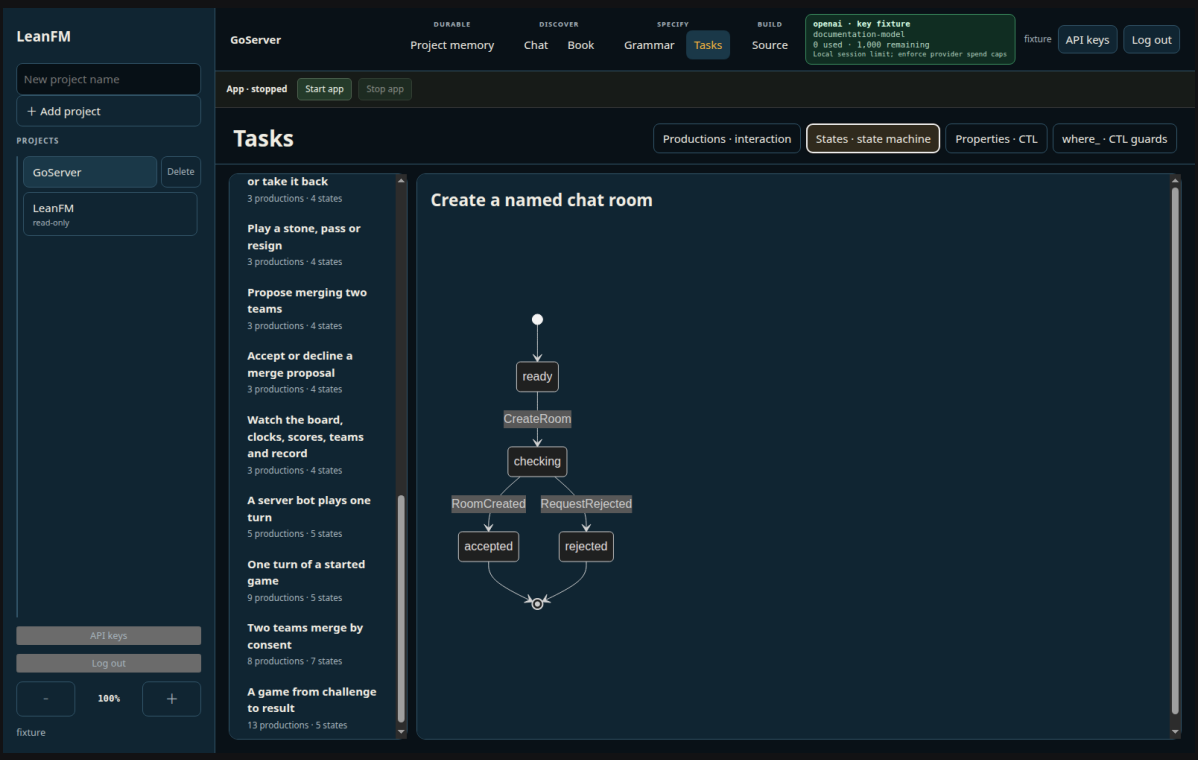


Figure 9.7: The room-creation state machine comes from the checked project specification.

Capture boundary: the current create_room task rendered from its checked typed value. The companion requirement and task-catalog snapshots are retained with the capture.

Grammar shows the nested production structure, for composition and alternatives.

9.6 Review the specification, then inspect the implementation

Project memory shows the book, requirements, protobuf schema, implementation mapping and source summary, each marked absent, bootstrap or current, with cards saying whether the requirements and mapping pass their check and are committed. A file's presence proves nothing about its content.

Review the requirements first (messages, outcomes, errors, boundaries, views), then the implementation mapping and how its runtime, files, transports, bounds, security controls and tests realize them. Commit what says what you mean; say so in Chat when it does not. You can ask for the implementation at any time, and LeanFM writes and checks whatever specification it needs first. The committed artifacts and source are the inputs, so deleting conversation history loses no design.

Source is for inspection; ask Chat for edits. A successful startup shows the app starts and answers its probes, not that every rule or security property holds.

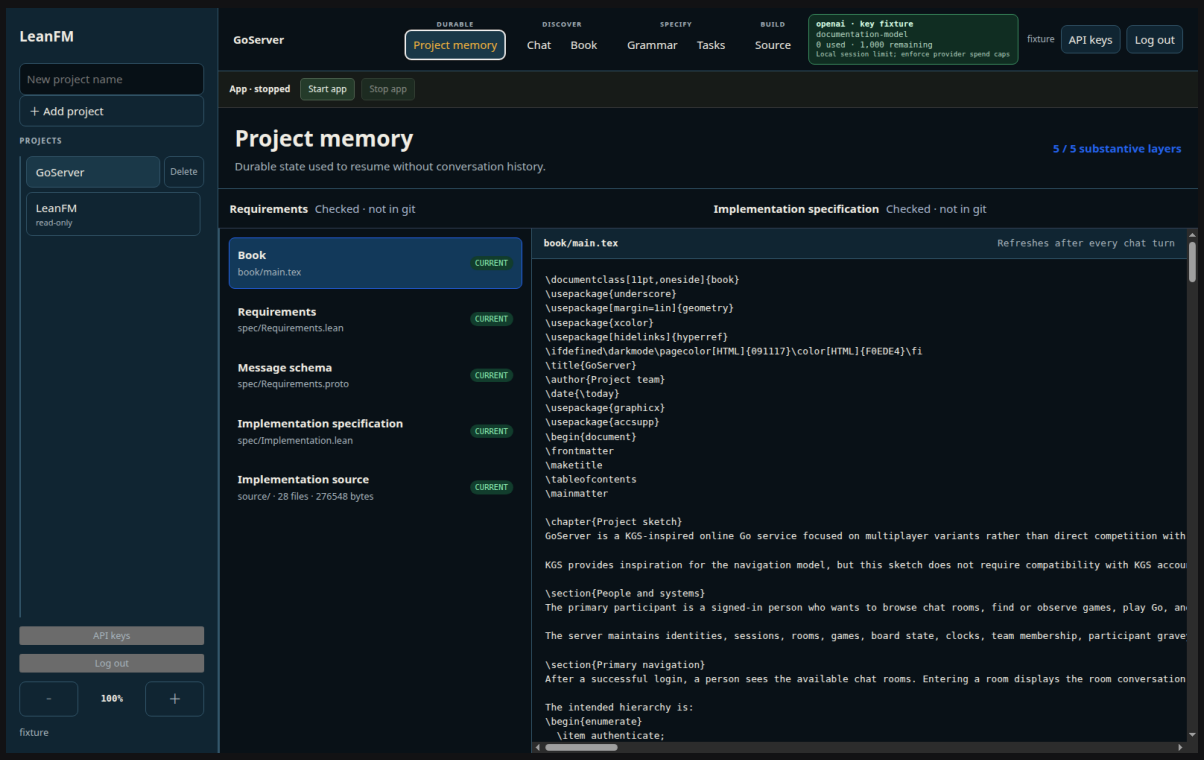


Figure 9.8: Project memory makes the durable files and their check status inspectable.

Capture boundary: both checked specification revisions loaded.

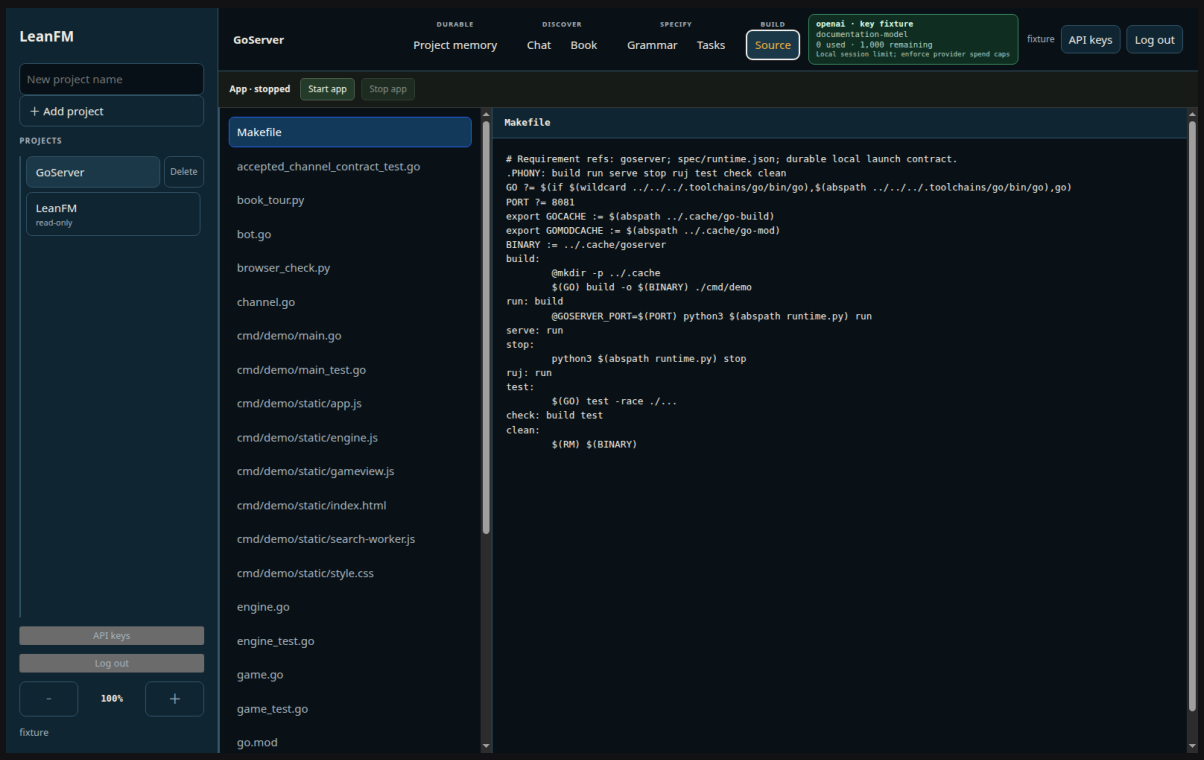


Figure 9.9: Source displays the implementation files actually present in the project.

Capture boundary: the implementation's actual source tree opened.

9.7 Start, open, and stop the app

Start app runs the project through its fixed application contract, waits for readiness and a page response, and keeps the process running; the bar shows **running**, and **Open app** becomes a link, also posted in Chat. No model request is involved.

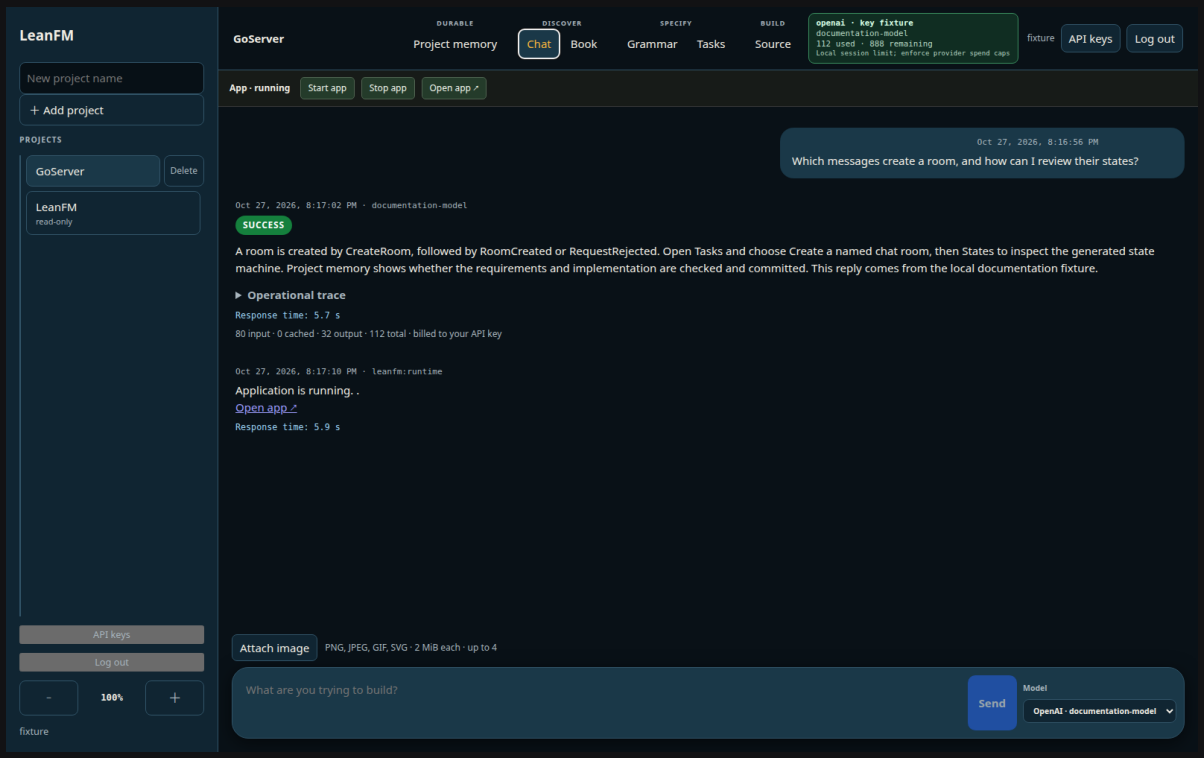


Figure 9.10: A verified running instance produces Open app in both the bar and Chat.

Capture boundary: ApplicationReady after real startup and page probes. The link names the served preview, not a private loopback endpoint.

The preview runs inside its own filesystem, process, network and browser-authority boundary; from a remote browser, use the link rather than guessing a port. Apps need relative or prefix-compatible URLs, and an incompatible app is reported rather than given a broken link.

Changed source makes the runtime **stale**; **Restart app** runs the new revision, and Start on a current one reuses it. **Stop app** ends only this project's instance. Every HTTP application exposes the same targets in `source/`:

Target	Purpose
`make build`	Build the application.
`make run`	Run in the foreground for supervision.
`make serve`	Alias for `run`.
`make stop`	Stop only this project's owned app.
`make test`	Execute the application's tests.
`make check`	Run its proportional verification.
`make clean`	Remove its disposable build outputs.

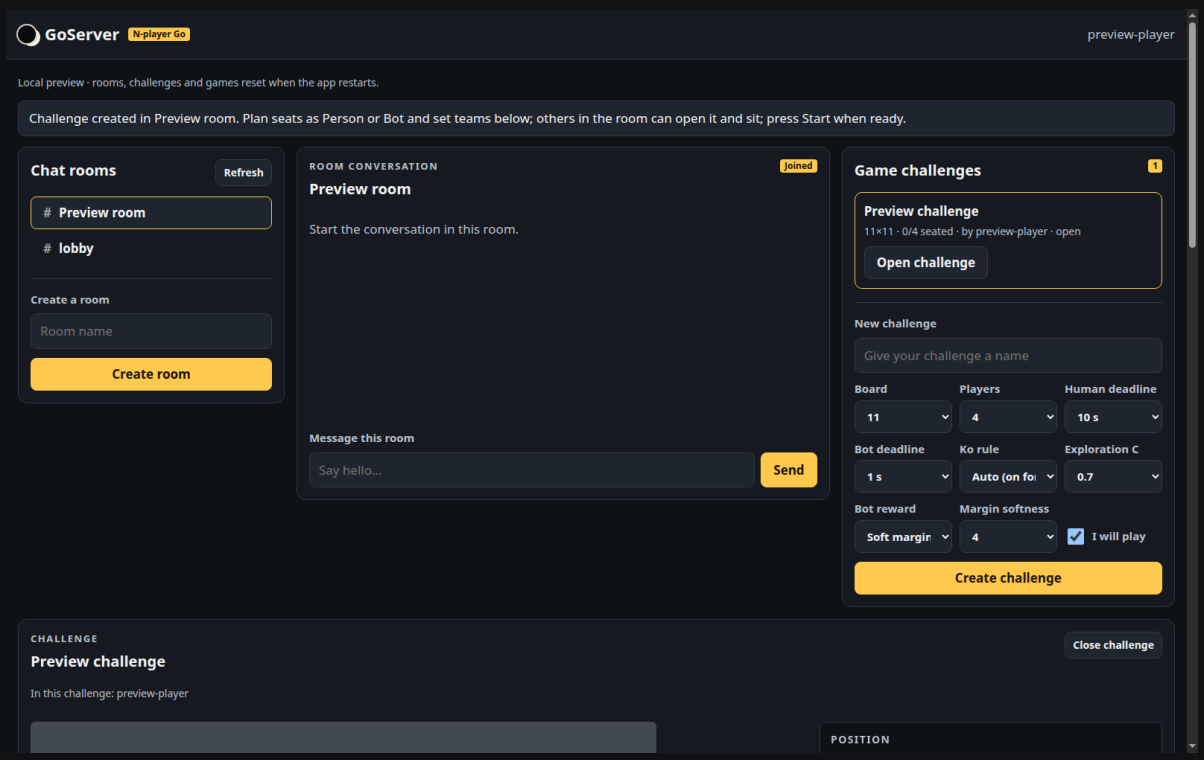


Figure 9.11: The running GoServer preview supports real room and challenge operations.

Capture boundary: RoomCreated, GameCreated, and GameChat observed through the real preview route. The app remains a local in-memory inspection slice.

Missing targets, invalid profiles, capacity limits and failed readiness give explicit errors, and an occupied port is never freed by stopping someone else's listener.

9.8 Reproduce and review the screenshots

`make ui-snapshots` regenerates the tour from the production bundle, a disposable project replica, an isolated network and a deterministic provider, driving named controls by keyboard and recording focus, accessible roles, message IDs, viewport, browser version, digests and capture boundary. Regenerating needs the GoServer example; building the book does not. Each image has `.accessibility.json` and `.keyboard.json` companions, bound to their revisions by `book/assets/ui-tour/manifest.json`, and the book build rejects missing, altered or stale captures. Alternative text in both PDF editions describes each control state.

Chapter 10

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow, starting above HTTP and protobuf, to exercise scenario decomposition, parameter flow, alternatives, requirements and rendering. In session-type terms it is a global type with five roles, a grant-or-deny choice by the authorization service, and parameters such as `state` and the code that flow from one message to a later one. Its channels are HTTP calls and browser redirects, whose physics (a redirect never followed, a callback replayed by an attacker) is where the security requirements live.

The listings are *proposed notation*, not compiled; the generated figures are real. The flow is simplified: the authorization request omits `client_id` and `response_type`, and the token request carries only the code, without `redirect_uri`, client authentication or PKCE, all required by RFC 6749 and current guidance. The knowledge claims hold for this simplified flow only.

10.1 Actors, values, and observable events

The person is modelled separately from the browser, because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State       : Type
opaque AuthCode    : Type
opaque AccessToken : Type
opaque Resource    : Type
opaque Profile     : Type
opaque Password    : Type
opaque Bytes       : Type
```

Every terminal elaborates to the common event envelope; “authorization redirected”, for example, becomes:

```
{ timeAt := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
```

```
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

OAuth does not prescribe how the authorization service authenticates the person; a password branch exercises knowledge flow without pretending the password is an OAuth parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
      (protectFor .authorizationServer
        (encode [field "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
      [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
      ]
```

`protectFor` promises confidentiality to the named endpoint, realized later by TLS, message-level encryption or a trusted local channel. Without it, a network observer would derive the password from the bytes.

10.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

The production establishes the correlation value `state` and exports it; nothing says how the application stores it.

10.3 Production 2: authorize, then grant or deny

```
def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
      [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
      [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
      [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
      [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state       <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
      [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
      [ include authorizationGranted
      , include authorizationDenied
      ]
```

Collapsed, the production is one node; expanded, it shows authentication and the choice. A probability distribution may be attached later without changing the legal traces.

10.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code     <- imported AuthCode "code"
    use resource <- imported Resource "resource"
    let token    <- declare AccessToken "accessToken"
    let profile  <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token is visible only where the protocol transmits it; the grammar says nothing about the server's database or the application's sessions.

10.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The `after` line is a guarded nonterminal reference: the denial branch ends without exchanging a code.

10.6 The parallel protobuf alphabet

The scenario says which messages may occur, in what order, with which parameter relationships; the protobuf file assigns field numbers and wire types to the same messages:

```
syntax = "proto3";

package leanfm.oauth;

enum Actor {
  ACTOR_UNSPECIFIED = 0;
  BROWSER = 1;
  APPLICATION = 2;
  AUTHORIZATION_SERVER = 3;
  RESOURCE_SERVER = 4;
}

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 clock_tick = 5;
  Actor src = 6;
  Actor dst = 7;

  oneof payload {
    ApplicationRequest application_request = 20;
    AuthorizationRedirect authorization_redirect = 21;
    AuthorizationRequest authorization_request = 22;
    CredentialsSubmitted credentials_submitted = 23;
    AuthenticationResult authentication_result = 24;
    AuthorizationCodeReturned authorization_code_returned = 25;
    AuthorizationCallback authorization_callback = 26;
    AuthorizationErrorReturned authorization_error_returned = 27;
    AuthorizationErrorCallback authorization_error_callback = 28;
    TokenRequest token_request = 29;
    AccessTokenIssued access_token_issued = 30;
    ProtectedResourceRequest protected_resource_request = 31;
    ProtectedResourceReturned protected_resource_returned = 32;
    ApplicationResponse application_response = 33;
  }
}

message ApplicationRequest {
  string resource = 1;
}

message AuthorizationRedirect {
  string state = 1;
  string redirect_uri = 2;
}

message AuthorizationRequest {
  string state = 1;
  string redirect_uri = 2;
}

message CredentialsSubmitted {
  bytes protected_credentials = 1;
}

message AuthenticationResult {
  bool accepted = 1;
}
```

```
message AuthorizationCodeReturned {
  string state = 1;
  bytes code = 2;
}

message AuthorizationCallback {
  string state = 1;
  bytes code = 2;
}

message AuthorizationErrorReturned {
  string state = 1;
  string reason = 2;
}

message AuthorizationErrorCallback {
  string state = 1;
  string reason = 2;
}

message TokenRequest {
  bytes code = 1;
}

message AccessTokenIssued {
  bytes access_token = 1;
}

message ProtectedResourceRequest {
  bytes access_token = 1;
  string resource = 2;
}

message ProtectedResourceReturned {
  string resource = 1;
  bytes profile = 2;
}

message ApplicationResponse {
  bytes profile = 1;
}
```

Each scenario message maps to one member of `EventEnvelope.payload`; the envelope carries identity, `prior_ids`, the `(session_id, task_id)` key, a clock tick and the endpoints. Order, queues, probability and dwell stay in the scenario; byte tags stay in the schema. Validation checks that every terminal has one variant with matching endpoints and field types. Protobuf does not promise one canonical byte sequence, so byte-for-byte comparison also fixes deterministic serialization and length-delimited framing.

10.7 Generate one legal trace and its artifacts

Generation chooses productions and field values; everything after is a deterministic projection of the resulting events. Choose the accepted and granted alternatives and instantiate

```
session = "s1"             task = "oauth"
resource = "profile"       state = "x7"
redirectUri = "https://a/cb"
protectedCredentials = aa bb
```

```
code = 01 02                accessToken = 03 04
profile = 05 06
```

The production derivation is

```
oauth
=> applicationRequestAndRedirect ;
   authorizeAndDecide ;
   after(authorizationGranted, exchangeAndFetch)
=> applicationRequestAndRedirect ;
   authorizationRequested ;
   credentialsSubmitted ; authenticationAccepted ;
   authorizationGranted ; exchangeAndFetch
```

The grant admits `exchangeAndFetch`; denial would disable it. Expanding the remaining nonterminals gives this terminal word:

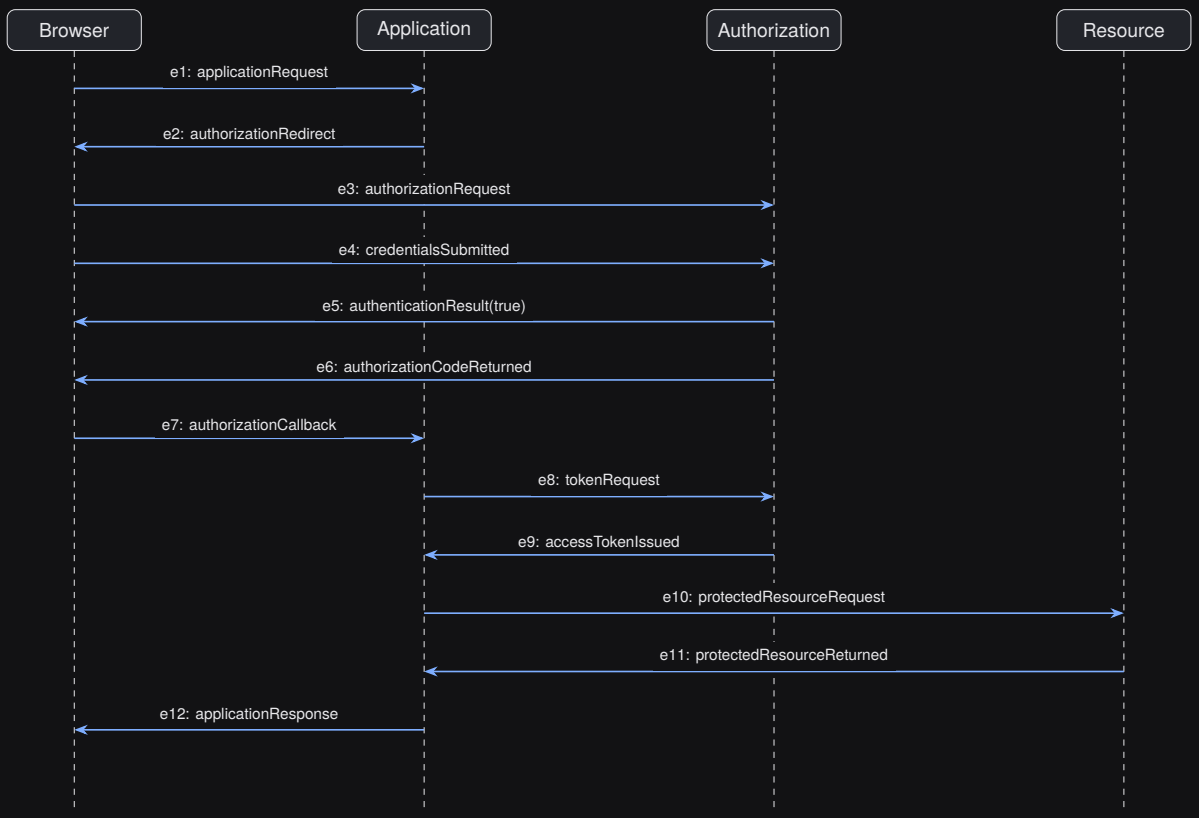


Figure 10.1: The selected terminal word as an interaction diagram. Event IDs label messages on arrows; no actor-to-actor arrow is written as implication-like text.

The choice has disappeared into ordinary terminals: `e5`'s bytes select the accepted authentication, and `e6`'s oneof tag, source and bytes distinguish grant from error. No separate "chooser" value is needed.

10.7.1 Envelope stream and concrete bytes

Clock ticks and identifiers increase; the chain is linear, so `priorIds(`e_n`)` is `[`$e_n - 1$`]` (a fork or join would change the lists, not the procedure). One envelope from the companion fixture:

```
e1   0  application_request          31  0a0265311a02733122056f6175746830013802a201090a0770726f66696c65
```

`examples/oauth-granted.bytes` holds all twelve (440 bytes; largest 46). There is no generator or round-trip check for this fixture, so it is an example, not evidence of reproduced serialization.

10.7.2 Event graph with stored backpointers

Parsing each envelope selects a payload alternative, checks its `src`/`dst`, and consumes one terminal. The event graph:



Figure 10.2: The accepted trace as an event graph. Arrows are stored backpointers: `e1` has none and each later event points to its immediate predecessor.

This is one path; the residual state machine, derived from every legal residual, also has the rejected and denied branches.

10.7.3 Actor projections

Projection keeps each actor's terminals in order: Resource receives `e10` and sends `e11`. The global word stays authoritative for cross-component order and parameter equality, which a projection cannot show.

10.7.4 Event order, queues, and memory

The interaction diagram and the state-machine path are the same order graph laid out by actor and by residual; neither is drawn independently. The word does not fix a transport schedule; the canonical one runs `produce; transport; receive` per event, so every queue peaks at one message and the global queued-byte peak is 46. Another interleaving may peak higher, so this schedule is a witness, not the memory-budget proof.

10.7.5 Measurements, plots, and verdicts

From the clocks and correlation key, the generated task attempt is

```
{ key = ("s1", "oauth"),
  started = { kind = applicationRequest, timeAt = 0 },
  completed = { kind = applicationResponse, timeAt = 23 },
  outcome = succeeded, latency = 23 ticks }
```

It contributes a latency point `(maxConcurrency=1, latency=23)`, a byte series ending at `(clock=23, bytes=440)` and a success to the outcome chart, but no success probability: estimating the grant's 0.87 needs many sessions, including incomplete ones. Trace monitors accept `state=x7`, the code `01 02`, and no exchange after denial; CTL obligations need every reachable residual. The pipeline:

```
grammar + field values
  -> legal terminal word
  -> typed EventEnvelope stream
  -> framed protobuf bytes
  -> partial-order graph + interaction diagram
  -> residual FSM path + per-actor projections
  -> queue schedule + byte-pressure samples
  -> task attempt + probability/latency/XY reducers
  -> replay fixture + trace-local property verdicts
```

Only branch and value generation, and the transport schedule, add choices; every other arrow is reproducible.

10.8 Interaction diagrams by scenario

10.9 Requirements as observable constraints

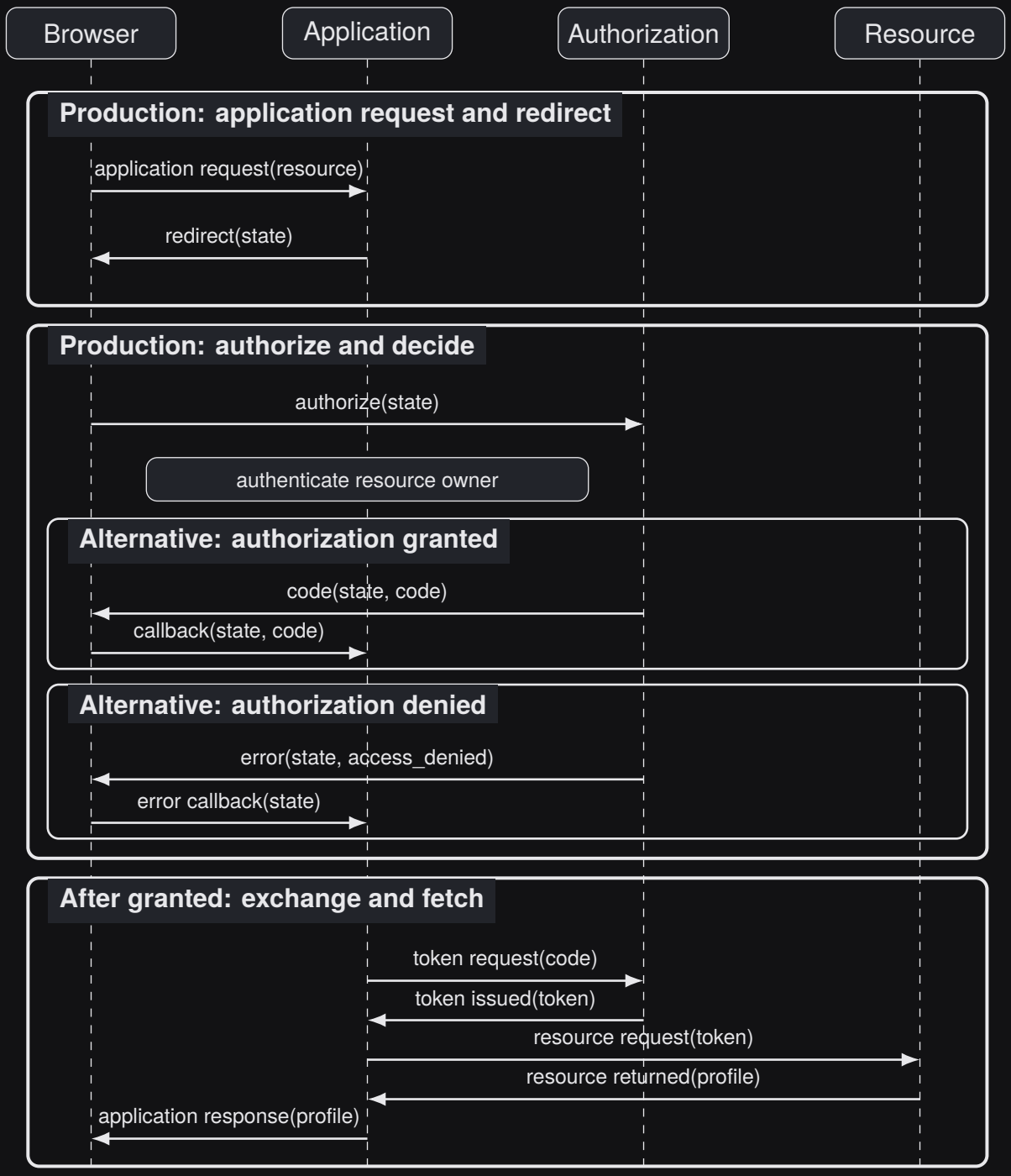


Figure 10.3: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

```
def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
      (on "authorization callback" (previouslySame "state"
        "authorization requested"))

  , forbid "code exchange after denial"
      (previously "authorization denied" && event "token requested")

  , require "issued token came from a code"
      (on "access token issued" (previouslySame "code" "token requested"))

  , ctl "every decision reaches the application"
      (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

  , ctl "a protected result is never returned without a token request"
      (AG (resourceReturned --> previously tokenRequested))
  ]
```

10.9.1 Who knows the password?

This OAuth instance applies the trace-derived knowledge closure from Section 17.5:

```
def passwordKnowers (trace : Trace WireEvent) : List Actor :=
  allActors.filter fun actor =>
    Derives (knowledgeFrom actor trace) (value password)

theorem oauth_password_knowers
    (trace : Trace WireEvent)
    (accepted : (oauthSystem resource password).Accepts trace)
    (initial : InitiallyKnown password [.browser, .authorizationServer])
    (honest : Uncompromised [.browser, .authorizationServer])
    (protection : PerfectProtection protectFor) :
    passwordKnowers trace = [.browser, .authorizationServer] := by
  exact knowledgeClosureOfProtectedField accepted initial honest protection
```

Proposed notation: neither the theorem nor the library lemma it uses exists yet.

The browser knows a typed password; a narrower boundary needs a trusted credential actor, and compromise, logging or failed protection change the result.

10.9.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
    , namedBytes "state" (encode state)
    , namedBytes "redirectUri" (encode callback)
    ]
```

```
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

An HTTP mapping might put these bytes in form fields, a protobuf mapping in numbered fields; either must prove that decoding and re-encoding preserve the terminal and its bindings. `previously` is a trace predicate for now.

10.10 Probabilities and charts

The randomness belongs to the user, who grants or denies consent; everything else reacts deterministically. As written, though, the server emits both grant and error, so by the deciding-actor rule of Chapter 5 the server chooses. A faithful model adds a consent event from the browser, `ConsentGiven` or `ConsentRefused`, making the user's choice observable. Weights live in an overlay (Chapter 13); the proposed `weightedOneOf` writes them inline:

```
def consentDecision : Scenario :=
  weightedOneOf "consent decision"
    [ weighted (87 / 100) authorizationGranted
    , weighted (13 / 100) authorizationDenied ]

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
      source events; groupBy terminalOutcome; value count
  , line "end-to-end authorization latency" do
      source (correlate events by "state")
      x clock; y (elapsed "authorization requested" "application response")
  ]
```

The branches then begin with different consent bytes, and every later transition may have probability one: the grant branch carries mass 0.87 through queues, transport and the deterministic response. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \qquad P(\text{denial callback}) = 0.13.$$

Each factor of one is a deterministic step; dwell affects latency, not mass. Scheduling (which enabled actor runs next) remains nondeterministic, so the composed system is an MDP, analysed under a fixed or quantified policy (Chapter 7). With several sessions in flight, its state holds every actor's queues and per-`(session, task)` residuals, since one user's choice changes queue pressure for others. The percentages are inputs or measurements; the grammar says only what is possible.

10.11 Residual state machine

The automaton is behavioural: cookies, database rows, threads and cryptographic state are refinements of this contract, not part of it.



Figure 10.4: Illustrative OAuth state machine, drawn by hand. The full machine also contains the rejected-authentication branch.

Chapter 11

CTL over Generated Futures

A well-formed global type already guarantees safety, fidelity and progress under its channels' assumptions. Everything else a stakeholder wants to know is a question about futures: can the transfer always still finish, must it, can this message ever precede that one? The global type and its channel physics generate a branching transition system, and CTL speaks directly about that branching: path quantifiers distinguish *some future* from *every future*, temporal operators the next state, eventuality, invariance and until. The author learned the modal logic used here mostly from Kröger and Merz [13].

11.1 Basic modal logic

Propositional logic evaluates a formula at one world or state. Basic modal logic adds an accessibility relation R between worlds and two operators:

$$w \models \Box p \quad \text{iff} \quad p \text{ holds at every } v \text{ such that } wRv,$$
$$w \models \Diamond p \quad \text{iff} \quad p \text{ holds at some } v \text{ such that } wRv.$$

$\Box p$ and $\Diamond p$ read "necessarily p" and "possibly p", and are dual: $\Diamond p \equiv \neg\Box\neg p$. The symbols do not fix what accessibility means: a next computation state, a world compatible with an actor's knowledge, a permitted world. The basic logic K allows any frame; reflexivity, transitivity, symmetry and the Euclidean condition give stronger systems, whose principles must not be imported because "necessarily" sounds strong. LeanFM's relation is a directed computation successor, not reflexive, symmetric or epistemic.

11.2 LTL: time along one path

Linear Temporal Logic (LTL) describes one path with X (next), F (eventually), G (always) and U (until): Fp says p holds at some suffix of the path, Gp at every suffix. Its operators answer *where along this path*, not whether one branch or every branch is meant; the common convention that every initial path must satisfy the formula is outside the syntax. "Some legal execution eventually recovers" and "every one does" are different claims that share the words "eventually recovery".

LeanFM does not implement LTL. It uses CTL because requirements combine two independent axes:

- *always* or *eventually* says where along forward time a predicate must hold; and
- *necessarily* or *possibly* says whether it must hold on every permitted forward continuation or may hold on at least one.

The construction is *temporally alethic*: possible means possible now or at some point along a legal future. English uses “always” for both universal modality and temporal persistence; CTL keeps the two coordinates apart.

11.3 CTL: path modality combined with time

CTL pairs a path quantifier with each temporal operator: E, some permitted continuation (temporal possibility), or A, every one (temporal necessity). They quantify over legal computations from the current observation, not timeless worlds. F versus G is eventuality versus invariance along a continuation; E versus A is some versus all continuations. The four core combinations and their dualities:

$EF\,p$: possibly eventually p, $\quad AF\,p \equiv \neg EG\,\neg p$: on every future, eventually p,
$EG\,p$: p always on some future, $\quad AG\,p \equiv \neg EF\,\neg p$: p holds on every future at every position.

A ranges over futures and G over positions along one. The dualities drive counterexample search: $AG\,p$ is tested by searching for $EF\,\neg p$, $AF\,p$ by searching for $EG\,\neg p$. A witness refutes the universal property; its absence establishes it only within the finite checked model.

11.3.1 Least upper bounds and greatest lower bounds

Eventuality and strong until choose the least solution of their recursive equations. Over the lattice of state sets, begin with the empty set and repeatedly add states justified by the previous approximation:

$$\mathrm{Sat}(EF\,p) = \mu Z.(P \cup \mathrm{Pre}_\exists(Z)) = \bigcup_{n\geq 0} Z_n, \qquad Z_0 = \varnothing.$$

The same construction gives existential until:

$$\mathrm{Sat}(p\ EU\ q) = \mu Z.(Q \cup (P \cap \mathrm{Pre}_\exists(Z))).$$

AF and strong AU are least fixed points too, with the universal predecessor, so a cycle that never reaches the target adds nothing. Invariance and weak until make the dual choice: start from all candidate states and remove unsupported ones, giving the greatest fixed point:

$$\mathrm{Sat}(EG\,p) = \nu Z.(P \cap \mathrm{Pre}_\exists(Z)) = \bigcap_{n\geq 0} Y_n.$$

LeanFM has two evaluators, and tests check that they agree. The recursive one in `LeanFM.CTL`, used by task metrics, searches with cycle awareness: an unresolved revisit is false for `EF`, `EU` and

strong `AU` (least fixed point) and a predicate-preserving revisit true for `EG` and weak until (greatest), with `AG` as $\neg EF\,\neg p$ and `AF` as $\neg EG\,\neg p$. The one in `LeanFM.Logic`, behind the MCP `check_ctl` tool, labels every reachable node with the set satisfying each subformula by fixed-point iteration, and returns a witness or counterexample path.

11.3.2 Asymptotic cost

Let V be the reachable observations, E the edges, L the formula's nodes and P the simple paths a search inspects. The recursive evaluator keeps a path-local list of visited states and memoizes nothing, so one temporal search costs $O(E_P + VP)$ time (E_P edge visits over those paths) and $O(V + L)$ memory: it may revisit a state once per path reaching it, and P can exceed 2^V in a dense graph. The fixed-point evaluator avoids this: each fixed point takes at most $V + 1$ iterations of $O(V + E)$, so a formula costs $O(L\,V\,(V + E))$, within its bound of 4096 nodes. Worklist propagation would reach the textbook $O(L(V + E))$; that is a target.

None of this compares LeanFM with TLC. LeanFM often builds a smaller graph by omitting private variables, but a well-abstracted TLA+ model may omit them too, and a speed claim would need the same protocol, properties, bounds, hardware and settings. The book claims only a different modeling boundary.

An observation is immutable once recorded: values change only on a forward transition to a successor, though they need not be monotone.

Formula	Reading	Typical use
$EF\,p$	possibly, eventually p	temporal possibility / witness
$AF\,p$	necessarily, eventually p	termination / response
$EG\,p$	possibly, always p	viable invariant path
$AG\,p$	necessarily, always p	safety invariant
$EX\,p$	possibly, next p	enabled successor / one-step witness
$AX\,p$	necessarily, next p	required one-step consequence
$(p\ AU\ q)$	every path keeps p until q	constrained progress
$(p\ EU\ q)$	some path keeps p until q	possible progress
$(p\ AW\ q)$	every path: p until q, or p forever	necessary weak until
$(p\ EW\ q)$	some path: p until q, or p forever	possible weak until

$AF\,p$ requires p on every legal future from the current prefix; $EF\,p$ needs one future as a witness. Branching is part of the requirement. Every temporal operator in CTL is governed by A or E; CTL*, which nests state and path formulas freely, is not implemented, so not every English alternation of "on all paths" and "eventually" is a formula here.

11.4 Projecting to LTL-style F and G

Requirements often want the simpler forms Fp and Gp, which LeanFM shows only with an explicit path policy. On one selected execution, AFp and EFp both project to Fp and AGp and EGp to Gp; under the convention that every initial path must satisfy the formula, Fp is AFp and Gp is AGp. Displaying AF and AG as F and G is an interface reduction, not an equivalence: the artifact keeps the exact CTL, so the checker can still use witnesses, negation and the right counterexample search.

11.5 Persistence and recurrence

Nested CTL operators express two common requirements:

$$AF\,AG\,p, \qquad AG\,AF\,p.$$

The first is eventual permanent safety: every path reaches a state from which every path keeps p (finish key rotation and never accept the old key again). The second is universal recurrence, "always eventually p" (service keeps returning to ready). For witness searches their duals are:

$$\neg(AF\,AG\,p) \equiv EG\,EF\,\neg p, \qquad \neg(AG\,AF\,p) \equiv EF\,EG\,\neg p.$$

The existential forms are $EF\,EG\,p$ and $EG\,EF\,p$; in the second, the eventual witnesses may leave the outer path, so it does not require infinitely many p on that path. These cover useful persistence and recurrence, not all LTL: GFp corresponds to $AG\,AF\,p$ on a total transition model, but $AF\,AG\,p$ is stronger than pathwise FGp, since it needs a state where *all* continuations stay safe. Terminal states follow the checker's continuation convention.

11.6 Knowledge, obligation, and permission

A multimodal model can have several accessibility relations: temporal reachability, $K_a/\widehat{K_a}$ for what actor a knows or considers possible, another pair for permission and obligation. They are not interchangeable. LeanFM's A and E range only over legal forward computations of the current model. They do not mean:

- probability one versus positive probability;
- known versus merely believed by an actor;
- obligatory versus permitted by a policy; or
- physically necessary versus logically possible.

Knowledge and policy are modelling techniques a project defines with the relations below, not built-in operators (Chapter 17 decides the state-local `knows` predicate). The Kerberos video [16] illustrates them; it does not verify the protocol.

For knowledge, let W be the declared set of legal histories and $\mathrm{obs}_a(h)$ actor a's local observation of history h:

$$K_a\varphi(h) \iff \forall h' \in W.\ (\mathrm{obs}_a(h) = \mathrm{obs}_a(h') \Rightarrow \varphi(h')).$$

Observations declare visible messages, private keys, clock information and retained memory. With the current history in W, knowledge is factive; belief needs a different relation and must not inherit that guarantee.

$$\widehat{K_a}\varphi \equiv \neg K_a \neg\varphi.$$

This is epistemic possibility, not temporal reachability.

For policy, let $\mathrm{Ideal}_a(h)$ be the declared policy-compliant futures from h, and let ψ be a condition on a whole future path:

$$O_a\psi(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi), \qquad P_a\psi(h) \iff \exists \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi).$$

The set of ideal futures must be nonempty, so contradictory policies are reported rather than certified vacuously. Legal executions may violate policy, and detecting that is part of the analysis; these are obligations under a named policy, not moral judgments.

$$P_a\psi \equiv \neg O_a \neg \psi.$$

when both use the same policy relation. For example:

$$O_a F\ \mathrm{audit}(r)(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \exists i \geq 0.\ \mathrm{audit}(r, \pi_i).$$

Here F is a path operator, kept distinct from state formulas. Temporal, epistemic and policy quantification then combine without confusing their domains: a legal future may be possible, unknown to Alice, and forbidden by policy at once. Probability, belief, strategic ability and program-indexed modalities remain separate work, and symbolic uncompromised-key assumptions do not establish computational cryptographic security.

11.7 Possibility means preparedness

A requirement that declares $EF\,p$ does more than admit that a checker can find a path to p: it says p is a supported future the system must prepare for. A `possibly` obligation therefore links three artifacts:

1. a legal grammar branch and witness trace reaching the condition;
2. a handling plan in `Requirements.lean`; and
3. generated implementation code whose justification cites that possibility requirement.

This is the target; today validation checks only that a handling plan is cited, not that generated code exists or covers the witness.

For example, “a bakery delivery may fail” requires a decodable `DeliveryFailed` message, a legal failure branch, and code that performs the specified refund or notification behavior. Removing every failure branch does not implement reliability; it makes the real failure trace out-of-spec.

Possibility is not probability: a possible condition may be rare or of unknown likelihood, and the handling obligation comes from its being supported, not from a percentage crossing a threshold. $EG\,p$ carries a different obligation: some future keeps p, so the system monitors the conditions that keep that future viable, exposes their loss, and controls choices that would destroy it. That every execution keeps p is the stronger $AG\,p$.

11.8 Strong implication and the Chewbacca argument

The “Chewbacca defense”, from the *South Park* episode “Chef Aid”, is a courtroom parody that uses an irrelevant claim to distract from the conclusion: a memorable non sequitur. Material implication $p \to q$ is true whenever p is false, which permits a vacuous global claim when no reachable state satisfies p. LeanFM names the non-vacuous combination

$$p \Rightarrow_s q \quad \stackrel{\mathrm{def}}{=} \quad EF\,p\ \wedge\ AG(p \to q).$$

It requires a legal future witnessing p and that every reachable p also satisfies q, so together they imply $EF\,q$. The absurd test:

$$p = \text{“Chewbacca lives on Endor”},$$
$$q = \text{“you must acquit”}.$$

Mentioning p does not obtain q: the model must reach a p witness and establish $AG(p \to q)$ on every continuation, and if the grammar relates residence to nothing, that obligation fails. Logic cannot judge the relevance of English propositions, but neither a missing antecedent nor one rhetorical occurrence of it can make the implication pass vacuously.

In Lean the definition is direct:

```
def stronglyImplies (p q : CTL S) : CTL S :=
  CTL.and (CTL.ef p) (CTL.ag (CTL.implies p q))
```

The path quantifier is part of the until operator: `AU` and `EU` are binary operators, not an outer `A` or `E` around a bracketed until, so a subexpression never changes meaning because of what surrounds it:

```
def boundedUntilTerminal : CTL Observation :=
  atom withinCapacity AU atom isTerminal

def someAuthorizedContinuation : CTL Observation :=
  atom hasValidAuth EU atom isSucceeded
```

11.9 Weak until: AW and EW

Strong until requires the right operand to occur; weak until allows it never to, provided the left operand keeps holding:

$$p\ AW\ q \text{ : on every future, } p \text{ holds until } q, \text{ or } p \text{ remains forever},$$
$$p\ EW\ q \text{ : on some future, } p \text{ holds until } q, \text{ or } p \text{ remains forever}.$$

The state satisfying q need not satisfy p. Weak until fits “remain available until shutdown, which is permitted but not required”. $p\ EW\ q$ equals $(p\ EU\ q) \vee EG\,p$, but $(p\ AU\ q) \vee AG\,p$ is *not* AW on a branching system: some futures may reach q while others keep p forever, so neither disjunct holds although every future satisfies weak until. LeanFM implements `AW` directly.

```
def serviceMayRunUntilShutdown : CTL Observation :=
  atom available AW atom shutdown

def someWorkerCanRemainReady : CTL Observation :=
  atom ready EW atom retired
```

In `r && (p AU q)` the right operand is the whole `AU` expression.

11.10 Past claims

CTL looks forward, but security claims often cite the past: “this capability was previously issued”, “this response matches an earlier request”. Since states are trace prefixes, past facts are predicates over the accumulated trace. Bounded past conditions can be compiled into forward monitor state, evaluated before the monitor takes the current event; on concurrent histories, “before” must be declared as causal ancestry or a chosen linearization, never screen order. Whether explicit past operators read and check better than trace predicates or monitors is open, judged by how specifications survive changes to the event schema.

11.11 Fairness and time bounds

AF alone does not distinguish a participant perpetually denied a turn from one that violates the protocol, so fairness assumptions are named, scoped and visible in theorem statements. “Eventually” is not “within 200 milliseconds”: time bounds belong in an explicit timed layer, not in timestamps and prose.

Chapter 12

A Legal Example: Lawful, Right, Known, Required

Section 17.5 needed two questions beyond temporal logic: *who knows which values*, and *who may hold them*. Law presses harder on the same machinery: obligations have deadlines, are triggered by what someone knew, create new ones when broken, and two codes can disagree. This chapter works one small example end to end; its tables and diagram are generated by `LegalDepositExample.lean` from the event histories.

The statute below is invented, and so is the moral code. Neither describes any real jurisdiction, and nothing here is legal advice.

12.1 Three meanings of "legal"

LeanFM already calls histories the global type admits *legal*. Two further meanings are needed, and conflating them is the commonest error in normative reasoning about software.

Term	Judged against	Operators
admissible	the protocol grammar: what can happen at all	CTL A, E, F, G over generated futures
lawful / unlawful	a declared statute's ideal futures	O_a^{Act}, P_a^{Act}
right / wrong	a declared moral code with a named author	O_a^{Code}, P_a^{Code}

Here "admissible" means what the grammar allows. An unlawful act is usually admissible, since people can break the law, so $EF\ unlawful$ is normally true: statutes constrain which futures are *ideal*, not which are *possible*. A moral code uses the same machinery with another relation. LeanFM never decides what is right; it checks histories against a code someone wrote and signed, and shows where codes disagree.

12.2 The Model Tenancy Deposit Act and a declared code

The actors are a tenant T, a landlord L, a deposit registry, the post and a court. Writing $Ideal_a^N(h)$ for the futures that code N regards as ideal for actor a after history h, the obligation and permission

operators of the previous chapter are indexed by the code:

$$O_a^N\psi(h) \iff \forall\pi \in Ideal_a^N(h).\ \psi(\pi), \qquad P_a^N\psi(h) \iff \exists\pi \in Ideal_a^N(h).\ \psi(\pi).$$

Prohibition is obligation of the negation: $Forbidden_a^N\psi \equiv O_a^N\neg\psi$. $F_{\leq d}$ is a bounded path operator: within d days.

Act §1 (protection). Within 30 days of receiving a deposit, the landlord registers it.

$$AG\,(paid \to O_L^{Act}\,F_{\leq 30}\,protected)$$

Act §2 (return, triggered by knowledge). Once the lease has ended *and the landlord knows the tenant's forwarding address*, the landlord returns the deposit or itemises deductions within 14 days.

$$AG\,((ended \wedge knows(L, address)) \to O_L^{Act}\,F_{\leq 14}\,(returned \vee itemised))$$

Act §3 (reparation). If §2 is violated, the landlord pays the deposit plus an equal penalty.

$$AG\,(violated_{\S 2} \to O_L^{Act}\,F\,penaltyPaid)$$

Act §4 (prohibition). A deposit is never applied to rent.

$$O^{Act}\,G\,\neg offset \;\equiv\; Forbidden^{Act}\,F\,offset$$

Code (declared by the stakeholder). (a) Never charge for normal wear and tear. (b) A landlord who holds another way to reach the tenant makes a reasonable effort to use it. (c) Help a tenant in documented hardship when it costs nothing you are owed.

12.3 Knowledge from the causal past

Act §2 depends on what the landlord knew, which LeanFM computes: an actor knows a value at an event when a message carrying it reached the actor somewhere in the event's causal past:

$$knows(a, v)@e \iff \exists e' \in past(e) \cup \{e\}.\ dst(e') = a \wedge v \in derive(msg(e'))$$

This is *possession* knowledge, the kind security protocols need (the MCP `knows` tool decides it); K_a, over histories a cannot tell apart, is for claims like "the landlord could not have known".

In the *lost in the post* history the tenant posts the address and the post never delivers it. She records that she *believes* §2 was violated, and is wrong: the address never enters the landlord's causal past, so §2 never triggers. Belief is an event and is not factive; computed knowledge is. The history is still *wrong* under Code (b): the landlord had the tenant's email from the lease (e1, in every causal past) and never used it.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Post	`AddressPosted`	letter to landlord	e4	
e6	400	Tenant → Tenant	`BelievesViolated`	no refund after 34 days	e5	

Figure 12.1: The *lost in the post* history. No event delivers `ForwardingAddress` to the landlord, so $knows(L, address)$ never holds and Act §2 is never triggered.

12.4 Violation and reparation

A naive deontic logic breaks on the late return: after day 380 the landlord seems obliged both to have returned the deposit and to pay for not having done so. This is the contrary-to-duty problem, Chisholm's paradox. Ideal futures computed *from the current history* dissolve it: before day 380 every ideal future has a timely return; after it, none is admissible, so the ideal futures are the best remaining ones, in which the penalty is paid. The obligations change as the history grows without becoming inconsistent, and the checker reports *fulfilled*, *violated*, or *violated, then repaired*.

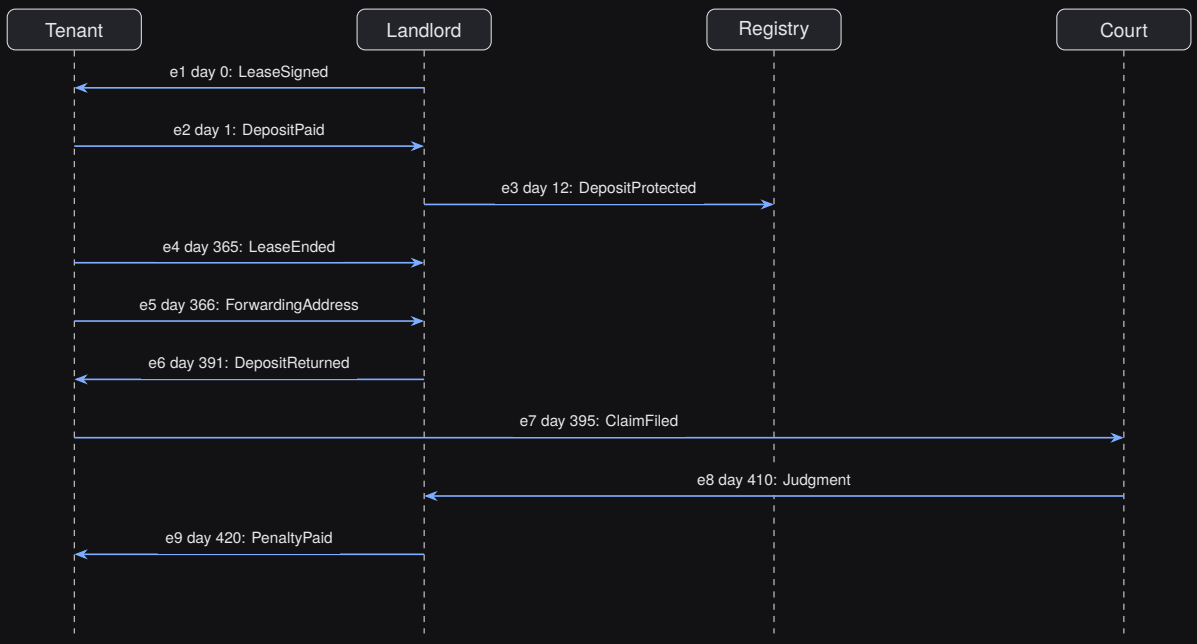


Figure 12.2: The *late return* history, rendered by `MDPDiagram.renderInteractionTeX` from its events. The address reaches the landlord on day 366, so the §2 deadline is day 380; the return on day 391 violates it, and the penalty on day 420 repairs it.

Id	Day	From → To	Message	Detail	prevIds	causeIds
e1	0	Landlord → Tenant	`LeaseSigned`	rent 1000, deposit 1000, tenant email on file		
e2	1	Tenant → Landlord	`DepositPaid`	1000	e1	e1
e3	12	Landlord → Registry	`DepositProtected`	scheme ref 7731	e2	e2
e4	365	Tenant → Landlord	`LeaseEnded`	keys returned	e2	
e5	366	Tenant → Landlord	`ForwardingAddress`	12 Elm St	e4	
e6	391	Landlord → Tenant	`DepositReturned`	1000	e5, e3	e5
e7	395	Tenant → Court	`ClaimFiled`	Act s2 deadline day 380 missed	e6	e6
e8	410	Court → Landlord	`Judgment`	pay penalty 1000 under s3	e7	e7
e9	420	Landlord → Tenant	`PenaltyPaid`	1000	e8	e8

Figure 12.3: The same history as events. `causeIds` pair each response with its cause (the return answers the address; the judgment answers the claim); `prevIds` order each actor's own events.

12.5 Lawful, right, and the four quadrants

Table 12.1 evaluates every rule in every history, and table 12.2 summarises each history under each code. Keeping the codes separate shows every combination:

- *lawful but wrong*: an on-time return that deducts for normal wear (Code a), and the lost address the landlord could have chased (Code b);
- *violated, then repaired*: the late return, under the Act, with no moral duty arising;
- *unlawful but right*: the landlord who, on a laid-off tenant's request, applied the deposit to the final month's rent. That breaks Act §4 and fulfils Code (c).

History	Act §1	Act §2	Act §3 (CTD)	Act §4	Code (a)	Code (b)	Code (c)
Itemised deduction on time	fulfilled	fulfilled	not triggered	fulfilled	violated	not triggered	not triggered
Returned late, penalty paid	fulfilled	violated, then repaired	fulfilled	fulfilled	not triggered	not triggered	not triggered
Address lost in the post	fulfilled	not triggered	not triggered	fulfilled	not triggered	violated	not triggered
Hardship: deposit used as last rent	fulfilled	fulfilled	not triggered	violated	not triggered	not triggered	fulfilled

Table 12.1: Every rule in every history, generated by `LegalDepositExample.lean`. §3 is a contrary-to-duty rule: it is triggered only on histories where §2 was violated.

The hardship history is a genuine dilemma: the Act requires $G\,\neg \mathit{offset}$ and the Code requires $F\,\mathit{offset}$, so no future is ideal under both. LeanFM reports the conflict:

$$\mathit{Conflict}(N_1, N_2, a, \psi, h) \iff O_a^{N_1}\psi(h) \wedge O_a^{N_2}\neg\psi(h).$$

It does not resolve it. Choosing law over conscience, or the reverse, is a stakeholder decision, recorded as an explicit ordering in the typed requirements.

12.6 What CTL says, and what it does not

A and E range over admissible futures, so they cannot express "ought", but they carry most of the weight once normative facts are computed as state predicates, as knows, allows and $\mathit{requires}$ are

History	Under the Act	Under the Code	Landlord knows address from	Normative conflict
Itemised deduction on time	lawful	wrong	day 366	none
Returned late, penalty paid	violated, then repaired	no duty arose	day 366	none
Address lost in the post	lawful	wrong	never	none
Hardship: deposit used as last rent	unlawful	right	day 366	$O_L^{Act}\, G\, \neg offset$ (Act §4) and $O_L^{Code}\, F\, offset$ (Code c): no future is ideal under both

Table 12.2: Each history under the Act and under the declared Code, when the landlord first knew the forwarding address, and any normative conflict.

(Section 17.5):

- A deposit registry's own conduct is ordinary safety: $AG\,\neg(receiptIssued \wedge \neg protected)$.
- Remedies must stay available. Following *possibility means preparedness*, a violation requires a supported path to a claim: $AG\,(violated_{\S 2} \rightarrow EF\ claimFiled)$, with a handling plan in the requirements.
- The *late return* history is simultaneously a *counterexample* for the universal claim that §2 is always met and a *witness* for $EF\ repaired$. Checker output should name both roles.
- Knowledge never exceeds permission, the security invariant, has the same shape here: a registry must not disclose the tenant's address beyond those permitted, $AG\,(knows(a, address) \rightarrow permitted^{Act}(a, address))$.

CTL does not provide per-code quantification over ideal futures, deadlines from a knowledge-triggered start, or the contrary-to-duty recomputation; the example computes those over each history.

12.7 Computing what is owed: Catala

The Act mixes two questions: *what* the law computes from the facts (amounts, deductions, penalties), and *when* duties arose, deadlines passed, and violations were repaired. This chapter answers only the second. The first is the territory of Catala [19], a language for literate law whose default logic writes "the rule is X, except when Y, except when Z" directly, which a priority between codes here handles only crudely. Its formally specified date library is what deadlines such as "within one month" of 31 January need before $AF_{\leq 14\,\text{days}}$ can be trusted on a real statute. The two compose: Catala computes what is owed, and the grammar and CTL check who owed what by when. A verdict stays conditional on its premises: the governing code is chosen, open-textured terms such as "reasonable effort" are atoms a person supplies, and the facts are only as good as the history.

12.8 Status

`LegalDepositExample.lean` is a book example that hard-codes four rules and three code clauses, not a general normative checker. A language feature would need a typed normative code: its author; each rule's actor, trigger (a state formula that may use $knows$), deadline, path condition and reparation; and an optional priority, with a status per rule per history and its trace. Regenerate with `lake env lean -run LegalDepositExample.lean`; provenance is in `book/generated/legal-provenance.txt`.

Chapter 13

Generated Examples and Probability Overlays

Most specifications explain themselves with hand-written examples: a trace someone typed, a sequence diagram someone drew. Those examples drift. This chapter takes the opposite approach. Every example below is the answer to a question asked of the specification, and every number is computed from it. Nothing in the figures was drawn by hand; regenerate them with `lake env lean -run ProbabilityOverlayExample.lean`.

A global type generates runs, and each run is an example of the conversation. The reading of the temporal operators is generative. If we generate event lists from the grammar, then

$EF\,p$ means some generated list contains p; the witness is that list, and it is the example.

$AG\,p$ means every generated list satisfies p; a counterexample is a generated list that does not.

$AF\,p$ means every generated list eventually reaches p; a counterexample is a list that loops forever without it.

13.1 The core says what can happen

A client sends a `Request`. The server answers with a `Response`, or a `Timer` fires a `Timeout` and the client retries. After the third timeout the client gives up. Retries are bounded, so the task unrolls them into states: one waiting state per attempt.

The core carries no numbers. It says only which messages can follow which, and it already answers the qualitative questions: every run ends, and giving up is possible.

13.2 The overlay says how likely and how long

A separate table, the *overlay*, annotates each transition with a branch probability and a delivery delay. This one is *declared*: it states what we expect (a 10% timeout rate, 40 ms responses, 200 ms timeouts). A *measured* overlay would have exactly the same shape, reduced from recorded traces instead of asserted, and the same questions could be asked of either. Keeping the numbers out of the grammar means a project without probabilities loses nothing, and declared and measured values can be compared on one unchanged specification.

The overlay is validated before use: every transition must be annotated, nothing else may be, and the probabilities leaving each state must sum to exactly 1. They are exact fractions and are never normalized silently.

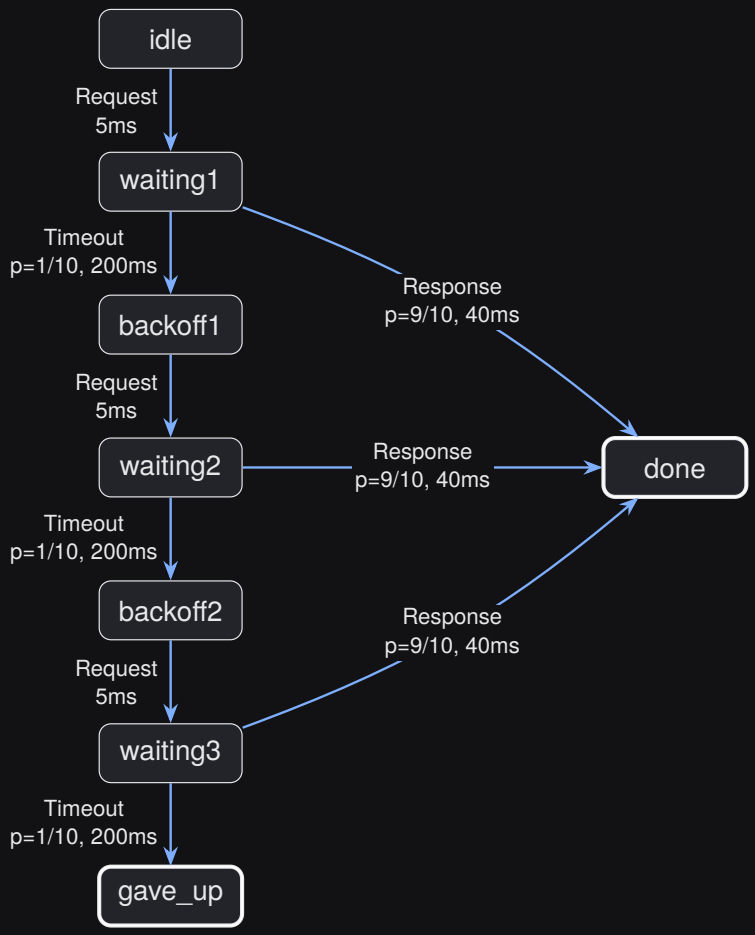


Figure 13.1: The bounded-retry task (the core) with the declared overlay drawn on its edges. Double borders mark terminal states.

Keeping probabilities outside the model is a deliberate break with tools such as PRISM [14], where branch probabilities are part of the model itself, and it follows FizzBee [1], which leaves the specification unchanged and supplies probabilities and costs in a separate performance file. The same separation lets one specification be checked under a declared overlay and a measured one.

13.3 Questions and their generated answers

Question	Formula	Answer
Does every run end? (core, no overlay)	`AF terminal`	proved
Can the client give up?	`EF gave_up`	proved; most likely witness has p=1/1000, 615 ms
How likely is success?	`P=? [F done]`	999/1000 (0.999)
Do 99% succeed within 300 ms?	`P>=99/100 [F<=300ms done]`	proved; exactly 99/100
Do 99.9% succeed within 300 ms?	`P>=999/1000 [F<=300ms done]`	disproved; most likely late run p=9/1000, 455 ms
Expected duration of a run	`E[duration]`	67.71 ms

Table 13.1: Every answer is computed from the task and the declared overlay. The first row uses the core alone.

The table reads like a service-level review. Success is overwhelmingly likely ($999/1000$). Ninety-nine percent of runs finish within 300 ms, and that bound is met *exactly*, with no margin. Tightening it to 99.9% fails, and the checker does not just say so: it produces the most likely run that misses the deadline.

13.3.1 The usual run

The most likely generated run is the one a newcomer should see first.



Figure 13.2: The most likely run: one request and one response, 45 ms, probability $9/10$.

13.3.2 A precedent: the client gives up

$EF\ gave_up$ holds, and its witness is a concrete case. It is rare, with probability $1/1000$, but it is possible, and this is what it looks like.

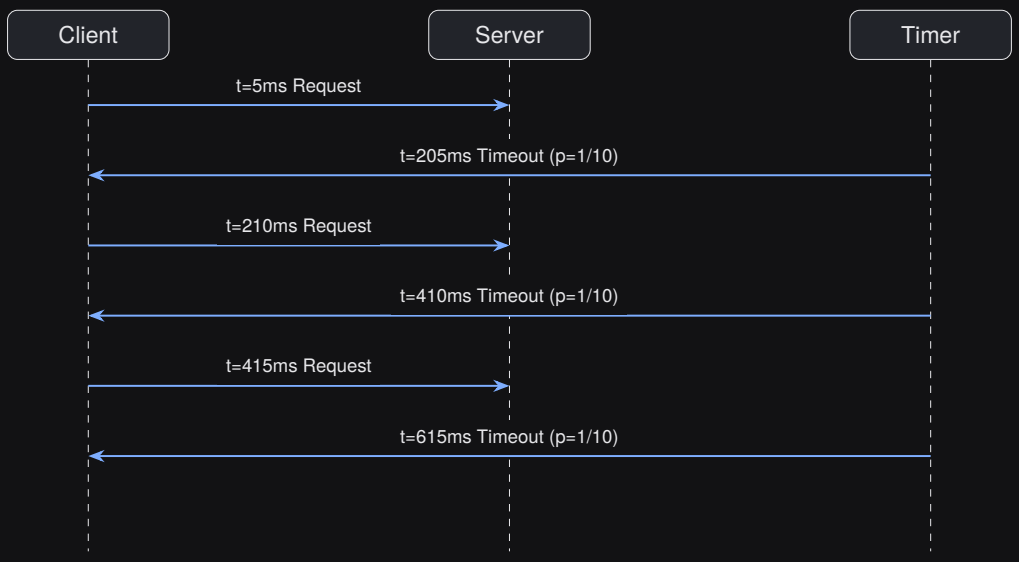


Figure 13.3: The witness for $EF\ gave_up$: three timeouts, 615 ms, probability $1/1000$.

13.3.3 A counterexample: the late success

The claim $P_{\geq 999/1000}[F_{\leq 300\,\mathrm{ms}}\ done]$ fails. Of all the runs that miss the deadline, the most likely is not the give-up run but a success on the third attempt. That is the run worth fixing first.

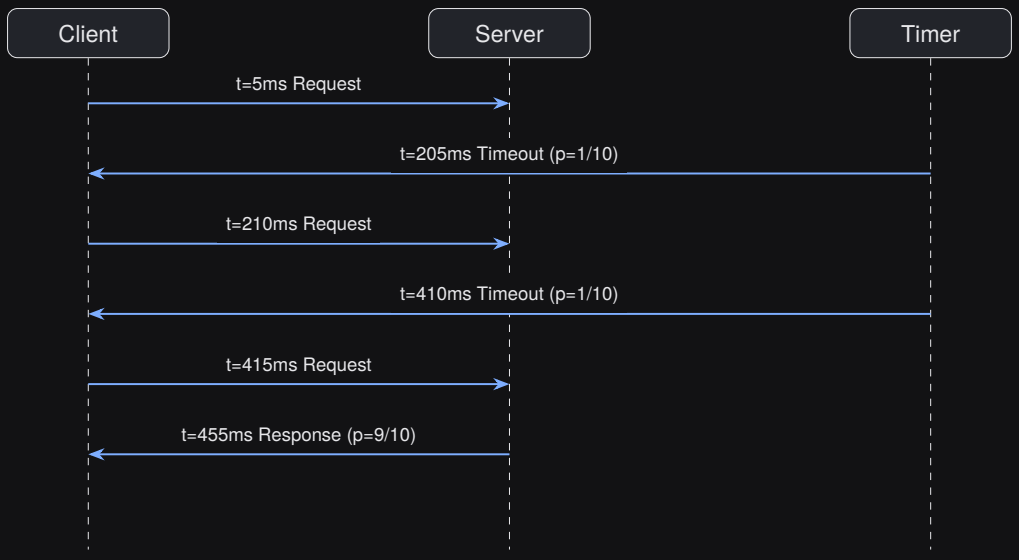


Figure 13.4: The most likely counterexample to the 99.9% / 300 ms objective: success on the third attempt at 455 ms, probability $9/1000$.

13.4 The distribution behind the numbers

The completion-time distribution is computed exactly by enumerating the generated runs; it is not sampled. The logarithmic scale shows the long tail that a mean of 67.71 ms hides.

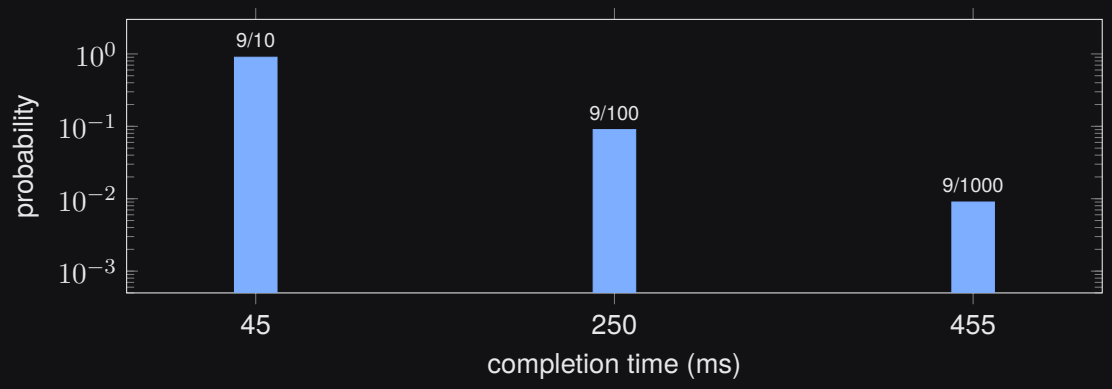


Figure 13.5: Completion times of successful runs under the declared overlay. The remaining $1/1000$ gives up at 615 ms.

13.5 What comes next

The same functions accept a measured overlay, reduced from recorded traces by counting how often each branch was taken and averaging the delay before each message. Chapter 4 does exactly that for 20,000 recorded sales, compares the result with a declared overlay, and shows why measured delays must eventually be distributions rather than means. Retry loops here are unrolled into bounded states; unbounded loops need the linear-equation solution of probabilistic model checking rather than run enumeration, and the current implementation reports them instead of truncating them.

Chapter 14

Feature Completion as Positive and Negative Proof

A request to show a message, list source, accept a transaction, render a diagram or produce an MP4 states a desirable observation. The change can also introduce stale state, disclosure, resource exhaustion, weaker authentication, corruption or success without an artifact. Completion therefore requires a paired argument:

1. a *positive invariant* states what the system must do; and
2. applicable *negative invariants* state what the system must never do.

For a feature that crosses a boundary, the positive invariant is a property of the global type, and the negative ones usually name a message that must never be sent or accepted under a channel's physics: a replayed request, a response to the wrong session, a call after its deadline. Review is proportional: button spacing needs no cryptographic audit, filesystem access needs authorization and path analysis, and a material risk without a mapped, tested control blocks completion.

14.1 The logical shape of a feature

Let $good_f$ mean the observable contract of feature f, and let $bad_{f,1}, \ldots, bad_{f,n}$ name reachable failures introduced or exposed by that feature. A common safety-shaped completion condition is

$$AG\, good_f \;\wedge\; \bigwedge_{i=1}^{n} AG\, \neg bad_{f,i}.$$

The first conjunct is positive; each other is negative: no permitted execution reaches a forbidden observation. The counterexample form

$$AG\, \neg bad \;\equiv\; \neg EF\, bad$$

is what a negative test does: it tries to build the $EF\, bad$ witness (another user's project path, a stale response after a project switch, an oversized upload, a secret in a log), and finding one refutes completion. Some features are liveness obligations, such as $AG(submitted \rightarrow AF(complete \vee failed))$; review must not swap liveness for safety or hide fairness and availability assumptions.

14.2 Stability review before security review

Before editing code, identify the existing invariant and review the applicable ways an environment change could break it:

Regression. Which accepted behavior, file format, route, proof, or UI state can this change invalidate? Preserve conforming behavior and add a regression test at the narrowest durable boundary.

Failure. What happens when a dependency, renderer, compiler, provider, filesystem operation, or network request fails? Failure must become an explicit terminal observation rather than a false success or endless wait.

Concurrency. Can a late response, simultaneous session, queued prompt, or overlapping write be attributed to the wrong project or revision? Correlation identity and revision checks must survive reordering.

Resources. What bounds memory, bytes, queue occupancy, recursion, process time, media dimensions, and response size? A new feature must not silently turn a finite resource contract into an unbounded one.

Durability and recovery. Which state is authoritative after restart or partial failure? Atomic writes, rebuildable indexes, and project-local ownership make recovery inspectable.

Compatibility. Which clients, stored revisions, generated artifacts, and build environments must continue to work? Version or migrate a changed representation rather than reinterpreting old bytes silently.

Record the concrete invariants at the changed boundary, and say briefly when a dimension is non-material.

14.3 Security as forbidden observable behavior

“A user never reads a project they do not own” states protected behavior; “use authentication” names a mechanism. Before choosing a control, identify:

1. the trust boundary crossed by the feature;
2. the asset being protected;
3. the attacker’s relevant capability; and
4. the forbidden observable outcome.

Forbidden behaviors include unauthorized access, cross-user or cross-project disclosure, secret persistence or logging, injection, path or symlink escape, replay, request/response confusion, privilege expansion, executable media, resource exhaustion and failing open. State them testably (“a source path absent from the authorized inventory never returns bytes”), and map each material one to a control (path confinement, authenticated identity, an allowlist, a byte limit, an atomic revision check) and a negative test or proof; a mechanism alone proves nothing.

14.4 The feature-completion ledger

For each feature, retain the following compact traceability record in the typed requirements and implementation mapping:

Field	Required content
Prompt	The stakeholder request whose completion is being claimed.
Positive claim	Observable always or liveness property, with actor, scenario, boundary, and revision.
Negative claims	Applicable never or always-not properties; omitted families are explicitly judged non-material.
Controls	Concrete implementation mechanisms mapped one-to-one or many-to-one to the negative claims.
Evidence	Positive conformance tests plus negative tests, proofs, generated traces, and named checks.
Residual risk	Stated limitation the stakeholder reviews in the diff, assumption, deferred mitigation, or an explicit completion blocker.

Both sides need evidence: ruling out failures does not demonstrate the feature, and a demonstration does not rule out disclosure or corruption.

14.5 Worked example: the project Source view

“Show implementation files in the Source tab.” Positive: selecting an authorized project lists its files, and selecting a readable UTF-8 file shows it; switching projects clears the selection, so a late response cannot show stale content. The feature crosses a filesystem boundary, protecting source, account records, repository metadata and credentials from a user who can choose URL paths and guess names. Negative invariants:

- a user never reads another user’s project source;
- a guessed ignored or metadata path never returns bytes;
- a path or symlink never escapes the authorized source root;
- a binary or oversized file never becomes an unbounded text response;
- a project switch never retains the previous project’s source state; and
- an empty source directory never accompanies a claim that implementation succeeded.

Controls are an authenticated explicit project ID, canonical root resolution, exact source-inventory membership, UTF-8 reads, a finite response limit and project-keyed React state. Negative tests request another account’s file and a guessed `.git/config`; both return not-found without revealing existence. The positive test creates, lists and opens a project-local file and checks its exact content.

14.6 Worked example: chat auto-scroll

Even a small convenience has stability boundaries. Positive: loading a conversation and every new message reveal the bottom of the chat pane, scrolling that pane, not the page or the sidebar, and resetting with the project-keyed component. No authorization boundary is crossed, so a threat model would be noise, but project isolation and stale responses still matter.

14.7 When the result is not complete

The ledger is incomplete with an untested positive claim, a missing negative requirement, an uncontrolled risk, a control without a negative test or proof, a liveness claim without its fairness assumptions, skipped validation, or a check of a substitute artifact. Report the blocker and return to the relevant layer, stating any residual risk where the stakeholder will see it before committing. A green build or a persuasive diagram does not fill these gaps.

Chapter 15

Lean as the Rendered Language

The notation for global types, local types and channel physics renders to a conservative subset of Lean, with no second opaque specification file: domain declarations are Lean definitions, behaviours Lean values, and requirements Lean propositions or checked queries. Rendering the proposed scenario DSL into Lean is a **future interface**; the typed LeanFM API is the current **executable code**, and a **Lean proof** claim applies only to a theorem checked in the declared build. Every displayed formula carries an English reading, in its accessible text, that accounts for each operator, index and bound.

```
def checkout : Grammar CheckoutEvent :=
  request >>> (accepted <|> rejected)

def eventuallyAnswered : CTL Observation :=
  CTL.ag (CTL.implies requestPending
    (CTL.af (CTL.or requestAccepted requestRejected)))
```

Semantic machinery comes from a fixed library; an author supplies only:

- typed actors, identifiers, event kinds, and payloads;
- parameterised grammar definitions;
- projections from trace prefixes to observable propositions;
- named temporal requirements and proofs or decidable checks.

so generated Lean stays small enough to review, and library proofs and algorithms are reused, not regenerated.

15.1 An unchecked exploratory fixture

`book/logic.lean`, below, is an unchecked side experiment, not part of the method. It explores a regularized assignment for $1 + 2 + 3 + \cdots$: for each cutoff n, the triangular prefix $n(n+1)/2$ plus a defined correction `Tail_S`(n) equals $-1/12$, which asserts nothing about convergence of the partial sums. It needs Mathlib, which this build lacks, so it claims no checked proof.

```
import Mathlib

set_option linter.style.whitespace false
set_option linter.unusedVariables false

/-!
# 1 - 1 + 1 - ... = 1/2,  1 - 2 + 3 - ... = 1/4,  1 + 2 + 3 + ... = -1/12

Everything lives in ℚ. No limits, no convergence, and no values are given:
all three totals are derived.

Partial sums:
* `Sum_G n` : 1 - 1 + 1 - 1 + ...   (n terms)
* `Sum_B n` : 1 - 2 + 3 - 4 + ...   (n terms)
* `Sum_S n` : 1 + 2 + 3 + ... + n

Three finite identities, proved for every n:
* `idG` : Sum_G (n+1) + Sum_G n       = 1
* `idB` : Sum_B (n+1) + Sum_B n       = Sum_G (n+1)
* `idS` : Sum_S (2n)  - Sum_B (2n)    = 4 * Sum_S n

The rules of the system (hypotheses, never baked into definitions):
1. A total does not depend on n:  Sum n + Tail n = constant
2. Tails obey the same three identities (the constant 1 in idG has no tail,
   so its tail version has 0 on the right)

Only these three identities are admitted. Admitting *every* finite identity
is inconsistent (e.g. the shift identity for S), so the list is explicit
and `tails_exist` proves this list is consistent.
-/

/-! ## Partial sums -/

/-- 1 - 1 + 1 - 1 + ... (n terms) -/
def Sum_G (n : ℕ) : ℚ :=
  (1 - (-1)^n) / 2

/-- 1 - 2 + 3 - 4 + ... (n terms) -/
def Sum_B (n : ℕ) : ℚ :=
  (1 - (2 * (n : ℚ) + 1) * (-1)^n) / 4

/-- 1 + 2 + ... + n -/
def Sum_S (n : ℕ) : ℚ :=
  n * (n + 1) / 2

/-! ## The finite identities -/

theorem idG (n : ℕ) :
    Sum_G (n+1) + Sum_G n = 1 := by
  unfold Sum_G
  ring

theorem idB (n : ℕ) :
    Sum_B (n+1) + Sum_B n = Sum_G (n+1) := by
  unfold Sum_B Sum_G
  push_cast
  ring

theorem idS (n : ℕ) :
    Sum_S (2*n) - Sum_B (2*n) = 4 * Sum_S n := by
  have h : ((-1 : ℚ))^(2*n) = 1 := by
    rw [pow_mul]
    norm_num
  unfold Sum_S Sum_B
  rw [h]
```

```
  push_cast
  ring

/-! ## The rules force all three totals -/

/-- Any tails obeying the rules force G = 1/2, B = 1/4, S = -1/12.
    None of these numbers appears in the hypotheses. -/
theorem totals_forced (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)                    -- rule 1
    (hB : ∀ n, Sum_B n + TB n = b)                    -- rule 1
    (hS : ∀ n, Sum_S n + TS n = s)                    -- rule 1
    (tG : ∀ n, TG (n+1) + TG n = 0)                   -- rule 2 for idG
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))            -- rule 2 for idB
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n) :      -- rule 2 for idS
    g = 1/2 ∧ b = 1/4 ∧ s = -1/12 := by
  have hg : g = 1/2 := by
    have i := idG 0
    have a := hG (0+1)
    have c := hG 0
    have t := tG 0
    linarith
  have hb : b = 1/4 := by
    have i := idB 0
    have a := hB (0+1)
    have c := hB 0
    have d := hG (0+1)
    have t := tB 0
    linarith
  have hs : s = -1/12 := by
    have i := idS 1
    have a := hS (2*1)
    have c := hB (2*1)
    have d := hS 1
    have t := tS 1
    linarith
  exact ⟨hg, hb, hs⟩

/-! ## The rules are consistent -/

/-- Witness tails (used only to show the rules can be met). -/
def Tail_G (n : ℕ) : ℚ := 1/2 - Sum_G n
def Tail_B (n : ℕ) : ℚ := 1/4 - Sum_B n
def Tail_S (n : ℕ) : ℚ := -1/12 - Sum_S n

theorem tails_exist : ∃ TS TB TG : ℕ → ℚ,
    (∀ n, Sum_G n + TG n = 1/2) ∧
    (∀ n, Sum_B n + TB n = 1/4) ∧
    (∀ n, Sum_S n + TS n = -1/12) ∧
    (∀ n, TG (n+1) + TG n = 0) ∧
    (∀ n, TB (n+1) + TB n = TG (n+1)) ∧
    (∀ n, TS (2*n) - TB (2*n) = 4 * TS n) :=
  ⟨Tail_S, Tail_B, Tail_G,
    fun n => by unfold Tail_G; ring,
    fun n => by unfold Tail_B; ring,
    fun n => by unfold Tail_S; ring,
    fun n => by
      have := idG n
      unfold Tail_G
      linarith,
    fun n => by
      have := idB n
      unfold Tail_B Tail_G
      linarith,
    fun n => by
      have := idS n
      unfold Tail_S Tail_B
```

```
      linarith⟩

/-! ## The tail is determined by the Sum -/

theorem tails_unique (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)
    (hB : ∀ n, Sum_B n + TB n = b)
    (hS : ∀ n, Sum_S n + TS n = s)
    (tG : ∀ n, TG (n+1) + TG n = 0)
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n)
    (n : ℕ) :
    TS n = Tail_S n := by
  obtain ⟨_, _, hs⟩ := totals_forced TS TB TG s b g hG hB hS tG tB tS
  have := hS n
  unfold Tail_S
  linarith

/-! ## Final statement -/

theorem Goal (TS TB TG : ℕ → ℚ) (s b g : ℚ)
    (hG : ∀ n, Sum_G n + TG n = g)
    (hB : ∀ n, Sum_B n + TB n = b)
    (hS : ∀ n, Sum_S n + TS n = s)
    (tG : ∀ n, TG (n+1) + TG n = 0)
    (tB : ∀ n, TB (n+1) + TB n = TG (n+1))
    (tS : ∀ n, TS (2*n) - TB (2*n) = 4 * TS n)
    (n : ℕ) :
    -1/12 = n * (n + 1) / 2 + TS n := by
  obtain ⟨_, _, hs⟩ := totals_forced TS TB TG s b g hG hB hS tG tB tS
  have := hS n
  unfold Sum_S at this
  linarith

#print axioms Goal
```

For a LeanFM runtime judgment, define the double turnstile $\models$ relative to the declared transition model and initial state. It means “at every reachable state, the premises require at least one of these conclusions.” Write it in a rule-like form, familiar from Prolog:

$$a \wedge b \wedge \cdots \wedge w \;\models\; x \vee \cdots \vee z.$$

Its executable definition is that the following CTL formula holds at the declared initial state:

$$AG\big((a \wedge b \wedge \cdots \wedge w) \rightarrow (x \vee \cdots \vee z)\big).$$

This is entailment within the declared model and initial state, not general entailment over all models, and the conclusion that holds may differ between states. In code, a counterexample is a reachable state where every premise is true and every conclusion false:

```
counterexample(state, premises, conclusions) :=
  premises.all(p => evaluate(p, state)) &&
  !conclusions.any(q => evaluate(q, state))

entails(successors, initialState, premises, conclusions) :=
  !anyReachable(successors, initialState,
    state => counterexample(state, premises, conclusions))
```

A sampled trace checks only that sample. Symbolically:

$$\neg EF(a \wedge b \wedge \cdots \wedge w \wedge \neg x \wedge \cdots \wedge \neg z).$$

which is the AG implication by $AG\,p = \neg EF\,\neg p$. With one conclusion it is the Prolog rule `x :- a, b, ..., w`, without Prolog's search order. $\Gamma \models p$ collects the premises into Γ; $\Gamma \vdash p$, by contrast, asserts a derivation.

For the proof-theoretic background behind derivations, natural deduction, sequent calculus, normalization, and cut elimination, see Mancosu, Galvan, and Zach's *An Introduction to Proof Theory* [17].

15.2 From specification to service

Lean hosts executable code, including LeanFM's own web server, so the formal artifact is part of the running project. The target is for typed definitions to drive validation endpoints, trace generation, schemas, views and runtime monitors; coverage is partial (Chapter 18), and a checked behavioural model stays distinct from unverified infrastructure.

Chapter 16

Scope and Open Questions

The implementation backlog and audit findings live in `CRITIQUE.md`. This chapter distinguishes current evidence from open technical questions using the labels defined in Chapter 1; proposed backlog items are **future interfaces**, not implemented behavior.

Self-hosting consolidation

The canonical requirements compose the ChatUI specification with a proposed revision-bound requirements-to-PR workflow. `make selfhost-check` validates the composition and regenerates `SELF_HOSTING_REVIEW.md` with its interaction diagram and state machine, and a component registry preserves decisions across fresh sessions. Knowledge is now decided (the `knows` tool and atom, `check_protocol`); policy checking remains deferred, as do check invalidation, probability interpretations and PR evidence. Error protocols, formula semantics and complete traceability remain prerequisites for a one-shot specification.

16.1 Proof obligations

The initial metatheory is session-type metatheory with the channel physics added. It should make the following questions precise:

- Which global types are well formed: every choice directed by one role, and every role that must learn a branch either told by a message or released by a declared timer?
- Does projection to local types preserve the global type, so that roles following their local types over the declared channels produce only runs of the global type (session fidelity)?
- Which channel physics preserve communication safety and progress, and which (loss, duplication, early timers) require the protocol itself to recover, as TFTP's retransmission does?
- Does a recorded production event stream conform to the global type, and does each event's `causeId` stay within its sender's own history?
- Under what finiteness restrictions is CTL checking decidable and executable?
- Which guards can be compiled into finite monitor state?

- When does modular composition preserve an established temporal property?
- Which fairness and timing assumptions are required by a liveness theorem?
- Which observation and policy relations justify each knowledge, permission, and obligation assertion, and which bounds make their combination with CTL executable?

Part II

Security Protocol Exercises

Chapter 17

Protocol Exercises: Roles, Attackers, and Knowledge

The author learned from Ryan et al. [22] to treat a security protocol as communicating roles plus an active intruder, and security as properties of every trace that intruder can produce. This chapter keeps that idea and none of that book's examples. Three small invented protocols each press on one part of the method:

1. an *on-call handoff*, where three separately written roles must compose into one protocol;
2. a *sealed-bid auction*, where an attacker who cannot read a bid can still copy it; and
3. *speaker pairing*, where a code shown on a screen is visible to some actors and not to others.

Status: the three examples are a design exercise. Their `RoleSpec`, `WireTerm` and `composeRoles` listings are proposed notation, and there is no role-composition engine yet. The attacker's deduction rules are implemented (`LeanFM.Session.Knowledge`), and so is a checker that generates the attacker from a declaration (`LeanFM.Session.Attack`), which Section 17.8 uses on a real protocol.

17.1 Roles compose by sharing events

This is the session-type picture seen from the other side: each role is written first, as a grammar over its own view of the events, and the global protocol is whatever their composition admits. Composition synchronizes on shared events; it does not concatenate scripts. The attacker is the role session types usually leave out: one that controls the channels and can read, drop, replay or forge what passes through them, the most hostile physics a channel can declare (Chapter 1).

```
structure RoleSpec where
  actor    : Actor
  accepts  : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor

def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t, let $t\restriction_a$ keep only the events actor a sends or receives. The composition accepts a trace exactly when every role accepts its own view of it:

$$L(\mathrm{composeRoles}(R_1, \ldots, R_n)) = \{t \mid \forall i,\ t\restriction_{a_i} \in L(R_i)\}.$$

A message is one global event with one identity, one byte string, one sender and one recipient, and both endpoint grammars constrain it. An unmatched send, an unexpected receive, or disagreement about the bytes empties the composed language from that prefix on, and the checker reports the prefix. A role may bind values from events it has seen and test later events against them; the bindings are a parser environment for the trace, not a claim about implementation memory.

17.2 A transport-neutral byte algebra

Security claims need to say which bytes expose which values, without choosing HTTP, protobuf or a cipher suite. Four symbolic constructors are enough for this chapter:

```
abbrev Bytes := ByteArray

inductive WireTerm where
  | atom    : Name -> Bytes -> WireTerm
  | tuple   : List WireTerm -> WireTerm
  | encrypt : KeyRef -> WireTerm -> WireTerm
  | hash    : WireTerm -> WireTerm

structure WireEvent where
  timeAt   : ClockTimestamp
  id       : EventId
  priorIds : List EventId
  session  : SessionId
  task     : TaskId
  from     : Actor
  to       : Actor
  channel  : Channel
  term     : WireTerm
  bytes    : Bytes
  encodedBy : bytes = encodeCanonical term
```

Anyone may hash a term they know; nobody may invert a hash; decryption needs the key. `channel` records who can observe an event at all, which the pairing example needs. A concrete refinement replaces this idealized algebra with real transports and cryptography whose assumptions justify it.

17.3 Example 1: an on-call handoff

A pager service records who is on call for a team. Exactly one engineer is on call at any time. Olu is on call and wants to hand off to Ines. The pager must not move the duty until Ines has actually accepted it, and Olu must learn that the handoff completed. Each person shares a long-term key with the pager: K_O for Olu, K_I for Ines.

1. Olu $\rightarrow$ Pager: $\{Ines, h\}_{K_O}$, with a fresh handoff id h.
2. Pager $\rightarrow$ Ines: $\{Olu, h, c\}_{K_I}$, with a fresh challenge c.
3. Ines $\rightarrow$ Pager: $\{h, c, accept\}_{K_I}$.
4. Pager $\rightarrow$ Olu: $\{Ines, h, done\}_{K_O}$.

17.3.1 Three roles, written separately

```
def oluRole (olu ines pager : Actor) (ko : SharedKey) : RoleSpec :=
  role olu "outgoing engineer" do
    let h <- fresh Nonce "h"
    from_ olu; to_ pager
    send (encrypt ko.ref (tuple [actor ines, nonce h]))
    from_ pager; to_ olu
    receive (encrypt ko.ref (tuple [actor ines, nonce "h", atom "done"]))
      where_ field "h" == h

def inesRole (olu ines pager : Actor) (ki : SharedKey) : RoleSpec :=
  role ines "incoming engineer" do
    from_ pager; to_ ines
    receive (encrypt ki.ref (tuple [actor olu, nonce "h", nonce "c"]))
    let h <- take Nonce "h"
    let c <- take Nonce "c"
    from_ ines; to_ pager
    send (encrypt ki.ref (tuple [nonce h, nonce c, atom "accept"]))

def pagerRole (olu ines pager : Actor) (ko ki : SharedKey) : RoleSpec :=
  role pager "pager service" do
    from_ olu; to_ pager
    receive (encrypt ko.ref (tuple [actor "next", nonce "h"]))
      where_ field "next" == ines
    let h <- take Nonce "h"
    let c <- fresh Nonce "c"
    from_ pager; to_ ines
    send (encrypt ki.ref (tuple [actor olu, nonce h, nonce c]))
    from_ ines; to_ pager
    receive (encrypt ki.ref (tuple [nonce "h", nonce "c", atom "accept"]))
      where_ field "h" == h && field "c" == c
    from_ pager; to_ olu
    send (encrypt ko.ref (tuple [actor ines, nonce h, atom "done"]))
```

The `where_` clauses are the context-sensitive part: the message sequence is context-free, but whether a reply is legal depends on values bound earlier in the trace.

17.3.2 What composition catches

Suppose a later edit lets the pager send *done* as soon as it has sent the offer, before Ines replies. Olu's role still accepts that order: Olu never sees messages 2 and 3. Ines's role still accepts it too. Only the composed protocol exposes the problem, through the duty properties below:

$$AG\ exactlyOneOnCall.$$

$$onCall(Olu)\ AW\ accepted(Ines, h).$$

$$AG(toldDone(Olu, h) \rightarrow \mathbf{P}\ accepted(Ines, h)).$$

The second formula uses weak until on purpose: Ines may never answer, and then Olu simply remains on call. Strong AU would wrongly demand that every handoff eventually succeeds. The third is a past claim ($\mathbf{P}$ reads "previously"). With the faulty edit, the checker returns a counterexample trace in which Olu is told *done* with no earlier acceptance, drawn both as the global interaction diagram and as Olu's local view, in which nothing looks wrong.

The same mechanism catches disagreement about values. If the pager's grammar could forward the offer to someone other than the engineer Olu named, the `where_ field "next" == ines` guard and Ines's projection cannot both accept the same event, and the composed language is empty at that prefix.

17.4 Example 2: a sealed-bid auction

An auctioneer collects bids in two phases. While bidding is open, each bidder sends a *commitment*: the hash of the amount and a fresh random salt. After bidding closes, each bidder reveals the amount and salt, and the auctioneer accepts a reveal only if it hashes to the commitment. The highest revealed bid wins. Ana and Ben bid honestly. Ivo is also a registered bidder; he is the attacker, and he sees every message on the network.

```
def bidderRole (b auctioneer : Actor) (amount : Amount) : RoleSpec :=
  role b "bidder" do
    let s <- fresh Nonce "salt"
    from_ b; to_ auctioneer
    send (tuple [actor b, hash (tuple [value amount, nonce s])])   -- commit
    from_ auctioneer; to_ b
    receive (atom "closed")
    from_ b; to_ auctioneer
    send (tuple [actor b, value amount, nonce s])                  -- reveal

def auctioneerRole (auctioneer : Actor) (bidders : List Actor) : RoleSpec :=
  role auctioneer "auctioneer" do
    let commits <- collectWhileOpen fun b =>
      receive (tuple [actor b, ?commitment]) where_ b in bidders
    broadcast (atom "closed")
    forEach commits fun (b, commitment) =>
      receive (tuple [actor b, ?amount, ?salt])
        where_ hash (tuple [amount, salt]) == commitment
    announce (highestRevealed commits)
```

17.4.1 Secrecy holds, and the auction is still broken

Before close, Ivo has seen Ana's commitment but cannot invert the hash, so he does not know her amount. That secrecy property holds:

$$\neg\,\mathrm{knows}(\mathit{Ivo}, a_{\mathit{Ana}})\ \mathit{AU}\ \mathit{closed}.$$

The auction is still broken. Ivo does not need to know Ana's amount to bid it. He copies her commitment and sends it under his own name. That is a legal attacker move: he observed the hash term, and resending a known term is always derivable. After close he waits for Ana's reveal, which he also observes, and sends the same amount and salt under his name. The auctioneer's check passes, because the hash really does match. Ivo has tied Ana's bid without ever having known it while bidding was open.

The property that fails is about who *formed* each commitment, not about who knows what:

$$\mathit{AG}\big(\mathrm{acceptsCommit}(\mathit{Auctioneer}, b, x) \rightarrow \mathbf{P}\ \mathrm{formed}(b, x)\big).$$

The checker's search for a violation, $\mathit{EF}(\mathrm{acceptsCommit}(\ldots) \wedge \neg\mathbf{P}\,\mathrm{formed}(\ldots))$, returns the copy trace above as its witness.

17.4.2 The fix: bind the bidder into the commitment

Put the bidder's own name inside the hash: $\mathit{hash}(b, \mathit{amount}, s)$, and have the auctioneer recompute the hash with the name of the bidder who sent the reveal. Ivo's copied commitment was formed with Ana's name. To open it under his own name he would need a salt that makes $\mathit{hash}(\mathit{Ivo}, \ldots)$ equal $\mathit{hash}(\mathit{Ana}, \ldots)$, and no rule derives one. Copying is now useless.

This is the acceptance test for an intruder model: it must find the copy trace for the original commitment and none for the fixed one. A model that cannot find the first attack is too weak to trust when it finds nothing in the second.

The symbolic model has a visible limit here. If amounts came from a small set, Ivo could hash each candidate and compare; the salt stops that. Dolev–Yao deduction has no notion of guessing, so "salts are fresh and unguessable" must be written down as an assumption.

17.5 Knowledge and agreement as theorems

The attacker is one more role. It observes the events its channels let it see, may block or replay them, and may send any term it can derive from them.

```
inductive Derives : Knowledge -> WireTerm -> Prop where
  | initial : term in knowledge -> Derives knowledge term
  | tuple   : (forall x in xs, Derives knowledge x) ->
              Derives knowledge (.tuple xs)
  | project : Derives knowledge (.tuple xs) -> x in xs ->
              Derives knowledge x
  | encrypt : Derives knowledge key -> Derives knowledge body ->
              Derives knowledge (.encrypt key body)
  | decrypt : Derives knowledge (.encrypt key body) ->
```

```
              Derives knowledge (inverse key) -> Derives knowledge body
  | hash    : Derives knowledge x -> Derives knowledge (.hash x)
  -- no rule recovers x from .hash x

def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe (channelsVisibleTo ivo)
    mayBlock
    mayReplay
    maySend fun knowledge term => Derives knowledge term
```

Knowledge is calculated for every actor from every accepted trace, not only the intended run, so security claims become theorems over accepted traces, derived knowledge and explicit compromise assumptions. For the fixed auction:

```
theorem auction_bid_sealed_until_close
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace)
    (fresh : SaltsFreshAndUnguessable trace) :
    forall b x, beforeClose trace x ->
      not (Derives (knowledgeAt .ivo trace x) (value (amountOf b))) := by
  exact hashHidesSaltedInput accepted fresh

theorem auction_commitments_not_copyable
    (trace : Trace WireEvent)
    (accepted : fixedAuction.Accepts trace) :
    forall b c, acceptsCommit trace b c -> formedBy trace b c := by
  exact boundNameDefeatsCopy accepted
```

The two theorems are still proposed notation: the lemmas they name do not exist yet. The deduction rules do. `LeanFM.Session.Knowledge` defines them as `Derives`, over atoms, symmetric keys, key pairs, pairs, encryption and hashes; a ciphertext under a private key is a signature, opened with the public key. It decides them one value at a time: `analz` takes what an actor has seen apart until nothing new appears, and `synth` asks whether the value can be built from the pieces. (Enumerating everything an actor could build would never end.) The decision is proved sound:

```
theorem knows_sound {K : List Msg} {v : Msg}
    (h : knows K v = true) : Derives K v
```

So when the checker says the attacker knows a value, a derivation exists. Completeness, that `knows` misses nothing `Derives` allows, is tested on examples but not yet proved, and secrecy verdicts lean on it.

17.5.1 Calculated knowledge and policy inside CTL

Knowledge and policy need not start as new modal operators. Define state predicates by reducing the event history, then use them inside ordinary CTL. `knows` says who can derive a value; `allows` and `requires` say what a declared policy permits and obliges:

```
def knows (actor : Actor) (value : Secret) (s : ProtocolState) : Bool :=
  value in knowledgeClosure actor s.events s.compromise
```

```
def allows (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.allows actor action s

def requires (policy : Policy) (actor : Actor)
    (action : Action) (s : ProtocolState) : Bool :=
  policy.requires actor action s
```

These functions label a state; CTL says when the labels may or must hold:

$$\neg\,\mathrm{knows}(\mathit{Auctioneer}, a_b)\ AU\ \mathit{closed}.$$

$$EF\ \mathrm{knows}(\mathit{Auctioneer}, a_b).$$

$$AG\big(\mathrm{knows}(x, c) \to \mathrm{allows}(\mathit{DutyPolicy}, x, \mathrm{possess}(c))\big).$$

$$AG\big(\mathit{dutyMoved}(h) \to (\mathrm{requires}(\mathit{DutyPolicy}, \mathit{Pager}, \mathrm{notify}(\mathit{Olu}, h)) \wedge AF\ \mathit{toldDone}(\mathit{Olu}, h))\big).$$

A universal claim is checked by searching for its violation, and the search result is the useful artifact:

$$AG(\mathrm{discloses}(a, s) \to \mathrm{allows}(P, a, \mathrm{disclose}(s))) \ \equiv\ \neg EF(\mathrm{discloses}(a, s) \wedge \neg\,\mathrm{allows}(P, a, \mathrm{disclose}(s))).$$

This state-local `knows` is not the epistemic modality $K_a\varphi$ over indistinguishable histories of Chapter 11. It tests whether one deduction closure contains a value, which suffices for most secrecy and authorization claims; K_a is for requirements that compare histories, as the next example shows.

17.6 Example 3: pairing a speaker by a code on its screen

A new speaker shows a six-digit code d on its own display. Its owner types the code into a phone, and the phone proves to the speaker that it knows d. The network is hostile; the display is not on the network.

1. Speaker $\to$ Owner, on channel *display*: d.
2. Owner $\to$ Phone, on channel *keypad*: d.
3. Phone $\to$ Speaker, on channel *network*: $p,\ \mathit{hash}(p, d, \mathit{Phone})$, with a fresh nonce p.
4. Speaker $\to$ Phone, on channel *network*: *paired* or *rejected*.

```
receive (tuple [nonce "p", ?proof]) on network
  where_ sender != speaker
  where_ proof == hash (tuple [nonce p, value shownCode, actor sender])
  where_ failedAttempts < 3
```

Three things are new compared with the first two examples.

Channels decide what each actor observes. The display and keypad events belong to the observations of the speaker, the owner and the phone, not to Ivo's. That is what keeps d out of Ivo's knowledge closure, and it is a modeling assumption about the room, so it is stated as one. Remove it and the checker should immediately find Ivo pairing his own phone.

A counter is context, too. `failedAttempts < 3` depends on how many earlier proofs failed in this trace. Like the equality guards, it keeps the grammar context-free and moves the counting into a named predicate.

Guessing is a probability, not a deduction. Ivo cannot derive d, but he can guess it. The symbolic model says $EF\ paired(Ivo)$ is true: some future has a lucky guess. That verdict is correct and useless on its own. The requirement is a bound on how likely that future is, carried by a probability overlay: with three attempts against 10^6 codes,

$$P[F\ paired(Ivo)] \leq 3/10^6.$$

The next section explains why the number attaches to E.

Some claims compare histories. "Ivo cannot tell which code was shown" is not true or false of one state: it says any two histories differing only in d look identical to Ivo. That is K_a over indistinguishable histories, a relational theorem rather than a CTL state formula, and the one claim in this chapter that the table below does not cover.

17.7 Where each modality attaches

The examples have used every kind of claim a requirement needs: what can or must happen, how likely, by when, who knows, who may, and who must. None needs a new logic. A CTL operator is a path quantifier, A or E, fused with a temporal operator such as F, G or U: probability belongs to the quantifier, deadlines to the temporal operator, and knowledge and policy are predicates on states.

The path quantifier E invites "how likely?" E says that *some* future satisfies the path formula; the refinement replaces "some" with a probability over futures, whatever follows the E:

$$EF\,p \rightsquigarrow P[F\,p], \qquad EG\,p \rightsquigarrow P[G\,p], \qquad p\ EU\ q \rightsquigarrow P[p\ U\ q].$$

The number comes from an overlay of branch probabilities, as in the bakery chapter, and is reported beside the CTL verdict, never in place of it. The overlay cannot make $EF\,p$ false: a future with probability $3/10^6$ is still possible, and so still needs the handling that possibility demands. Nor does A become "probability one": a set of futures can have probability one and still leave out a future that A must cover, which is why Chapter 11 keeps A and E about legal futures only.

The temporal operator F invites "by when?" F has no bound, and a user waiting on a handoff needs one, so the deadline is written on F (and on the right of U): $AF_{\le t}\,q$ says every future reaches q within t, $EF_{\le t}\,q$ that some future does. A deadline under A is a promise the grammar must keep: every waiting state needs a timeout branch ending in an explicit outcome, or some future waits past t.

$$AF_{\le 15\,\mathrm{min}}\big(accepted(Ines, h) \lor expired(h)\big).$$

$$AG\Big(closed \to AF_{\le T_{reveal}}\big(revealed(b) \lor forfeited(b)\big)\Big).$$

So the handoff needs an *Expired* message to both engineers when the offer times out, and the auction a *Forfeited* outcome for a bidder who never reveals: new legal branches, with decodable messages and code that handles them.

The two annotations compose. Probability on the quantifier and a deadline on F combine into the question stakeholders usually ask: how likely is it to happen in time? An owner pairing a speaker within a minute is

$$P[F_{\le 60\,\mathrm{s}}\, paired(Phone)],$$

which a measured overlay reports from real pairing attempts. Keep a guaranteed deadline ($AF_{\le t}$, enforced by a timeout branch) separate from a usual one ($P[F_{\le t}]$, measured). The bakery chapter shows why: a declared delay promised a 1.2 s sale while only about half the recorded sales met it.

`knows`, `allows` and `requires` complete the set. These are about actors rather than futures, and each is computed from the history up to a state, so the temporal operators carry them unchanged. Secrecy is `knows` under AU; authorization is `knows` implying `allows` under AG; an obligation is `requires` paired with $AF_{\le t}$ on the act that discharges it.

Question	Attaches to	Example from this chapter
can it happen? must it?	E, A	$EF\, paired(Ivo)$
how likely?	E	$P[F\, paired(Ivo)] \le 3/10^6$
by when?	F, U	$AF_{\le 15\,\mathrm{min}}(accepted \lor expired)$
who knows?	state: `knows`	$\neg\,\mathrm{knows}(Ivo, a_{Ana})\ AU\ closed$
who may?	state: `allows`	$AG(\mathrm{knows}(x, c) \to \mathrm{allows}(Duty, x, \ldots))$
who must?	state: `requires`	$AG(dutyMoved \to \mathrm{requires}(\ldots) \land AF\ toldDone)$

17.8 Needham–Schroeder, checked

The public-key protocol of Needham and Schroeder (1978) has three messages:

```
protocol NS (A, B) {
  m1: Msg1 from A to B;
  m2: Msg2 from B to A cause m1 where m2.na = m1.na;
  m3: Msg3 from A to B cause m2;
}
```

Alice runs it once, with a peer she chooses: Bob, or Ivo. Bob answers once, whoever claims to be calling. Ivo is a legitimate user with his own key pair, so Alice may well choose him; he is also the network, seeing every message and sending anything he can derive.

Nobody builds the state space. Each honest role is a short script over message patterns: a choice, sends, receives and claims; a variable in a received pattern binds whatever the message carries there.

```
def alice (v : Variant) : Role := { name := "Alice", steps :=
  [ .choose "peer" [.atom "B", .atom "I"]
  , .claim "running" [name "A", .var "peer"]
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.tuple [.lit na, name "A"]))
  , .recv (.enc (.pk (name "A")) (match v with
      | .original => .tuple [.lit na, .var "n"]
      | .lowe => .tuple [.lit na, .var "n", .var "peer"]))
  , .send (.var "peer") (.enc (.pk (.var "peer")) (.var "n"))
  , .claim "finished" [name "A", .var "peer"] ] }

def bob (v : Variant) : Role := { name := "Bob", steps :=
  [ .recv (.enc (.pk (name "B")) (.tuple [.var "m", .var "x"]))
  , .send (.var "x") (.enc (.pk (.var "x")) (match v with
      | .original => .tuple [.var "m", .lit nb]
      | .lowe => .tuple [.var "m", .lit nb, name "B"]))
  , .recv (.enc (.pk (name "B")) (.lit nb))
  , .claim "commit" [name "B", .var "x"] ] }

def declared (v : Variant) : Protocol :=
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", .atom "I", .pk "A", .pk "B", .pk "I", .sk
     "I", ni]
  , domain := [na, nb, ni]
  , domains := [("x", [.atom "A", .atom "B", .atom "I"])]
  , roles := [alice v, bob v]
  , flags :=
    [ ⟨"SecrecyBreach", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.knows
     nb⟩
    , ⟨"Impersonation", fun w => w.claimed "commit" [.atom "B", .atom "A"] && !w.
    claimed "running" [.atom "A", .atom "B"]⟩
    , ⟨"HonestRun", fun w => w.claimed "commit" [.atom "B", .atom "A"] && w.claimed "
    finished" [.atom "A", .atom "B"]⟩ ] }
```

`LeanFM.Session.Attack` generates the model. Every message an honest role sends goes to the network, which is Ivo; every message it receives is one Ivo can build in the expected shape. That

is the Dolev–Yao channel, declared once rather than written into each role. A state is each role's position and bindings, Ivo's knowledge (kept taken apart, so `knows` is an ordinary label), and the claims made so far.

Requirements become flags that events trip. `SecrecyBreach` trips when Bob has committed to Alice and Ivo can derive Bob's nonce; `Impersonation` when Bob has committed to Alice although Alice never claimed to be running with Bob. `check_ctl` decides `never` of each, and that an honest run can complete. The last row asks secrecy with `knows` and `claimed` atoms instead of a flag and gets the same verdicts. The same declaration, as JSON, is what the MCP tool `check_protocol` takes:

LeanFM `check_ctl`	1978 (242 states)	Lowe (222 states)
Ivo never learns the nonce Bob shares with Alice	disproved	proved
Bob never commits to Alice unless Alice ran with Bob	disproved	proved
Alice and Bob can complete an honest run	proved	proved
the same secrecy, as `never (claimed commit(B, A) and knows(Ivo, nb))`	disproved	proved

For the 1978 protocol the counterexample is the attack Lowe found in 1995, in the checker's own words:

```
1. Alice chooses peer = I
2. Alice claims running(A, I)
3. Alice -> I: {na, A}pk(I)
4. Bob <- {na, A}pk(B)
5. Bob -> A: {na, nb}pk(A)
6. Alice <- {na, nb}pk(A)
7. Alice -> I: {nb}pk(I)
8. Bob <- {nb}pk(B)
9. Bob claims commit(B, A)
10. SecrecyBreach
```

Alice talks to Ivo, as she meant to. Ivo re-encrypts her first message for Bob, who answers "Alice" with his nonce under Alice's key. Ivo cannot read that answer, but he does not need to: he forwards it, and Alice, who believes it is Ivo's reply, decrypts it and sends Bob's nonce back to Ivo under Ivo's key. Ivo now knows the nonce, and returns it to Bob, who finishes believing he talked to Alice. Lowe's fix adds Bob's name to the second message, and Alice checks it is the peer she chose. With it both properties hold on every reachable state.

The model is bounded, and the bounds are where attacks could still hide: each session runs once, and a variable binds a name or nonce from a declared domain, never a whole ciphertext, so Ivo forwarding a sealed blob he cannot read is out of reach. Lowe's attack needs one session of each role; an attack needing a second run of Bob needs a second session, which a declaration can ask for.

17.8.1 Two sessions: reflection

Some attacks play one session of a role against another. The standard example is a challenge–response with one key, `kAB`, shared by Alice and Bob. Alice challenges with a fresh nonce; Bob answers it under the key and adds his own challenge; Alice answers his. The flag `named` adds a fix:

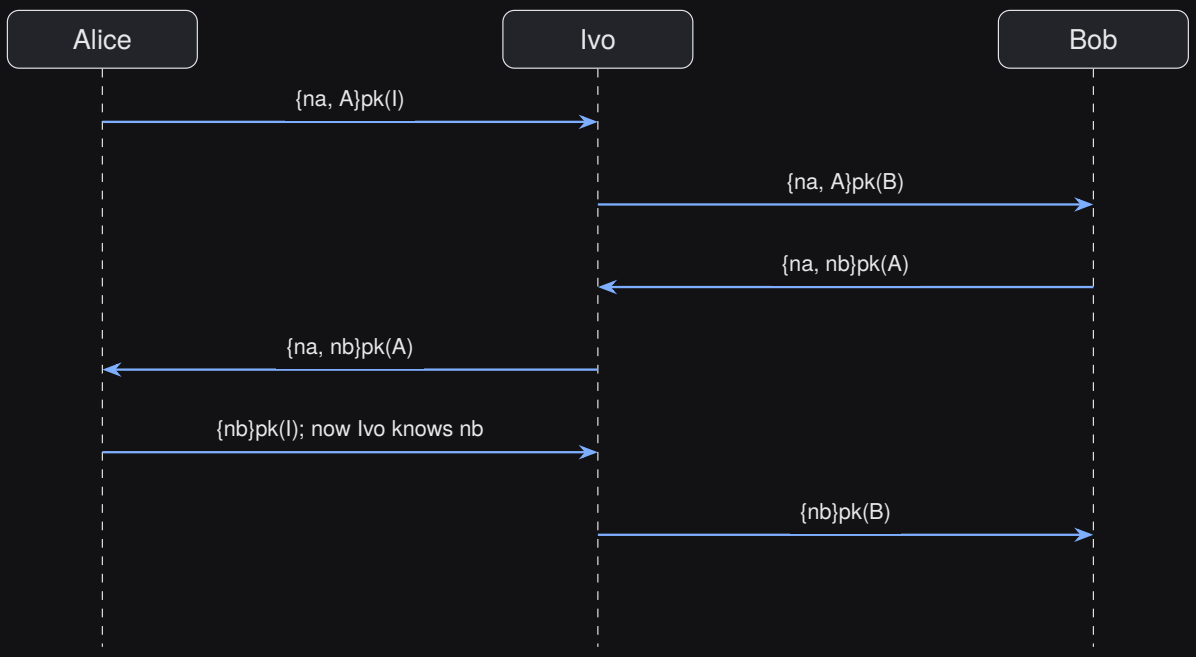


Figure 17.1: Lowe's attack as it crosses the wire: the checker's walk replayed through the generated model, which draws every message through Ivo, because Ivo is the network. The label marks where his knowledge first contains Bob's nonce.

```
/-- With ‘named‘, each answer carries the name of the party who made it. -/
def answer (named : Bool) (x : String) (who : String) : Pat :=
  .enc kAB (if named then .pair (.var x) (name who) else .var x)

def challenger (named : Bool := false) : Role := { name := "Alice", steps :=
  [ .fresh "na"
  , .claim "running" [name "A", name "B"]
  , .send (name "B") (.var "na")
  , .recv (.pair (answer named "na" "B") (.var "m"))
  , .send (name "B") (answer named "m" "A")
  , .claim "finished" [name "A", name "B"] ] }

def responder (named : Bool := false) : Role := { name := "Bob", steps :=
  [ .recv (.var "c")
  , .fresh "nb"
  , .send (name "A") (.pair (answer named "c" "B") (.var "nb"))
  , .recv (answer named "nb" "A")
  , .claim "commit" [name "B", name "A"] ] }

def reflection (bobSessions : Nat) (named : Bool := false) : Protocol :=
  let roles := (challenger named).sessions 1 ++ (responder named).sessions
    bobSessions
  { attacker := "Ivo"
  , attackerKnows := [.atom "A", .atom "B", ni]
  , domain := coverFresh roles [na, nb, ni]
```

```
  , roles
  , flags := [] }
```

A `fresh` step makes a value no other session has and the attacker does not know: `nb` in Bob's first session, `nb#2` in his second, `Bob#2`. The requirement is agreement: Bob never commits to Alice unless Alice started a run with him.

LeanFM `check_ctl`	one Bob (33)	two Bobs (527)	two Bobs, names inside (433)
Bob never commits to Alice unless Alice ran with Bob	proved	disproved	proved
Alice and Bob can complete an honest run	proved	proved	proved

With one session of Bob the property holds: the only party that encrypts under `kAB` on demand is Bob, and he does it before he has made the nonce that would need answering. With two sessions Ivo needs no key at all:

```
1. Bob <- ni
2. Bob makes nb
3. Bob -> A: {ni}kAB, nb
4. Bob#2 <- nb
5. Bob#2 makes nb#2
6. Bob#2 -> A: {nb}kAB, nb#2
7. Bob <- {nb}kAB
8. Bob claims commit(B, A)
```



Figure 17.2: Reflection: Bob's own second session answers the challenge of his first. Alice never appears; the label marks where Ivo first holds the answer he could not compute.

Ivo opens a session with Bob as Alice and gets Bob's challenge `nb`, then offers `nb` as the challenge in a second session. Bob's second session encrypts it, as it should, and Ivo hands that answer to the

first, which commits to Alice. Nothing was decrypted; the key was used as an oracle. The usual fixes put the sender's name or the direction inside the encryption, or use a key per direction. With `named`, Bob's second session produces `{nb, B}kAB` while his first waits for `{nb, A}kAB`, which only Alice makes; the table's third column checks the fix with the same two sessions.

17.8.2 A defender on the wire

A real path often has a defender on it too: an intrusion-prevention system, a filtering proxy, a gateway that intercepts TLS. A declaration may put one in front of named roles; every message delivered to them passes through it, and it drops what matches a drop pattern, rewrites what matches a rewrite rule, and forwards the rest. It matches only what it can see, which is `knows` from the defender's side: it looks inside a ciphertext only when it can build the opening key, and a blob it cannot open matches nothing, since real encryption is randomised. Here Bob accepts whatever reaches him, and Guard drops `bad`:

```
def guarded (bobReceives : Json) (defender : Option Json) (formula : Json) : Json :=
  let base := json% {
    roles: [{name: "Bob", steps: [
      {recv: $(bobReceives)},
      {claim: {name: "accept", args: [{atom: "B"}, {var: "x"}]}}]}],
    attacker: {name: "Ivo", knows: [{atom: "B"}, {atom: "good"}, {atom: "bad"}, {pk:
    "B"}]},
    domain: [{atom: "good"}, {atom: "bad"}],
    formula: $(formula) }
  match defender with
  | some d => base.setObjVal! "defender" d
  | none => base

def plain := json% {var: "x"}
def sealedForBob := json% {enc: {key: {pk: "B"}, body: {var: "x"}}}
def filter := json% {name: "Guard", protects: ["Bob"], drop: [{atom: "bad"}]}
def intercepting := json% {name: "Guard", protects: ["Bob"], knows: [{sk: "B"}], drop
    : [{atom: "bad"}]}
```

Bob receives, through	Bob never accepts bad	good still arrives
no defender, plaintext	disproved	proved
filter, plaintext	proved	proved
filter, sealed for Bob	disproved	proved
intercepting filter (holds sk(B)), sealed for Bob	proved	proved

Sealed for Bob, `bad` goes straight past the filter: the counterexample shows Guard itself delivering `{bad}pk(B)`. Give Guard Bob's private key, as an intercepting gateway has, and it drops it again. The table is the argument for and against interception in four rows.

17.8.3 Reassembly

Signature matching has a subtler gap. A defender scanning a TCP stream for `EVIL` must reassemble it as the host behind it will, and the attacker controls segmentation: any byte range, in any order,

repeated with different bytes. Where segments overlap, one implementation keeps the first bytes and another the last, and operating systems have differed in exactly this. `check_reassembly` models each stream position as empty, the signature's byte, or another byte (written `.`), applies every segment to the defender's and the host's copies with their own policies, and resets the stream when a blocking defender matches. The search covers every segmentation:

defender keeps	host keeps	never delivered	evasion
first	last	disproved	`.` at 0, then `EVIL` at 0
last	first	disproved	`E` at 0, then `.VIL` at 0
first	first	proved	—
last	last	proved	—

When the policies differ, two segments are enough: a first-wins defender fed a decoy byte at offset 0 and then the whole signature keeps the decoy and sees `.VIL`, while a last-wins host sees `EVIL`. When they agree, no segmentation separates the copies. The same disagreement makes a host fingerprintable: send overlapping segments and see which bytes it acts on. The fix, as later intrusion-prevention systems did, is to normalize: the defender forwards the bytes it scanned, leaving the host no overlap to resolve differently.

17.9 What these exercises demand from the library

The three examples produce a concrete checklist:

1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables and prefix-dependent guards, including equalities and counters;
4. a transport-neutral term-to-byte relation with encryption and hashing;
5. channels, so that each actor's observations are declared;
6. explicit initial knowledge, compromise and freshness assumptions;
7. attacker closure under construction, decryption and hashing (done: `LeanFM.Session.Knowledge`, decision proved sound), and an attacker generated from role declarations (done: `LeanFM.Session.Attack`);
8. past operators for agreement claims such as "accepted only if previously formed";
9. probabilities on the E quantifier, from overlays, reported next to CTL verdicts;
10. deadlines on F and U, with timeout branches wherever A must meet them;
11. relational theorem schemas for indistinguishability; and
12. counterexamples drawn both as a global interaction diagram and as each actor's local projection.

None of this is security decoration. It tests whether separately written, context-sensitive role grammars stay modular while still expressing the information flow that real behavioural claims depend on.

Part III

Machine Reference

Chapter 18

The LeanFM Web-Server Protocol

This is the protocol reference for the LeanFM web server, written mainly for an implementation agent. The behavior is defined by `LeanFM/ChatUI/Requirements.lean`, the bytes by `LeanFM/ChatUI/Requirements.proto`, and the transports by `LeanFM/ChatUI/Implementation.lean`; the tables and grammars below are generated from them. The *accepted protocol* is the complete grammar and intended HTTP mapping; the *running adapter* is the subset `LeanFM/ChatUI/Server.lean` dispatches today.

18.1 One exchange across four representations

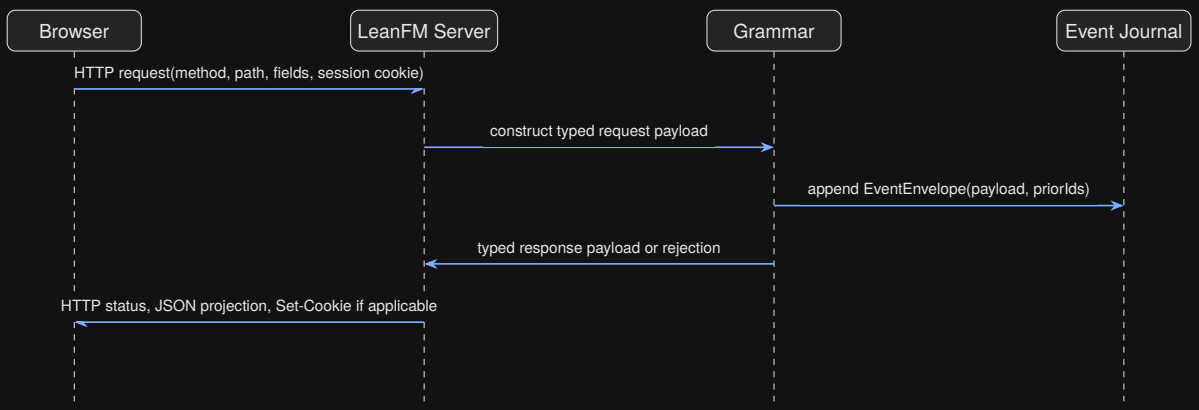


Figure 18.1: HTTP is an adapter around the typed LeanFM conversation. Source, reducer, question, and unit are recorded in `generated/web-protocol-provenance.txt`.

HTTP is decoded into a grammar-permitted protobuf value, recorded as an `EventEnvelope`, and the response or rejection is projected back. Provider messages stay in-process. Protobuf fixes values and tags; HTTP chooses the browser representation (today, form-encoded POST bodies and JSON responses, with a secure session cookie), and the journal stores length-delimited binary envelopes.

18.1.1 Conservative default HTTP projection

Without an explicit binding in `Implementation.lean`, LeanFM generates an HTTP projection and its OpenAPI description: an initiating browser-to-server message is the request, its correlated server-to-browser alternatives are the responses, and their protobuf schemas the operation schemas. The default is conservative. Only a field the implementation classifies as a bounded, URL-safe scalar may go in a path or query; bytes, repeated or nested fields, secrets and unbounded strings go in a `POST` body, encoded with the protobuf JSON mapping as `application/json`, so a generator cannot leak credentials into URLs. `GET /openapi.yaml` publishes the result: method, path, content type, request schema, every permitted response and status, authentication, correlation fields and requirement identifier. The protobuf and grammar remain the definitions.

18.2 The conversion, field by field

For a request binding `httpRequest(method, pathTemplate)`, conversion is defined in this order:

1. Match the HTTP method exactly and split the path into decoded segments.
2. For each `{field}` in the template, take the protobuf field of that name, encode it as one path segment, and reject slashes, traversal, or a value that escapes the authenticated project root.
3. For the currently implemented POST subset, encode remaining browser input fields as UTF-8 form values. A protobuf `bytes` field is opaque bytes at the protocol boundary; a form adapter must use an explicitly declared textual encoding and must never place credentials in a URL, trace, diagnostic, or returned object.
4. Recover the authenticated user from the `leanfm_session` HttpOnly, SameSite cookie. The cookie is transport authorization, not a field silently added to every protobuf message. It authorizes model calls and use of that user's provider keys; it does not decide which project may be read, because projects are shared.
5. Validate required fields and task `where_` constraints. On success, assign the envelope metadata and the oneof payload. On failure, emit the named rejection alternative when the grammar has one; otherwise return an HTTP error without pretending a successful protocol event occurred.

A response binding `httpResponse(RequestName)` reverses this over the request's finite response set: scalars become JSON values, repeated fields arrays, nested messages objects, `bytes` an explicit download or Base64 field, and secrets are redacted. Statuses follow convention (200, 201, 400, 401/403, 404, 409, 502), but the named response message, not the status, decides the grammar transition.

Several exchanges may be in flight, so every response envelope sets `cause_id` to its request's event identifier in the same session and task, and the adapter follows it to answer on the request's connection, never by adjacency, arrival order or a global "current request". A projection is valid only for a causally closed exchange: each request named by exactly one terminal response, a failure being a response too. An orphan request, a causeless or cross-session response, or two terminal responses fail validation; interim or streaming responses are modeled explicitly, with one terminal.

18.3 The running adapter

The implemented subset (static assets, diagnostics, downloads, MCP and health checks are outside the task grammar):

POST `/api/accounts`
Request: AccountCreateSubmitted. Response: AccountCreated or AccountCreateRejected. Encoding: application/x-www-form-urlencoded.

POST `/api/login`
Request: LoginSubmitted. Response: LoginAccepted or LoginRejected. Encoding: application/x-www-form-urlencoded.

GET `/api/me`
Request: authenticated session. Response: user identity JSON. Encoding: cookie only.

GET `/api/projects`
Request: ProjectListRequested. Response: ProjectListReturned. Encoding: cookie plus path.

POST `/api/projects`
Request: ProjectCreateRequested. Response: ProjectCreated. Encoding: application/x-www-form-urlencoded.

GET `/api/projects/{project_id}/conversation`
Request: project identifier. Response: conversation JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/tasks`
Request: project identifier. Response: task model JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/artifacts`
Request: ArtifactListRequested. Response: ArtifactListReturned. Encoding: path field plus cookie.

POST `/api/projects/{project_id}/chat`
Request: ChatPromptSubmitted. Response: ChatResponseRendered or ChatTurnFailed. Encoding: application/x-www-form-urlencoded.

Project routes are shared and need no login; an unknown project is 404. Model calls, key management and application launch need a session, or return 401. The server binds to loopback; exposing it needs an authenticating proxy.

18.4 The accepted HTTP projection

Rows missing from the running table are still to be implemented.

POST `/api/login`
Request: `LoginSubmitted`. Response: `LoginAccepted`, `LoginRejected`.

POST `/api/accounts`
Request: `AccountCreateSubmitted`. Response: `AccountCreated`, `AccountCreateRejected`.

GET `/api/projects/{project_id}/tasks/{task_id}`
Request: `TaskModelRequested`. Response: `TaskModelReturned`.

GET `/api/projects/{project_id}/tasks/{task_id}/sessions/{session_id}/events`
Request: `MessageSequenceRequested`. Response: `MessageSequenceReturned`.

GET `/api/projects/{project_id}/events/{event_id}`
Request: `ProtobufObjectRequested`. Response: `ProtobufObjectReturned`.

POST `/api/projects/{project_id}/research`
Request: `WebResearchRequested`. Response: `WebResearchReturned`, `WebResearchFailed`.

POST `/api/projects/{project_id}/markerboard/assist`
Request: `MarkerboardAssistRequested`. Response: `MarkerboardSyntaxHelpReturned`, `MarkerboardPatchPro`

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/accept`
Request: `MarkerboardPatchAccepted`. Response: `MarkerboardDraftUpdated`.

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/reject`
Request: `MarkerboardPatchRejected`. Response: `HTTP204(nopayload)`.

GET `/api/projects`
Request: `ProjectListRequested`. Response: `ProjectListReturned`.

POST `/api/projects`
Request: `ProjectCreateRequested`. Response: `ProjectCreated`.

GET `/api/projects/{project_id}`
Request: `ProjectOpenRequested`. Response: `ProjectOpened`, `ProblemInquiryPresented`.

DELETE `/api/projects/{project_id}`
Request: `ProjectDeleteRequested`. Response: `ProjectDeleted`.

POST `/api/projects/{project_id}/specification/problem`
Request: `ProblemStatementSubmitted`. Response: `AmbiguityQuestionPresented`, `RequirementsReviewPrese`
`ImplementationReviewPresented`.

POST `/api/projects/{project_id}/specification/ambiguities/{ambiguity_id}`
Request: `ClarificationSubmitted`. Response: `ProjectBookRevised`.

POST `/api/credentials`
Request: `ProviderCredentialSubmitted`. Response: `ProviderCredentialAccepted`, `ProviderCredentialRej`

POST `/api/projects/{project_id}/conversations/{conversation_id}/messages`
Request: `ChatPromptSubmitted`. Response: `ChatResponseRendered`, `ChatTurnFailed`.

POST `/api/projects/{project_id}/markerboard/mermaid`
Request: `MermaidDraftSaved`. Response: `MermaidPreviewRendered`.

POST `/api/projects/{project_id}/markerboard/media`
Request: `MediaUploaded`. Response: `MediaRendered`.

POST `/api/projects/{project_id}/artifacts/{artifact_id}/promotions`
Request: `ArtifactPromoted`. Response: `CandidateRequirementUpdated`.

GET `/api/projects/{project_id}/artifacts`
Request: `ArtifactListRequested`. Response: `ArtifactListReturned`.

GET `/api/projects/{project_id}/artifacts/{artifact_id}/revisions/{revision}`
Request: `ArtifactOpenRequested`. Response: `ArtifactOpened`.

Server-to-provider messages use the bounded in-process channels `provider.requests` and `provider.results`, keeping their message and request identities. Because key, session, user, project, conversation and request stay request-local rather than in a process-wide variable, two users can use different credentials at once.

18.5 Accepted message endpoints

Every event is a length-delimited `EventEnvelope`; payload tags 20 and up identify the message. Senders and receivers:

`LoginSubmitted`
UserBrowser sends to LeanFMServer.

`LoginAccepted`
LeanFMServer sends to UserBrowser.

`LoginRejected`
LeanFMServer sends to UserBrowser.

`AccountCreateSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderKeyCheckRequested`
LeanFMServer sends to ModelProvider.

`ProviderKeyVerified`
ModelProvider sends to LeanFMServer.

`ProviderKeyRejected`
ModelProvider sends to LeanFMServer.

`AccountCreated`
LeanFMServer sends to UserBrowser.

`AccountCreateRejected`
LeanFMServer sends to UserBrowser.

`TaskModelRequested`
UserBrowser sends to LeanFMServer.

`TaskModelReturned`
LeanFMServer sends to UserBrowser.

`MessageSequenceRequested`
UserBrowser sends to LeanFMServer.

`MessageSequenceReturned`
LeanFMServer sends to UserBrowser.

`ProtobufObjectRequested`
UserBrowser sends to LeanFMServer.

`ProtobufObjectReturned`
LeanFMServer sends to UserBrowser.

`WebResearchRequested`
UserBrowser sends to LeanFMServer.

`WebResearchDispatched`
LeanFMServer sends to ModelProvider.

`WebSearchPerformed`
ModelProvider sends to LeanFMServer.

`WebPageOpened`
ModelProvider sends to LeanFMServer.

`WebResearchReturned`
LeanFMServer sends to UserBrowser.

`WebResearchFailed`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistRequested`
UserBrowser sends to LeanFMServer.

`MarkerboardSyntaxHelpReturned`
LeanFMServer sends to UserBrowser.

`MarkerboardAssistDispatched`
LeanFMServer sends to ModelProvider.

`MarkerboardAssistReturned`
ModelProvider sends to LeanFMServer.

`MarkerboardPatchProposed`
LeanFMServer sends to UserBrowser.

`MarkerboardPatchAccepted`
UserBrowser sends to LeanFMServer.

`MarkerboardPatchRejected`
UserBrowser sends to LeanFMServer.

`MarkerboardDraftUpdated`
LeanFMServer sends to UserBrowser.

`ProjectListRequested`
UserBrowser sends to LeanFMServer.

`ProjectListReturned`
LeanFMServer sends to UserBrowser.

`ProjectCreateRequested`
UserBrowser sends to LeanFMServer.

`ProjectCreated`
LeanFMServer sends to UserBrowser.

`ProjectOpenRequested`
UserBrowser sends to LeanFMServer.

`ProjectOpened`
LeanFMServer sends to UserBrowser.

`ProjectDeleteRequested`
UserBrowser sends to LeanFMServer.

`ProjectDeleted`
LeanFMServer sends to UserBrowser.

`ProblemInquiryPresented`
LeanFMServer sends to UserBrowser.

`ProblemStatementSubmitted`
UserBrowser sends to LeanFMServer.

`AmbiguityQuestionPresented`
LeanFMServer sends to UserBrowser.

`ClarificationSubmitted`
UserBrowser sends to LeanFMServer.

`ProjectBookRevised`
LeanFMServer sends to UserBrowser.

`RequirementsReviewPresented`
LeanFMServer sends to UserBrowser.

`ImplementationReviewPresented`
LeanFMServer sends to UserBrowser.

`ProviderCredentialSubmitted`
UserBrowser sends to LeanFMServer.

`ProviderCredentialAccepted`
LeanFMServer sends to UserBrowser.

`ProviderCredentialRejected`
LeanFMServer sends to UserBrowser.

`ChatPromptSubmitted`
UserBrowser sends to LeanFMServer.

`ModelRequestIssued`
LeanFMServer sends to ModelProvider.

`ModelResponseReturned`
ModelProvider sends to LeanFMServer.

`ChatResponseRendered`
LeanFMServer sends to UserBrowser.

`ChatTurnFailed`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointRequested`
UserBrowser sends to LeanFMServer.

`ConversationCheckpointBlocked`
LeanFMServer sends to UserBrowser.

`ConversationCheckpointCommitted`
LeanFMServer sends to UserBrowser.

`ConversationHistoryCleared`
LeanFMServer sends to UserBrowser.

`MermaidDraftSaved`
UserBrowser sends to LeanFMServer.

`MermaidPreviewRendered`
LeanFMServer sends to UserBrowser.

`MediaUploaded`
UserBrowser sends to LeanFMServer.

`MediaRendered`
LeanFMServer sends to UserBrowser.

`ArtifactPromoted`
UserBrowser sends to LeanFMServer.

`CandidateRequirementUpdated`
LeanFMServer sends to UserBrowser.

`ArtifactListRequested`
UserBrowser sends to LeanFMServer.

`ArtifactListReturned`
LeanFMServer sends to UserBrowser.

`ArtifactOpenRequested`
UserBrowser sends to LeanFMServer.

`ArtifactOpened`
LeanFMServer sends to UserBrowser.

18.5.1 Canonical byte schema

This schema is normative: a reconstruction reproduces its tags, cardinalities, enum values and names exactly.

```
syntax = "proto3";

package leanfm.chatui;

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 time_at_ms = 5;
  string src = 6;
  string dst = 7;
  // Direct causal parent for any event, including nonterminal and intermediate
  // events. Empty for a causal root. Unlike prior_ids, this does not constrain
  // otherwise legal concurrent interleavings. Renderers may use the causal
  // subtree as nested interaction boundaries.
  optional string cause_id = 8;

  // Former approval messages; review and acceptance now happen in git.
  reserved 81, 83, 84;

  oneof payload {
    LoginSubmitted login_submitted = 20;
    LoginAccepted login_accepted = 21;
    LoginRejected login_rejected = 22;
    ProjectListRequested project_list_requested = 23;
    ProjectListReturned project_list_returned = 24;
    ProjectCreateRequested project_create_requested = 25;
    ProjectCreated project_created = 26;
    ProjectOpenRequested project_open_requested = 27;
    ProjectOpened project_opened = 28;
    ProjectDeleteRequested project_delete_requested = 29;
    ProjectDeleted project_deleted = 30;
    ProviderCredentialSubmitted provider_credential_submitted = 31;
    ProviderCredentialAccepted provider_credential_accepted = 32;
    ProviderCredentialRejected provider_credential_rejected = 33;
    ChatPromptSubmitted chat_prompt_submitted = 34;
    ModelRequestIssued model_request_issued = 35;
    ModelResponseReturned model_response_returned = 36;
    ChatResponseRendered chat_response_rendered = 37;
    ChatTurnFailed chat_turn_failed = 38;
    MermaidDraftSaved mermaid_draft_saved = 39;
    MermaidPreviewRendered mermaid_preview_rendered = 40;
    MediaUploaded media_uploaded = 41;
    MediaRendered media_rendered = 42;
    ArtifactPromoted artifact_promoted = 43;
    CandidateRequirementUpdated candidate_requirement_updated = 44;
    ArtifactListRequested artifact_list_requested = 45;
    ArtifactListReturned artifact_list_returned = 46;
    ArtifactOpenRequested artifact_open_requested = 47;
    ArtifactOpened artifact_opened = 48;
    AccountCreateSubmitted account_create_submitted = 49;
    ProviderKeyCheckRequested provider_key_check_requested = 50;
    ProviderKeyVerified provider_key_verified = 51;
    ProviderKeyRejected provider_key_rejected = 52;
    AccountCreated account_created = 53;
    AccountCreateRejected account_create_rejected = 54;
    TaskModelRequested task_model_requested = 55;
    TaskModelReturned task_model_returned = 56;
    MessageSequenceRequested message_sequence_requested = 57;
    MessageSequenceReturned message_sequence_returned = 58;
    ProtobufObjectRequested protobuf_object_requested = 59;
    ProtobufObjectReturned protobuf_object_returned = 60;
    WebResearchRequested web_research_requested = 61;
    WebSearchPerformed web_search_performed = 62;
    WebPageOpened web_page_opened = 63;
    WebResearchReturned web_research_returned = 64;
    WebResearchFailed web_research_failed = 65;
    WebResearchDispatched web_research_dispatched = 66;
    MarkerboardAssistRequested markerboard_assist_requested = 67;
    MarkerboardSyntaxHelpReturned markerboard_syntax_help_returned = 68;
    MarkerboardAssistDispatched markerboard_assist_dispatched = 69;
    MarkerboardAssistReturned markerboard_assist_returned = 70;
    MarkerboardPatchProposed markerboard_patch_proposed = 71;
    MarkerboardPatchAccepted markerboard_patch_accepted = 72;
```

```
    MarkerboardPatchRejected markerboard_patch_rejected = 73;
    MarkerboardDraftUpdated markerboard_draft_updated = 74;
    ProblemInquiryPresented problem_inquiry_presented = 75;
    ProblemStatementSubmitted problem_statement_submitted = 76;
    AmbiguityQuestionPresented ambiguity_question_presented = 77;
    ClarificationSubmitted clarification_submitted = 78;
    ProjectBookRevised project_book_revised = 79;
    RequirementsReviewPresented requirements_review_presented = 80;
    ImplementationReviewPresented implementation_review_presented = 82;
    ConversationCheckpointRequested conversation_checkpoint_requested = 85;
    ConversationCheckpointBlocked conversation_checkpoint_blocked = 86;
    ConversationCheckpointCommitted conversation_checkpoint_committed = 87;
    ConversationHistoryCleared conversation_history_cleared = 88;
  }
}

enum ContentKind {
  CONTENT_KIND_UNSPECIFIED = 0;
  MARKDOWN = 1;
  LATEX = 2;
  MERMAID = 3;
  GRAPHVIZ_DOT = 4;
  IMAGE = 5;
  VIDEO = 6;
  CODE = 7;
  FILE = 8;
  LEANFM_VIEW = 9;
  URL_CITATION = 10;
}

message ChatContentPart {
  string part_id = 1;
  ContentKind kind = 2;
  string text = 3;
  string artifact_id = 4;
  uint64 artifact_revision = 5;
  string media_type = 6;
  bytes inline_content = 7;
  string alt_text = 8;
  string url = 9;
  string title = 10;
}

message LoginSubmitted {
  string username = 1;
  bytes credential = 2;
  bytes openai_api_key = 3;
  bytes anthropic_api_key = 4;
  bytes new_password = 5;
  bytes new_password_confirm = 6;
}
message LoginAccepted { string user_id = 1; string session_id = 2; }
message LoginRejected { string reason = 1; }

message ProblemInquiryPresented { string project_id = 1; string question = 2; }
message ProblemStatementSubmitted { string project_id = 1; string statement = 2; }
message AmbiguityQuestionPresented { string project_id = 1; string ambiguity_id = 2; string question = 3; repeated string affected_artifacts
       = 4; }
message ClarificationSubmitted { string project_id = 1; string ambiguity_id = 2; string answer = 3; }
message ProjectBookRevised { string project_id = 1; string book_revision = 2; repeated string decision_ids = 3; string light_pdf = 4; string
       dark_pdf = 5; }
message RequirementsReviewPresented { string project_id = 1; string requirements_revision = 2; repeated string derived_artifact_ids = 3;
      repeated string open_ambiguity_ids = 4; }
message ImplementationReviewPresented { string project_id = 1; string requirements_revision = 2; string implementation_revision = 3; string
      traceability_report = 4; }

message AccountCreateSubmitted {
  string username = 1;
  bytes login_credential = 2;
  string provider = 3;
  bytes api_key = 4;
  bytes openai_api_key = 5;
  bytes anthropic_api_key = 6;
}
message ProviderKeyCheckRequested { string provider = 1; string verification_id = 2; }
message ProviderKeyVerified { string verification_id = 1; string provider_account = 2; }
message ProviderKeyRejected { string verification_id = 1; string reason = 2; }
message AccountCreated { string user_id = 1; string credential_ref = 2; }
message AccountCreateRejected { string reason = 1; }

message TaskModelRequested { string project_id = 1; string task_id = 2; string requirements_revision = 3; }
message TaskModelReturned {
  string task_id = 1;
  string interaction_artifact_id = 2;
  string state_machine_artifact_id = 3;
  string message_sequence_artifact_id = 4;
  repeated string constraint_ids = 5;
}
message MessageSequenceRequested { string project_id = 1; string task_id = 2; string session_id = 3; }
```

```
message MessageSequenceReturned { string task_id = 1; string session_id = 2; repeated string event_ids = 3; }
message ProtobufObjectRequested { string project_id = 1; string event_id = 2; }
message ProtobufObjectReturned { string event_id = 1; string protobuf_type = 2; bytes encoded_object = 3; string redacted_json = 4; }
message WebResearchRequested { string project_id = 1; string conversation_id = 2; string request_id = 3; string question = 4; repeated
      string supplied_urls = 5; }
message WebResearchDispatched { string request_id = 1; string model = 2; string research_mode = 3; }
message WebSearchPerformed { string request_id = 1; repeated string queries = 2; }
message WebPageOpened { string request_id = 1; string url = 2; string title = 3; }
message WebResearchReturned { string request_id = 1; repeated ChatContentPart parts = 2; repeated string source_urls = 3; }
message WebResearchFailed { string request_id = 1; string reason = 2; }
message MarkerboardAssistRequested { string project_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; string mode = 7; string credential_ref = 8; }
message MarkerboardSyntaxHelpReturned { string artifact_id = 1; uint64 base_revision = 2; string help_markdown = 3; }
message MarkerboardAssistDispatched { string request_id = 1; string artifact_id = 2; uint64 base_revision = 3; string diagram_kind = 4;
      string source = 5; string instruction = 6; }
message MarkerboardAssistReturned { string request_id = 1; string proposed_source = 2; string explanation = 3; uint64 input_tokens = 4;
      uint64 output_tokens = 5; }
message MarkerboardPatchProposed { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; string explanation = 4; }
message MarkerboardPatchAccepted { string artifact_id = 1; uint64 base_revision = 2; string proposed_source = 3; }
message MarkerboardPatchRejected { string artifact_id = 1; uint64 base_revision = 2; }
message MarkerboardDraftUpdated { string artifact_id = 1; uint64 revision = 2; string source = 3; }

message ProjectListRequested { string user_id = 1; }
message ProjectListReturned { repeated string project_ids = 1; }
message ProjectCreateRequested { string display_name = 1; }
message ProjectCreated { string project_id = 1; string display_name = 2; }
message ProjectOpenRequested { string project_id = 1; }
message ProjectOpened { string project_id = 1; string current_requirements_revision = 2; }
message ProjectDeleteRequested { string project_id = 1; string confirmation = 2; }
message ProjectDeleted { string project_id = 1; }

message ProviderCredentialSubmitted {
  string provider = 1;
  bytes api_key = 2;
  string retention = 3;
}
message ProviderCredentialAccepted { string credential_ref = 1; string billing_source = 2; }
message ProviderCredentialRejected { string reason = 1; }

message ChatPromptSubmitted {
  string project_id = 1;
  string conversation_id = 2;
  string message_id = 3;
  repeated ChatContentPart parts = 4;
  repeated string attachment_ids = 5;
  string credential_ref = 6;
  string model = 7;
}
message ModelRequestIssued {
  string project_id = 1;
  string conversation_id = 2;
  string request_id = 3;
  string model = 4;
  string research_mode = 5;
}
message ModelResponseReturned {
  string request_id = 1;
  repeated ChatContentPart parts = 2;
  uint64 input_tokens = 3;
  uint64 cached_input_tokens = 4;
  uint64 output_tokens = 5;
  uint64 context_window_tokens = 6;
}
message ChatResponseRendered {
  string conversation_id = 1;
  string message_id = 2;
  repeated ChatContentPart parts = 3;
  repeated string artifact_ids = 4;
  uint64 input_tokens = 5;
  uint64 cached_input_tokens = 6;
  uint64 output_tokens = 7;
  uint64 context_window_tokens = 8;
  string usage_warning = 9;
  uint64 response_time_ms = 10;
}
message ChatTurnFailed { string request_id = 1; string reason = 2; }
message ConversationCheckpointRequested { string project_id = 1; string conversation_id = 2; string through_message_id = 3; }
message ConversationCheckpointBlocked { string project_id = 1; string conversation_id = 2; string checkpoint_id = 3; string reason_code = 4;
       repeated string blocking_artifact_ids = 5; }
message ConversationCheckpointCommitted { string project_id = 1; string conversation_id = 2; string through_message_id = 3; repeated string
      artifact_revisions = 4; string manifest_digest = 5; }
message ConversationHistoryCleared { string project_id = 1; string conversation_id = 2; string cleared_through_message_id = 3; uint64
      retained_message_count = 4; }

message MermaidDraftSaved {
  string project_id = 1;
  string artifact_id = 2;
  uint64 revision = 3;
  string diagram_kind = 4;
```

```
  string source = 5;
}
message MermaidPreviewRendered { string artifact_id = 1; uint64 revision = 2; bytes svg = 3; }
message MediaUploaded {
  string project_id = 1;
  string artifact_id = 2;
  string media_type = 3;
  bytes content = 4;
}
message MediaRendered { string artifact_id = 1; string media_type = 2; string content_url = 3; }
message ArtifactPromoted { string artifact_id = 1; uint64 revision = 2; string target_status = 3; }
message CandidateRequirementUpdated { string requirements_revision = 1; repeated string source_artifact_ids = 2; }

message ArtifactListRequested {
  string project_id = 1;
  string kind_filter = 2;
  string scenario_filter = 3;
  string status_filter = 4;
}
message ArtifactSummary {
  string artifact_id = 1;
  uint64 current_revision = 2;
  uint64 accepted_revision = 3;
  string status = 4;
  string validation = 5;
  string media_type = 6;
  string title = 7;
  string scenario_id = 8;
  string diagram_kind = 9;
  string updated_by = 10;
  uint64 updated_at_ms = 11;
}
message ArtifactListReturned { repeated ArtifactSummary artifacts = 1; }
message ArtifactOpenRequested { string artifact_id = 1; uint64 revision = 2; }
message ArtifactOpened {
  string artifact_id = 1;
  uint64 revision = 2;
  string status = 3;
  string media_type = 4;
  bytes content = 5;
  string provenance = 6;
}
```

`prior_ids` are immutable predecessor links (any successor index is derived); `session_id` separates conversations, `task_id` selects the grammar, `src`/`dst` must match the schema, and `cause_id` is the acyclic direct-cause relation of Chapter 2.

18.6 Complete LeanFM task grammar

The whole ChatUI grammar, rendered from its typed values. `A ! B : Message` reads “A sends Message to B”; `then` sequences; `choose`/`or` alternates; `optional` allows zero or one; `fork`/`join all` waits for every branch; `race`/`join first` takes the first and cancels the rest (unlike the threshold join of Chapter 5). A message the current residual does not admit is illegal even if its bytes decode.

18.7 Authenticate an individual user

Task: `authenticate_user`. Entry: `signed_out`; terminals: `authenticated, rejected`.

```
UserBrowser ! LeanFMServer : LoginSubmitted
then
choose
  LeanFMServer ! UserBrowser : LoginAccepted
or
  LeanFMServer ! UserBrowser : LoginRejected
```

18.8 Create an account using a user-supplied provider API key

Task: `create_account`. Entry: `registration`; terminals: `created, rejected`.

```
UserBrowser ! LeanFMServer : AccountCreateSubmitted
then
LeanFMServer ! ModelProvider : ProviderKeyCheckRequested
then
choose
  ModelProvider ! LeanFMServer : ProviderKeyVerified
  then
  LeanFMServer ! UserBrowser : AccountCreated
or
  ModelProvider ! LeanFMServer : ProviderKeyRejected
  then
  LeanFMServer ! UserBrowser : AccountCreateRejected
```

18.9 List, create, open, and delete owned projects

Task: `manage_projects`. Entry: `ready`; terminals: `listed, created, opened, deleted`.

```
choose
  UserBrowser ! LeanFMServer : ProjectListRequested
  then
  LeanFMServer ! UserBrowser : ProjectListReturned
or
  UserBrowser ! LeanFMServer : ProjectCreateRequested
  then
  LeanFMServer ! UserBrowser : ProjectCreated
or
  UserBrowser ! LeanFMServer : ProjectOpenRequested
  then
  LeanFMServer ! UserBrowser : ProjectOpened
or
  UserBrowser ! LeanFMServer : ProjectDeleteRequested
  then
  LeanFMServer ! UserBrowser : ProjectDeleted
```

18.10 Turn a fresh project conversation into checked requirements and implementation

Task: `develop_project_specification`. Entry: `project_loaded`; terminals: `ready_to_implement`.

```
LeanFMServer ! UserBrowser : ProblemInquiryPresented
then
UserBrowser ! LeanFMServer : ProblemStatementSubmitted
```

```
then
repeat
  choose
    empty
  or
    LeanFMServer ! UserBrowser : ProjectBookRevised
  then
  LeanFMServer ! UserBrowser : AmbiguityQuestionPresented
  then
  UserBrowser ! LeanFMServer : ClarificationSubmitted
then
choose
  empty
or
  LeanFMServer ! UserBrowser : ProjectBookRevised
then
LeanFMServer ! UserBrowser : RequirementsReviewPresented
then
LeanFMServer ! UserBrowser : ImplementationReviewPresented
```

18.11 Select user-funded or platform-funded model access

Task: `configure_provider`. Entry: `unconfigured`; terminals: `configured, rejected`.

```
UserBrowser ! LeanFMServer : ProviderCredentialSubmitted
then
choose
  LeanFMServer ! UserBrowser : ProviderCredentialAccepted
or
  LeanFMServer ! UserBrowser : ProviderCredentialRejected
```

18.12 Conduct one durable project-scoped requirements turn

Task: `argue_requirements`. Entry: `ready`; terminals: `rendered, failed`.

```
UserBrowser ! LeanFMServer : ChatPromptSubmitted
then
LeanFMServer ! ModelProvider : ModelRequestIssued
then
choose
  ModelProvider ! LeanFMServer : ModelResponseReturned
  then
  LeanFMServer ! UserBrowser : ChatResponseRendered
or
  LeanFMServer ! UserBrowser : ChatTurnFailed
```

18.13 Compact a project conversation into durable specifications before clearing it

Task: `checkpoint_conversation`. Entry: `retained`; terminals: `cleared, blocked`.

```
UserBrowser ! LeanFMServer : ConversationCheckpointRequested
then
choose
  LeanFMServer ! UserBrowser : ConversationCheckpointBlocked
or
  LeanFMServer ! UserBrowser : ConversationCheckpointCommitted
  then
  LeanFMServer ! UserBrowser : ConversationHistoryCleared
```

18.14 Sketch and promote multimodal requirement artifacts

Task: `markerboard`. Entry: `editing`; terminals: `candidate`.

```
choose
  UserBrowser ! LeanFMServer : MermaidDraftSaved
  then
  LeanFMServer ! UserBrowser : MermaidPreviewRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
or
  UserBrowser ! LeanFMServer : MediaUploaded
  then
  LeanFMServer ! UserBrowser : MediaRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
```

18.15 Inspect current, accepted, superseded, and stale artifacts

Task: `inspect_artifacts`. Entry: `closed`; terminals: `listed, opened`.

```
UserBrowser ! LeanFMServer : ArtifactListRequested
then
LeanFMServer ! UserBrowser : ArtifactListReturned
then
choose
  empty
or
```

```
  UserBrowser ! LeanFMServer : ArtifactOpenRequested
  then
  LeanFMServer ! UserBrowser : ArtifactOpened
```

18.16 Inspect one task's linked interaction, state machine, constraints, and protobuf events

Task: `inspect_task_model`. Entry: `closed`; terminals: `linked, sequence_visible, object_visible`.

```
UserBrowser ! LeanFMServer : TaskModelRequested
then
LeanFMServer ! UserBrowser : TaskModelReturned
then
choose
  empty
or
  UserBrowser ! LeanFMServer : MessageSequenceRequested
  then
  LeanFMServer ! UserBrowser : MessageSequenceReturned
  then
  choose
    empty
  or
    UserBrowser ! LeanFMServer : ProtobufObjectRequested
    then
    LeanFMServer ! UserBrowser : ProtobufObjectReturned
```

18.17 Search the public web, follow supplied HTTP URLs, and return clickable citations

Task: `research_web`. Entry: `ready`; terminals: `returned, failed`.

```
UserBrowser ! LeanFMServer : WebResearchRequested
then
LeanFMServer ! ModelProvider : WebResearchDispatched
then
choose
  ModelProvider ! LeanFMServer : WebSearchPerformed
  then
  ModelProvider ! LeanFMServer : WebPageOpened
  then
  LeanFMServer ! UserBrowser : WebResearchReturned
or
  LeanFMServer ! UserBrowser : WebResearchFailed
```

18.18 Recall diagram syntax or propose AI-generated detail for one markerboard revision

Task: `assist_markerboard`. Entry: `editing`; terminals: `helped, updated, rejected`.

```
UserBrowser ! LeanFMServer : MarkerboardAssistRequested
then
choose
  LeanFMServer ! UserBrowser : MarkerboardSyntaxHelpReturned
or
  LeanFMServer ! ModelProvider : MarkerboardAssistDispatched
  then
  ModelProvider ! LeanFMServer : MarkerboardAssistReturned
  then
  LeanFMServer ! UserBrowser : MarkerboardPatchProposed
  then
  choose
    UserBrowser ! LeanFMServer : MarkerboardPatchAccepted
    then
    LeanFMServer ! UserBrowser : MarkerboardDraftUpdated
  or
    UserBrowser ! LeanFMServer : MarkerboardPatchRejected
```

18.19 The grammar as an executable canvas

Future interface. The grammar becomes one navigable canvas: a production closes to its typed boundary or opens to its messages, states, parameters, constraints, measurements and nested productions, as presentation only. Changing a typed parameter (a capacity, population, timeout, probability) would mark every dependent stale and regenerate the affected productions, reducers, proofs, codecs and views before the revision is committed. A production that crosses HTTP attaches its binding there (method, path, protobuf types, statuses, content type, codec), with fixtures that emit and decode the exact bytes, so the canvas opens all the way down to bytes.

18.20 What is checked, and what remains

Lean elaboration checks actors, schemas, states, transitions, grammars, constraints, output provenance and mappings; the generator rejects a request with no named response and incomplete route records; and the repository checks validate the protobuf against the requirements. Not every accepted route has a handler yet, and the server does not yet journal every exchange. The accepted grammar is the contract and the running table the honest conformance statement.

Chapter 19

The LeanFM Project Bootstrap Contract

This chapter defines the minimum machine-readable context that makes a LeanFM project self-hosting, which an agent applies before turning the rest of the book into checked artifacts. The bootstrap defines the language and its tools, never a hidden copy of the product's requirements or implementation.

The intended handoff is simple: send someone this PDF's URL and ask Claude, Gemini, Codex or another capable agent to read it and make the software. New projects start from built-in templates, not an example directory; the live references and read-only MCP surface supply the authoring types and commands below. An agent must not copy LeanFM's own source or prior specifications as an answer key.

19.1 Required project inputs

A copyable project root contains the following stable inputs:

`book/main.pdf`
: Central product-design argument and maintained system-prompt context.

`book/main-dark.pdf`
: Accessible rendering of the same source.

`LeanFM/`
: Versioned generic authoring types and deterministic validators. Product actors, messages, tasks and routes do not belong in this directory.

Bootstrap programs
: Whatever prints the language references and interrogation checklist, validates the conventional specification, and derives review views from its typed value. In this repository these are the `make` targets and the LeanFM server's routes and MCP tools listed below; a portable project may ship a `bootstrap/` directory instead.

`lean-toolchain` **and** `lakefile.toml`
: The exact compiler selection and module roots needed to run the bootstrap locally.

`project.json`
: Project identity, stable artifact paths, and the classification of design versus language inputs.

All project-owned state lives below the exclusive root: inputs, conversations, revisions, source, tests, views, traces, reports, logs and caches, with temporary files and atomic promotion inside it. Outputs and symlinks that escape are rejected; imports are copied in or recorded as immutable content-addressed dependencies, and shared tools and caches outside may hold no project state. A global index is reconstructible from manifests, and deleting a root removes all its state.

The root is also the complete input to a fresh agent conversation: the agent finds the paths from the manifest and repository instructions, with no transcript to paste. A root with no problem statement gets a minimal book shell and the question “What problem are you trying to solve?”, which starts requirements discovery.

19.2 Conventional generated interface

The requirements candidate lives at `spec/Requirements.lean` (this repository keeps its own in `LeanFM/LLMGenerated/`), imports `LeanFM.Artifacts`, and defines:

```
namespace LeanFMProject.Requirements
def spec : LeanFM.RequirementSpec := ...
def all : List LeanFM.GeneratedRequirement := [.requirement spec]
end LeanFMProject.Requirements
```

The byte vocabulary is `spec/Requirements.proto` and, once the requirements check, the implementation mapping is `spec/Implementation.lean`. Capitalization matters: these are Lean module paths.

19.3 Required commands and order

The project supplies deterministic equivalents of these commands. In this repository the references and checklist come from the running server (`/tools/leanfm-language/requirements-reference`, `/tools/requirements/interrogation`, or the MCP resources of the same name), and `make check` validates and renders:

```
make reference       # print the complete requirements-language reference
make interrogation   # print questions that expose missing observations
make validate        # elaborate and run LeanFM plus protobuf validation
make render          # derive interaction and residual-state review views
make check           # validate, then render
```

An agent runs `reference` and `interrogation` before authoring, and never infers constructors from prose or invents a lookalike. The validator and the renderer both consume the exported `RequirementSpec`: elaboration alone is not validation, and a separately drawn diagram is not evidence.

19.4 Three layers, reviewed in git

Work proceeds in three layers, each checked before the next: observable requirements, their realization mapping, then source, which follows the security review of Chapter 14. Review and acceptance

are git commits. LeanFM records no approvals and makes nothing wait for one; its status shows only whether each layer passes its check and is committed.

`Requirements.lean` is the contract: its multi-actor wire grammar is the conformance oracle, under which complete decoded traffic is legal or has a counterexample (an incomplete capture establishes nothing). `Requirements.proto` fixes bytes without choosing a transport. `Implementation.lean` records the choices that realize accepted behavior (queues, transports, journals, browser mechanisms), each mapped to its requirement. An answer that changes nothing observable belongs there; one that changes messages, fields or ordering revises the requirements; an answer made moot by a later requirement is removed. The book explains what is built, the generic distribution makes the language executable, and committed artifacts keep the decisions; none substitutes for another.

19.5 Construction is not a runtime dependency

The construction agent is not part of the delivered server. The user starts LeanFM, opens it in a browser, signs in, supplies a provider key, and argues requirements through LeanFM's own chat, which talks to the provider directly. A reconstruction succeeds only if it builds a standalone executable, serves the whole browser workflow, starts without a platform key, isolates concurrent users and their credentials, and passes its conformance tests with no coding agent present.

19.6 Build, runtime, and publication dependency inventory

One developer's workstation is not an implicit build input. The normative list is `toolDependencyInventory` in `LeanFM/ChatUI/Requirements.lean`; this is its review view. "Required" means the default check, server, book or media path uses it. Versions are fixed by `ui/package-lock.json`, `scripts/chatui_requirements.t` `lean-toolchain` and the `Makefile`.

Lean and Lake — required.
Lean 4.35.0 from `lean-toolchain` compiles the DSL, validators, renderers, tests, and server. Lake resolves modules and drives native linking.

C toolchain and libc headers — required.
A compiler and linker supported by Lean 4.35 link the native executables emitted by Lean.

GNU Make, Bash, and coreutils — required.
The workflows use `timeout`, `cmp`, `cp`, `tar`, `gzip`, and `sha256sum` for bounded commands and verified tool downloads.

Node and npm — required.
Node 22.12 or newer and `npm ci` type-check and bundle the React UI. The lockfile fixes React, ReactDOM, TypeScript, Vite, KaTeX, and their transitive packages.

Mermaid renderer — required for task diagrams.
The existing Mermaid 11 renderer is served locally. `assets/vendor/mermaid.json` records its resolved version, source, MIT license and SHA-256 digest, so diagrams and screenshot scenarios do not require a CDN connection.

Python packages — required.
Python 3.12 or newer with Cryptography 41–50, Pillow 10–12, and Protobuf 5–6 protects retained keys, performs protobuf round trips and reductions, and generates animated GIFs. CairoSVG 2.7–2.x and defusedxml 0.7–0.x normalize static SVG attachments without external resources; Pillow supplies bounded raster decoding. Sandboxed media generators run the system `/usr/bin/python3`, so Pillow must be installed for it too (Debian and Ubuntu: `python3-pil`).

Protocol Buffers compiler — required.
A `protoc` supporting proto3 and Python output generates the temporary byte-round-trip binding. Target-language generators are declared separately when needed.

Graphviz — required.
`dot` with PNG output rasterizes generated DOT diagrams during `make diagrams`.

TeX Live and pdfLaTeX — required.
Install the base, recommended, extra, pictures, and recommended-font collections. The source uses `lmodern`, TeX Gyre Heros, geometry, microtype, xcolor, graphicx, tagpdf, AMS math, mathtools, booktabs, longtable, enumitem, listings, natbib, imakeidx, TikZ, pgfplots, hyperref, and cleveref.

Go — required.
Go 1.23.12, installed under `.toolchains/` by `make go-toolchain` with its archive hash checked, compiles and tests the Go projects and generated applications.

Bubblewrap — required on the Linux server.
`bwrap` with user, process, network, and mount namespaces isolates executable Lean checks, project commands, persistent applications, and project-local media generators. Ubuntu/Debian supplies it in the `bubblewrap` package; Ubuntu 24.04 also needs `kernel.apparmor_restrict_unprivileged_` `make sandbox-check` verifies the host before the full suite; missing or denied support fails explicitly and never allows unsandboxed execution.

FFmpeg — required for shareable video.
FFmpeg with `libx264` H.264 and AAC encoders converts GIF/WebM to validated MP4, adds fast-start metadata, and muxes optional narration.

curl, ripgrep, and iproute2 — required for development checks.
`curl`, `rg`, and `ss` exercise HTTP contracts, select validation evidence, and identify the local server.

Java and TLA+ tools — optional.
A Java runtime and the checksummed `tla2tools.jar` 1.7.4 run the separate `tla-check` example.

Manim and Piper — optional.
A project-local video Python environment and selected voice model render explanatory videos and synthetic narration.

Target compilers and SDKs — conditional.
The accepted `ImplementationSpec` inventories every compiler, code generator, SDK, runtime, native library, and version bound before source generation.

An agent can reconcile this list with the host, reporting each item satisfied, missing, incompatible or optional; installing software is an explicit action, never a side effect of inspection. A tool enters the product contract only through this inventory and the implementation mapping (DuckDB, for instance, is not used and so is not listed).

19.7 Local and server-hosted references

The local commands are authoritative when no server runs. A running server also exposes the language references, the interrogation checklist and read-only MCP context without login; these hold language documentation, not project or account data.

Selected Books and Further Reading

These annotations identify the role each work plays in this book; they are not reviews of the books.

Richards and Hamburger, *Logic and Language Models for Computer Science*.
A source of the author's background in formal languages and logic, not the originators of those methods [21].

Gensler, *Introduction to Logic*.
Background for symbolization, validity, proof, quantification, and modal reasoning [8].

Ryan et al., *The Modelling and Analysis of Security Protocols*.
The CSP treatment of protocol analysis behind the approach of the protocol exercises, whose examples are the author's own [22].

Honda; Honda, Vasconcelos, and Kubo; Honda, Yoshida, and Carbone.
The origin of binary and then multiparty session types, the backbone of the method [10–12].

Yoshida, Hu, Neykova, and Ng, *The Scribble Protocol Language*.
The source of the `from`/`to`/`choice` surface style used for global types in this book (LeanFM drops Scribble's `at`: the role that decides is the one that starts every branch) [25].

Deniélou and Yoshida; Brand and Zafiropulo.
The link between session types and communicating finite-state machines, and why unbounded queues make verification undecidable, which motivates LeanFM's declared capacities [4, 6].

Barwell, Scalas, Yoshida, and Zhou.
One line of work that adds failure to session types, here crash-stop roles; LeanFM instead declares failure per channel [2].

Kröger and Merz, *Temporal Logic and State Systems*.
The author's principal source for temporal-logic background [13].

Lamport, *Specifying Systems*.
A reference point for the book's comparison with TLA+ state and action specifications [15].

Gunther, *Guerrilla Capacity Planning*.
The cited background for the Universal Scalability Law and capacity planning [9].

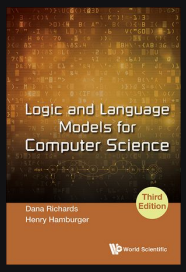
Menascé, Almeida, and Dowdy, *Performance by Design*.
Background for the queueing and capacity-planning discussion [18].

Mancosu, Galvan, and Zach, *An Introduction to Proof Theory*.
Further reading on natural deduction, sequent calculus, normalization, and cut elimination [17].

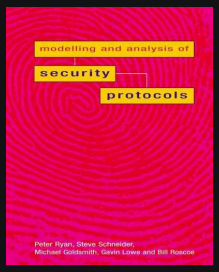
Pavlovic, *Programs as Diagrams*.
Related further reading; the current book does not rely on a specific claim from it [20].

Book-cover snapshots

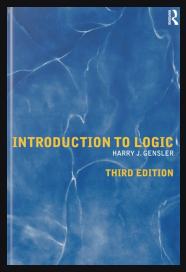
These cover snapshots identify the referenced editions.



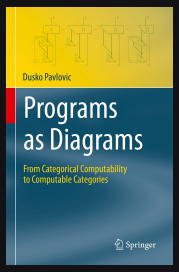
Logic and Language Models for Computer Science
Dana Richards and Henry Hamburger [21]



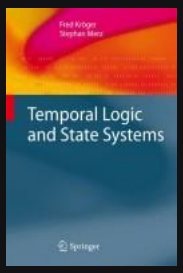
The Modelling and Analysis of Security Protocols
Peter Ryan, Steve Schneider, Michael Goldsmith, Gavin Lowe, and Bill Roscoe [22]



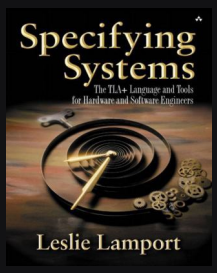
Introduction to Logic
Harry J. Gensler [8]



Programs as Diagrams
Dusko Pavlovic [20]



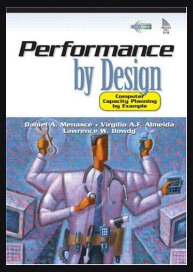
Temporal Logic and State Systems
Fred Kröger and Stephan Merz [13]



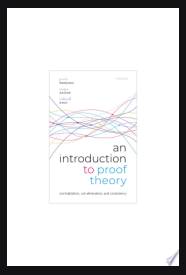
Specifying Systems
Leslie Lamport [15]



Guerrilla Capacity Planning
Neil J. Gunther [9]



Performance by Design
Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy [18]



An Introduction to Proof Theory
Paolo Mancosu, Sergio Galvan, and Richard Zach [17]

Cover-image sources, accessed September 25 and 30, 2026: World Scientific edition via Kobo for Richards and Hamburger; Open Library for Ryan et al.; Apple Books for Gensler; and Kobo's Springer edition for Pavlovic. The Kröger and Merz cover snapshot is from Open Library. The Lamport and Menascé–Almeida–Dowdy snapshots are from Open Library; the Gunther snapshot is from Springer Nature; and the Mancosu–Galvan–Zach snapshot is from Google Books. Cover artwork remains the property of its respective publisher.

Bibliography

[1] FizzBee: Performance modeling. `https://fizzbee.io/tutorials/performance-modeling`, 2026. Probabilities and counters are supplied in a separate YAML file passed to the performance checker.

[2] Adam D. Barwell, Alceste Scalas, Nobuko Yoshida, and Fangyi Zhou. Generalised multiparty session types with crash-stop failures. In *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 35:1–35:25, 2022. doi: 10.4230/LIPIcs.CONCUR.2022.35.

[3] Robert Braden. Requirements for internet hosts – application and support. RFC 1123, 1989. Section 4.2.3.1 describes and fixes the Sorcerer's Apprentice Syndrome.

[4] Daniel Brand and Pitro Zafiropulo. On communicating finite-state machines. *Journal of the ACM*, 30(2):323–342, 1983. doi: 10.1145/322374.322380.

[5] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.

[6] Pierre-Malo Deniélou and Nobuko Yoshida. Multiparty session types meet communicating automata. In *Programming Languages and Systems (ESOP 2012)*, volume 7211 of *Lecture Notes in Computer Science*, pages 194–213. Springer, 2012. doi: 10.1007/978-3-642-28869-2_10.

[7] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL `https://github.com/rfielding/leanFM`. Accessed 2026-09-21.

[8] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-4. doi: 10.4324/9781315693361.

[9] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, Berlin, Heidelberg, 2007. ISBN 978-3-540-31010-5. doi: 10.1007/978-3-540-31010-5.

[10] Kohei Honda. Types for dyadic interaction. In *CONCUR '93: 4th International Conference on Concurrency Theory*, volume 715 of *Lecture Notes in Computer Science*, pages 509–523. Springer, 1993. doi: 10.1007/3-540-57208-2_35.

[11] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *Programming Languages and Systems*

(ESOP 1998), volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi: 10.1007/BFb0053567.

[12] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2008)*, pages 273–284. ACM, 2008. doi: 10.1145/1328438.1328472.

[13] Fred Kröger and Stephan Merz. *Temporal Logic and State Systems*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2008. ISBN 978-3-540-68635-4. doi: 10.1007/978-3-540-68635-4.

[14] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Computer Aided Verification (CAV 2011)*, volume 6806 of *Lecture Notes in Computer Science*, pages 585–591. Springer, 2011. doi: 10.1007/978-3-642-22110-1_47.

[15] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.

[16] LeanFM. Kerberos: Time, knowledge, and obligation. YouTube video, 2026. Illustration of the proposed multimodal semantics, not an implemented general checker. `https://www.youtube.com/watch?v=f2efwxq8BaY`.

[17] Paolo Mancosu, Sergio Galvan, and Richard Zach. *An Introduction to Proof Theory: Normalization, Cut-Elimination, and Consistency Proofs*. Oxford University Press, Oxford, 2021. ISBN 978-0-19-289593-6. doi: 10.1093/oso/9780192895936.001.0001.

[18] Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy. *Performance by Design: Computer Capacity Planning by Example*. Prentice Hall PTR, Upper Saddle River, NJ, 2004. ISBN 978-0-13-090673-1.

[19] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP), 2021. doi: 10.1145/3473582.

[20] Dusko Pavlovic. *Programs as Diagrams: From Categorical Computability to Computable Categories*. Springer Nature Switzerland, Cham, Switzerland, 2023. ISBN 978-3-031-34826-6. doi: 10.1007/978-3-031-34827-3.

[21] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.

[22] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

[23] Karen R. Sollins. The TFTP protocol (revision 2). RFC 783, 1981.

[24] Karen R. Sollins. The TFTP protocol (revision 2). RFC 1350, 1992.

[25] Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *Trustworthy Global Computing (TGC 2013)*, volume 8358 of *Lecture Notes in Computer Science*, pages 22–41. Springer, 2014. doi: 10.1007/978-3-319-05119-2_3.

Index

agreement, 134
alphabet
  vague, 44
API boundary, 5

bytes, 131

channel declaration, 5
channel kind, 37
chart specification, 22
choice
  directed, 32
Codex, 59
communicating finite-state machines, 5
composition
  requirement intersection, 56
CSP, 130
CTL, 100
cut, 11

deadline, 137
device pairing, 136

event, 17

failure
  masked, 6
  surfaced, 6
fairness, 42
formal methods, 2

generated examples, 113
global type, 2, 31
grammar
  context-free, 31
  multiparty, 31

hiding, 10

HTTP
  call discipline, 11
  protobuf adapter, 146

interaction diagram
  collapsed, 10
interface, 6
  hierarchy, 6
internal choice, 12
intruder model, 134
island, 7

knowledge
  derived from events, 97

Lean, 122
local type, 2, 36
Lowe's attack, 139
LTL, 100

modal logic, 100
modality, 137
Model Context Protocol, 59
  logic consultation, 60
multiparty session type, 42

naming
  implicit, 7
Needham–Schroeder, 139

OAuth, 87
observable event, 44
on-call handoff, 131

parameter flow, 36
past-time logic, 106
physics of a channel, 5, 37, 40
probability, 44
probability overlay, 113, 137

measured, 26
profit
derived from events, 26
project
directory boundary, 70
projection, 2, 36, 130
protocol
LeanFM web server, 146
protocol role, 130
protocol syntax, 31

queue capacity, 5, 41

realizability, 12
requirements
durable, 57

scenario, 44
sealed-bid auction, 133
secrecy, 134
session identifier, 38
session type, 2
multiparty, 2
run of, 17
sharding, 6
Sorcerer's Apprentice, 40

temporal logic, 100
TFTP, 40
time, 21
topology, 7
trace, 17

UDP, 41

vibecoding, 57

waste
derived from events, 26

XY plot, 26